

NVIDIA AI Factory: A Kubernetes-Based MLOps and AI Infrastructure Platform

Abiyu Salam

Table of Content

Phase 0 — Introduction.....	3
Phase 1 — AI Infrastructure Foundation	21
Phase 2 — Kubernetes AI Platform.....	30
Phase 3 — AI Inference Platform (CPU Validation)	36
Phase 4 — AI Training & MLOps Pipeline	40
Phase 5 — AI Workload Scheduling.....	59
Phase 6 — AI Observability Platform	67
Phase 7 — Multi-Tenant AI Platform	77
Phase 8 — GitOps, Model Serving & Automated MLOps Operations	91
End-to-End Platform Validation.....	119

Phase 0 — Introduction

1. Project Overview

NVIDIA AI Factory merupakan platform AI infrastructure berbasis Kubernetes yang dibangun untuk menyediakan lingkungan terintegrasi bagi AI workload, model training, model serving, workload scheduling, MLOps, observability, GitOps, multi-tenancy, FinOps, dan automated ML operations. Platform ini dibangun di atas local GPU-enabled environment, sehingga seluruh lifecycle AI dapat dijalankan secara lokal tanpa bergantung pada managed Kubernetes atau cloud provider.

Secara konseptual, platform ini membentuk alur: Infrastructure → Kubernetes → GPU Workload → Training → Model Registry → Model Serving → Observability → Drift Detection → Automated Retraining → GitOps Operations

Tujuan akhirnya bukan sekadar menjalankan model machine learning, tetapi membangun sebuah AI platform yang mampu mengelola lifecycle workload dan model secara terstruktur.

2. Project Objectives

- Project ini memiliki beberapa tujuan utama:
- Membangun GPU-enabled container infrastructure menggunakan NVIDIA GPU, Docker, dan NVIDIA Container Toolkit.
- Membangun Kubernetes AI platform menggunakan Minikube.
- Menyediakan kemampuan GPU scheduling untuk AI workload.
- Menyediakan platform model inference menggunakan KServe dan Triton Inference Server.
- Membangun reproducible ML training pipeline menggunakan Argo Workflows dan DVC.
- Menyediakan experiment tracking dan model registry menggunakan MLflow.
- Menyediakan object storage untuk dataset dan model artifacts menggunakan MinIO.
- Mengimplementasikan workload queueing dan resource management menggunakan Kueue.
- Membangun observability stack untuk Kubernetes, workload, GPU, metrics, dan logs.
- Menerapkan multi-tenancy dan RBAC agar beberapa persona pengguna dapat menggunakan platform secara terisolasi.
- Menerapkan GitOps deployment management menggunakan Argo CD.
- Membangun data drift monitoring dan automated retraining pipeline.
- Menyediakan backup dan disaster recovery menggunakan Velero.
- Menyediakan resource cost visibility menggunakan OpenCost.

3. Target Platform

Platform dirancang sebagai local AI infrastructure environment dengan karakteristik:

- Host OS: Windows
- Linux Environment: WSL2 / Ubuntu
- GPU: NVIDIA GPU
- Container Runtime: Docker Engine
- GPU Container Runtime: NVIDIA Container Toolkit
- Kubernetes: Minikube
- Kubernetes CLI: kubectl
- Package Management: Helm
- Service Mesh: Istio
- Model Serving: KServe + Triton Inference Server
- Workflow Orchestration: Argo Workflows
- GitOps: Argo CD
- Event Automation: Argo Events
- Dataset Versioning: DVC
- Experiment Tracking / Model Registry: MLflow
- Metadata Database: PostgreSQL
- Object Storage: MinIO
- GPU Scheduling: NVIDIA Device Plugin + Kueue
- Metrics: Prometheus
- GPU Metrics: NVIDIA DCGM Exporter
- Logs: Grafana Alloy + Loki
- Visualization: Grafana
- Alerting: Alertmanager
- Drift Monitoring: Evidently AI
- Backup / Disaster Recovery: Velero
- Cost Monitoring: OpenCost

4. Initial Environment Preparation

Sebelum membangun Kubernetes dan komponen AI platform, environment Linux disiapkan terlebih dahulu agar seluruh operasi selanjutnya dapat dilakukan menggunakan user non-root. User abiyu dibuat sebagai user utama yang digunakan untuk menjalankan aktivitas platform. User tersebut kemudian diberikan akses sudo untuk administrative operations dan dimasukkan ke group docker agar dapat berinteraksi dengan Docker Engine tanpa harus menjalankan seluruh command menggunakan sudo. Setelah perubahan group diterapkan, session kemudian berpindah ke user abiyu.

```
adduser abiyu
```

```
# Masukkan ke sudo
```

```
usermod -aG sudo abiyu
```

```
# Masukkan ke group docker
```

```
usermod -aG docker abiyu
```

su - abiyu

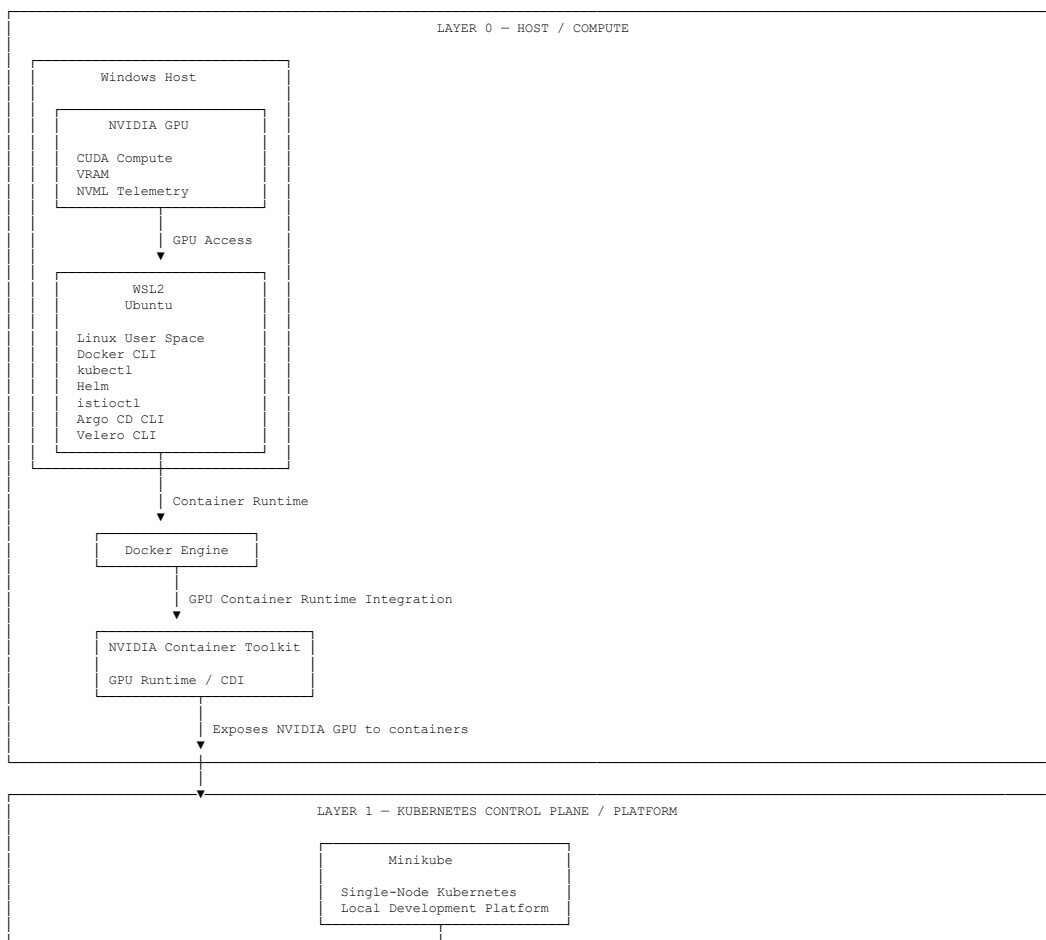
Dengan konfigurasi ini, root digunakan hanya ketika diperlukan untuk administrative operations, sedangkan aktivitas pembangunan dan pengoperasian AI platform dilakukan menggunakan user abiyu.

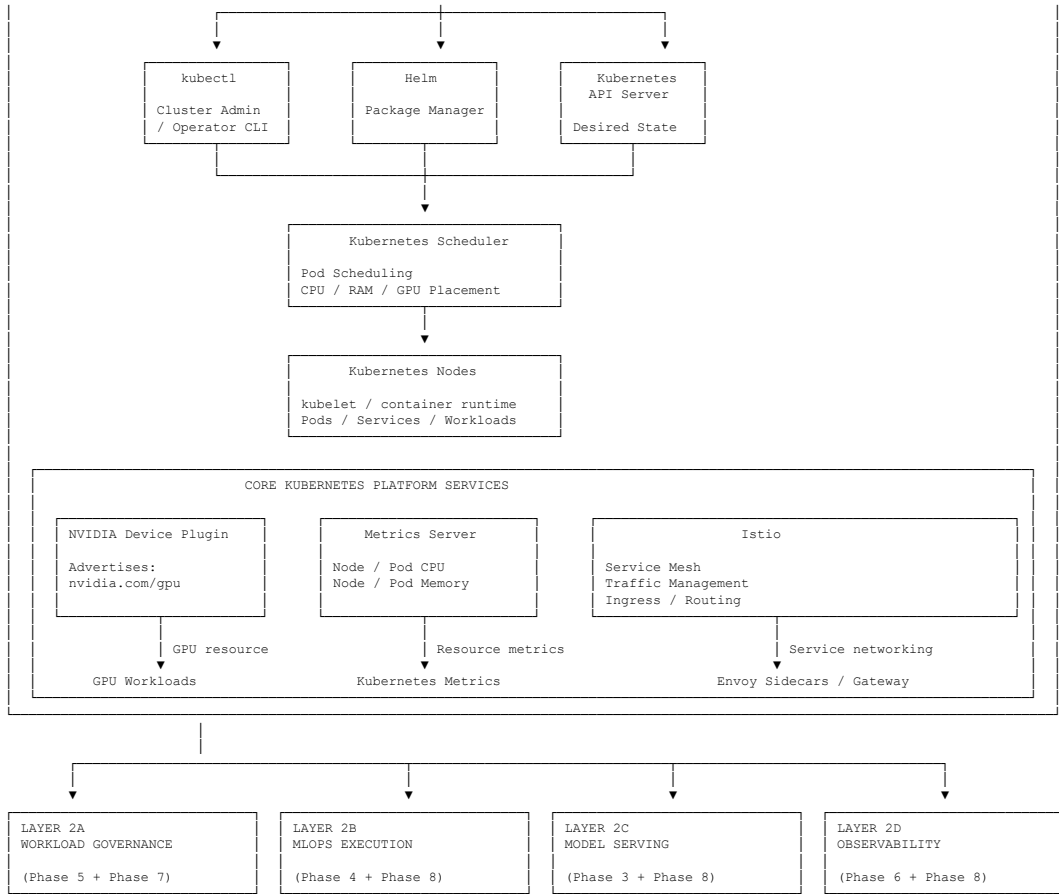
5. Overall System Architecture

Arsitektur keseluruhan NVIDIA AI Factory dibagi menjadi Layer 0 sampai Layer 14. Pembagian layer digunakan untuk menunjukkan posisi setiap komponen berdasarkan tanggung jawabnya di dalam platform.

Layer tersebut membentuk dependency dari infrastructure paling bawah sampai automation paling atas: Host / Hardware → Kubernetes Control Plane / Platform → Multi-Tenancy / Rbac / Resource Governance / Gpu Scheduling → Gitops Control Plane → Data / Code / Model Versioning → Training / Mlops Pipeline → Model Serving → Model Lifecycle → Inference Observability / Drift → Automated Retraining Event Flow → Observability Platform → Alerting → Disaster Recovery → Finops / Cost Visibility → Core Storage Architecture

5.1 Master System Architecture

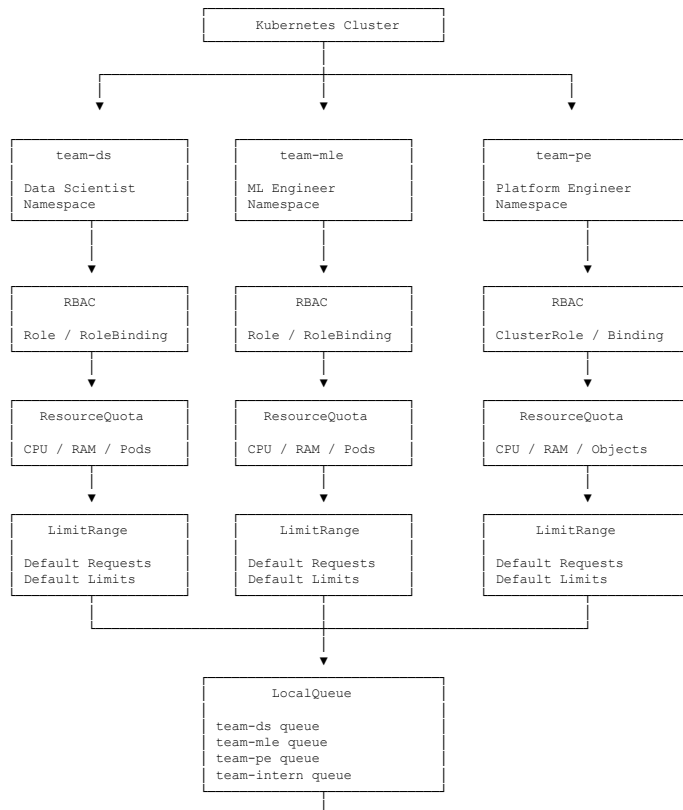


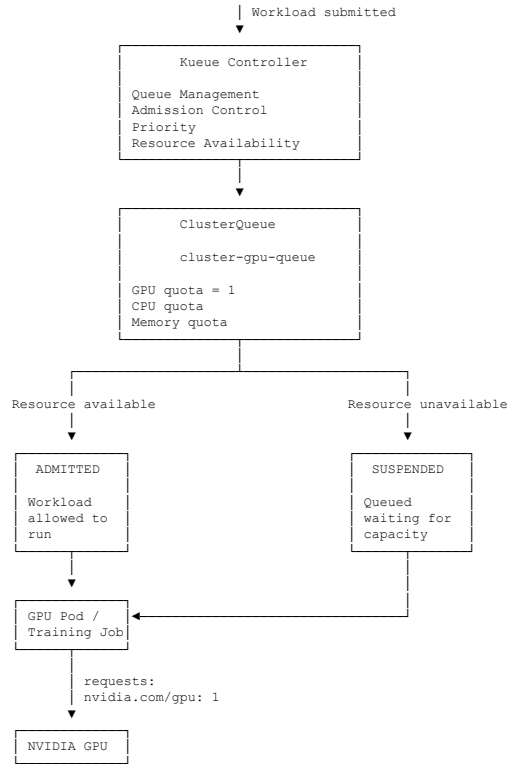


=====

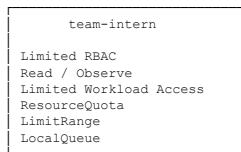
LAYER 2A - MULTI-TENANCY / RBAC / RESOURCE GOVERNANCE / GPU SCHEDULING

=====





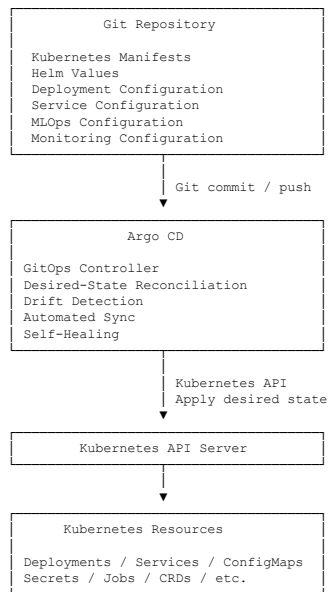
Additional tenant:



=====

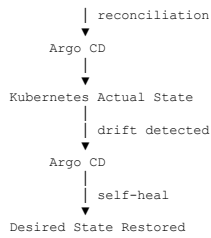
LAYER 3 – GITOPS CONTROL PLANE

=====

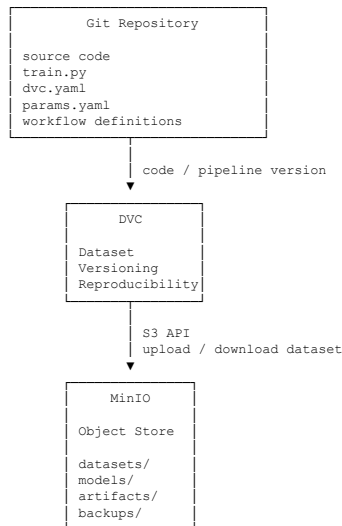


GitOps principle:

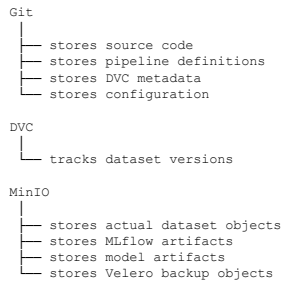
Git Desired State
|



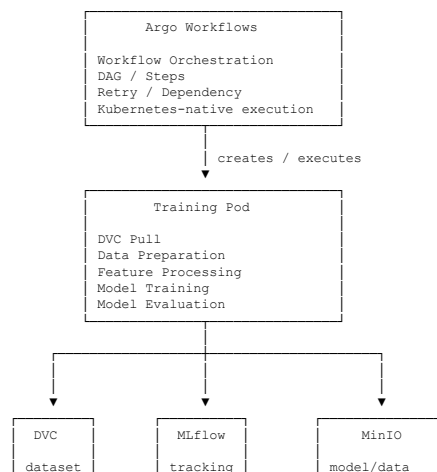
LAYER 4 - DATA / CODE / MODEL VERSIONING

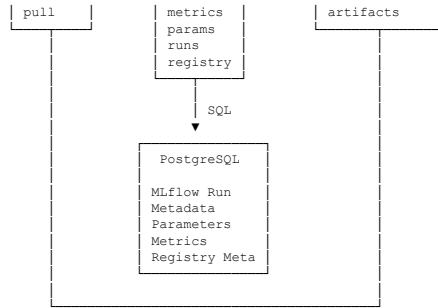


Important distinction:

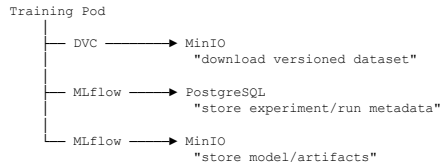


LAYER 5 - TRAINING / MLOPS PIPELINE

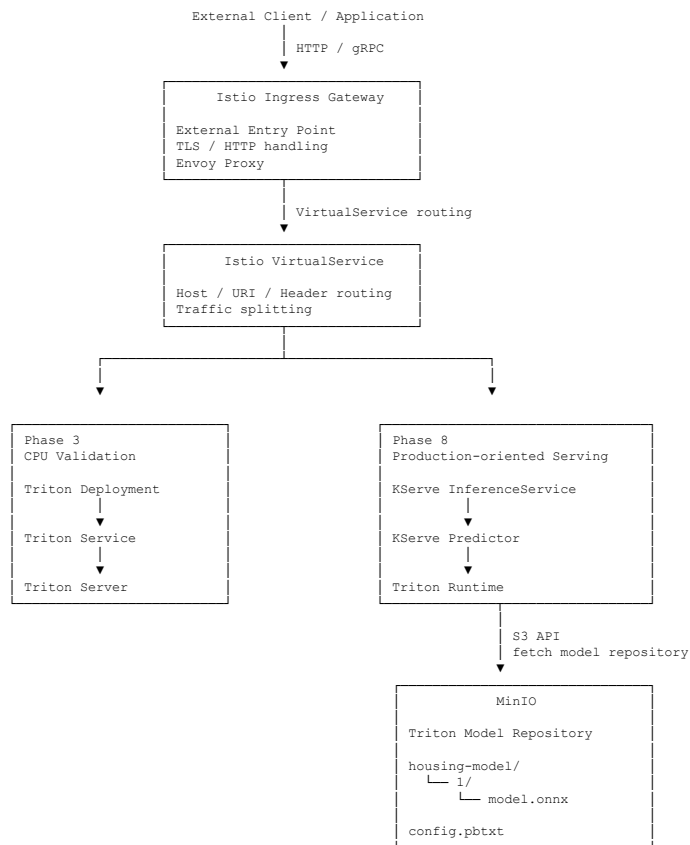




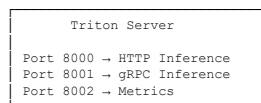
Training communication:



LAYER 6 - MODEL SERVING

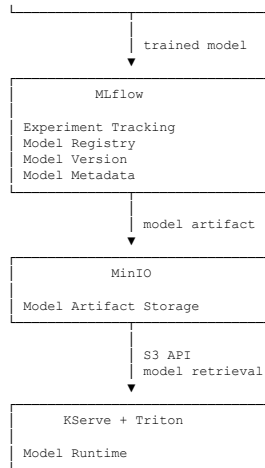


Triton interface:



LAYER 7 - MODEL LIFECYCLE

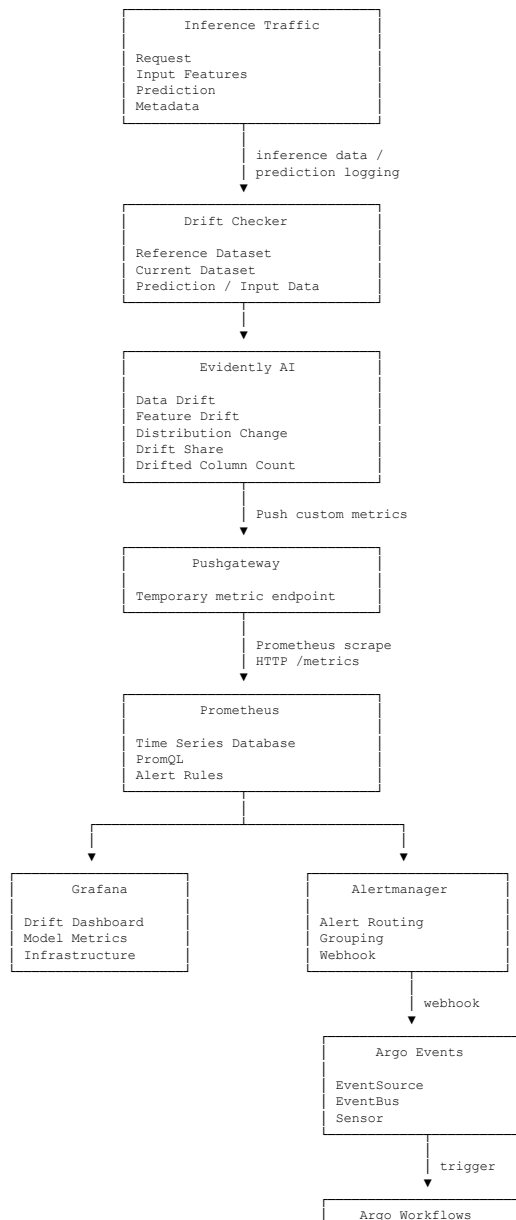
Training Pipeline

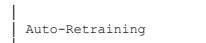


=====

LAYER 8 – INFERENCE OBSERVABILITY / DRIFT

=====

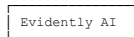
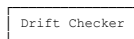




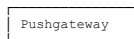
new training run
MLflow

LAYER 9 - AUTOMATED RETRAINING EVENT FLOW

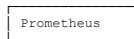
CURRENT DATA



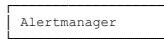
metric:
model_data_drift_share



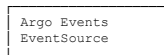
scrape



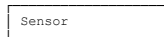
PromQL alert rule
model_data_drift_share > threshold



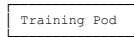
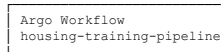
HTTP webhook
alertname=DataDriftDetected



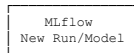
event dependency



submit Workflow



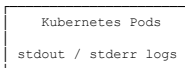
new model



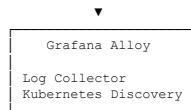
NOTE:
This is an AUTOMATED RETRAINING LOOP.
It becomes a true CLOSED-LOOP MODEL DEPLOYMENT only after:

MLflow
↓
Model Evaluation
↓
Champion vs Challenger
↓
Promotion
↓
KServe Model Update
↓
Production Rollout
↓
Monitoring

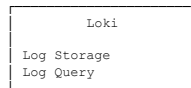
LAYER 10 - OBSERVABILITY PLATFORM



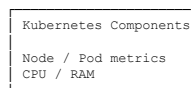
log collection



push logs



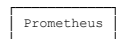
query



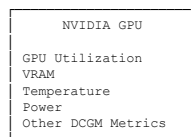
metrics



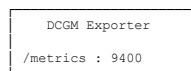
resource metrics



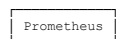
PromQL



NVML / DCGM



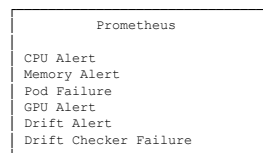
Prometheus scrape



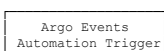
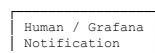
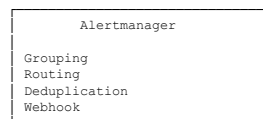
=====

LAYER 11 - ALERTING

=====



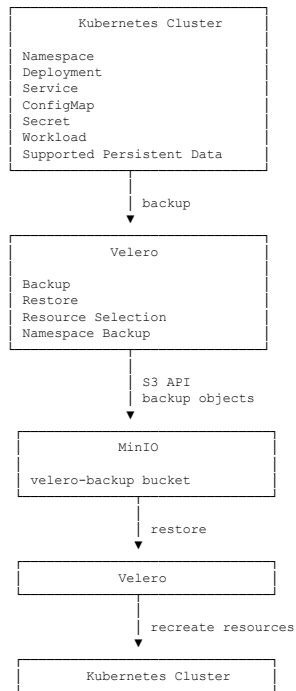
alert rule fired



=====

LAYER 12 - DISASTER RECOVERY

=====



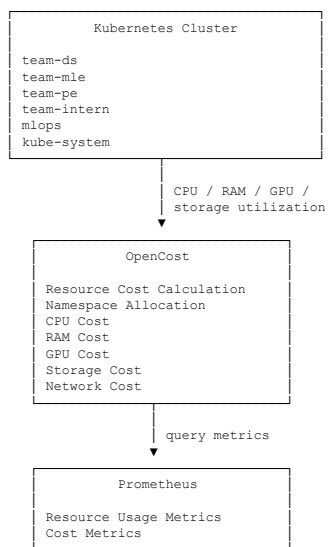
DR validation flow:

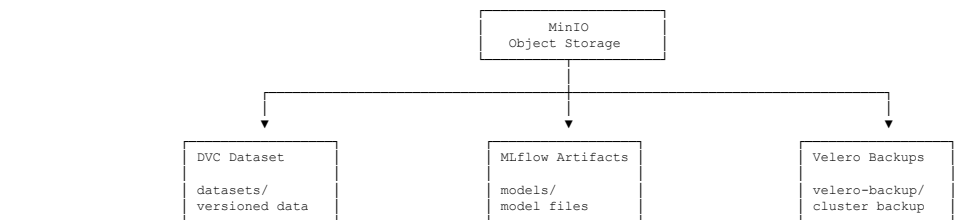
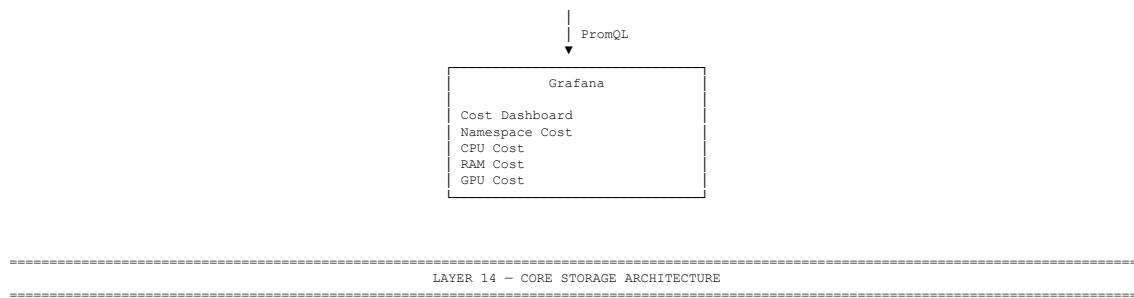
```
Backup
↓
MinIO
↓
Simulated Disaster
↓
Namespace Deleted
↓
Velero Restore
↓
Namespace Recreated
↓
Pod Recreated
↓
Application Validated
```

=====

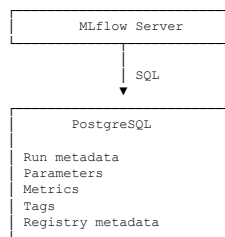
LAYER 13 - FINOPS / COST VISIBILITY

=====

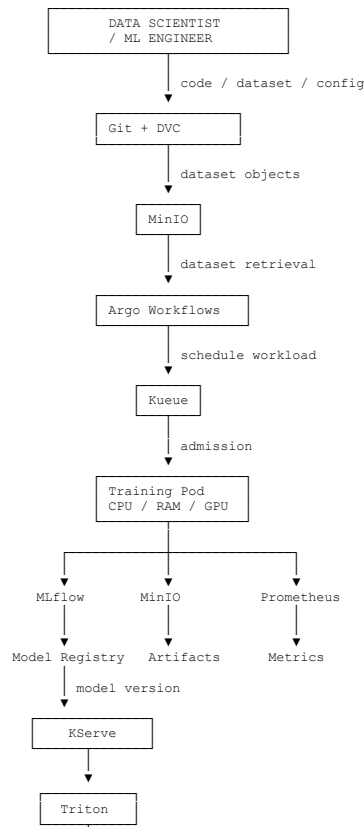


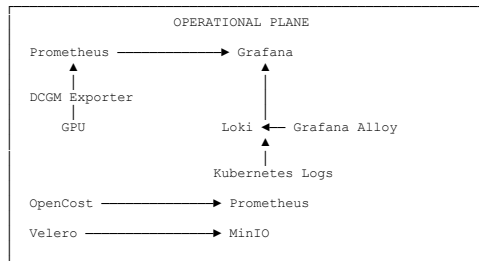
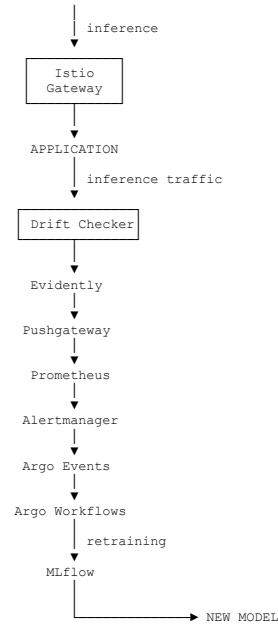


Additional MLflow metadata storage:



===== END-TO-END AI FACTORY FLOW =====





6. Communication Legend

Karena platform terdiri dari banyak service yang saling berkomunikasi, setiap koneksi pada architecture diagram diberi label berdasarkan mekanisme komunikasi dan jenis data yang dipertukarkan.

6.1 Communication Legend

Connection	Fungsi komunikasi
kubectl → Kubernetes API	Mengirim command / membaca / mengubah resource Kubernetes
Helm → Kubernetes API	Install / upgrade / manage package Kubernetes
Argo CD → Git	Membaca desired state dari repository
Argo CD → Kubernetes API	Reconcile dan apply desired state
DVC → MinIO	Upload / download dataset object melalui S3 API
Argo Workflow → K8s API	Membuat dan menjalankan workflow / workload
Training Pod → MinIO	Download dataset / upload artifact
Training Pod → MLflow	Mengirim parameters, metrics, tags, run information
MLflow → PostgreSQL	Menyimpan metadata experiment dan model registry
MLflow → MinIO	Menyimpan / membaca model artifact dan artifact files
KServe → Kubernetes API	Mengelola lifecycle InferenceService
KServe → MinIO	Mengambil model artifact / model repository
KServe → Triton	Menjalankan model inference runtime
Client → Istio Gateway	Mengirim HTTP/gRPC inference request
Istio → KServe/Triton	Routing traffic menuju model serving workload
Triton → Client	Mengembalikan prediction response
Inference → Drift Checker	Menyediakan input/prediction data untuk monitoring
Drift Checker → Evidently	Menjalankan statistical drift analysis
Evidently → Pushgateway	Push custom drift metrics
Prometheus → Pushgateway	Scrape `/metrics` endpoint
Prometheus → DCGM Exporter	Scrape GPU telemetry metrics
Prometheus → Kubernetes	Scrape application/platform metrics
Grafana → Prometheus	Query metrics menggunakan PromQL
Grafana → Loki	Query logs menggunakan LogQL
Alloy → Loki	Push container/Kubernetes logs
Prometheus → Alertmanager	Mengirim alert yang memenuhi alert rule
Alertmanager → Argo Events	Mengirim webhook ketika kondisi automation terpenuhi
Argo Events → Argo Workflow	Trigger automated retraining workflow
Velero → MinIO	Upload Kubernetes backup object melalui S3 API
Velero → MinIO	Mengambil backup object untuk proses restore

OpenCost → Prometheus	Membaca resource metrics untuk cost calculation
OpenCost → Grafana	Cost data divisualisasikan melalui Prometheus/Grafana
Kueue → Workload	Admission / suspension berdasarkan resource availability dan queue policy
Device Plugin → Kubernetes	Mendaftarkan GPU sebagai allocatable resource 'nvidia.com/gpu'

7. Architectural Responsibility Map

Setiap component dalam platform memiliki tanggung jawab yang berbeda. Responsibility map digunakan untuk memastikan tidak terjadi overlap fungsi antar-component.

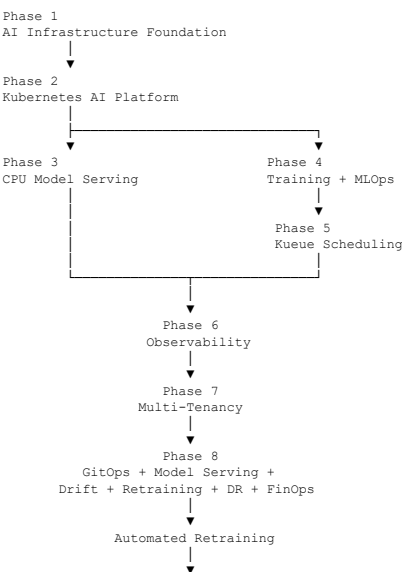
7.1 Architectural Responsibility Map

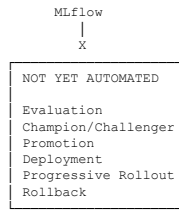
Component	Tanggung jawab utama
WSL2 / Ubuntu	Linux execution environment
Docker	Container runtime
NVIDIA Container Toolkit	GPU enablement untuk container
Minikube	Local Kubernetes cluster
kubect1	Kubernetes CLI
Helm	Kubernetes package management
NVIDIA Device Plugin	Expose GPU ke Kubernetes
Metrics Server	CPU/RAM resource metrics
Istio	Service mesh, ingress, traffic routing
Kueue	Workload queueing, admission, quota, scheduling
RBAC	Identity dan authorization
ResourceQuota	Namespace resource/object limits
LimitRange	Default/min/max resource constraints
Git	Source/configuration version control
DVC	Dataset versioning dan reproducibility
MinIO	Object storage
Argo Workflows	Training/retraining workflow orchestration
MLflow	Experiment tracking + model registry
PostgreSQL	MLflow metadata database
Triton	Model inference runtime
KServe	Model serving abstraction/lifecycle
Evidently AI	Data/model drift analysis
Pushgateway	Push-based Prometheus metric gateway
Prometheus	Metrics storage, PromQL, alert evaluation
Alertmanager	Alert routing dan notification
Argo Events	Event-driven automation
Grafana Alloy	Log collection
Loki	Log storage/query
Grafana	Metrics/log/cost visualization
DCGM Exporter	NVIDIA GPU telemetry exporter
Velero	Kubernetes resource backup/restore
OpenCost	Resource cost estimation/allocation

8. Phase Dependency Map

Phase 1–8 tidak berdiri sendiri. Setiap phase menggunakan capability yang dibangun oleh phase sebelumnya.

8.1 Phase Dependency Map

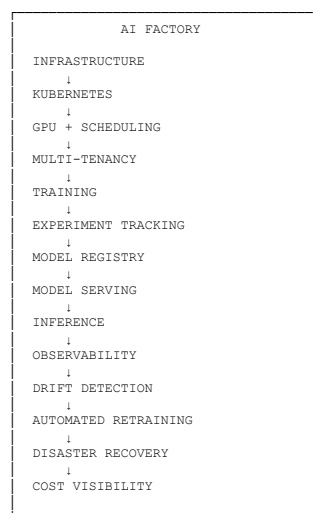




9. Platform Boundary

NVIDIA AI Factory memiliki batas yang jelas antara komponen yang menjadi bagian dari platform dan komponen eksternal yang hanya berinteraksi dengan platform.

9.1 Final Platform Boundary



10. Project Architecture Principles

Arsitektur NVIDIA AI Factory dibangun berdasarkan beberapa prinsip utama:

10.1 Layered Architecture

Setiap layer memiliki tanggung jawab spesifik dan bergantung pada layer di bawahnya. Dengan demikian, perubahan pada satu layer dapat dikelola tanpa mengubah seluruh platform.

10.2 Kubernetes as the Execution Platform

Kubernetes menjadi execution layer utama bagi workload AI, training pipeline, model serving, monitoring, dan automation components.

10.3 Declarative Infrastructure

Resource Kubernetes dan konfigurasi platform didefinisikan secara deklaratif sehingga environment dapat direproduksi dan dikelola menggunakan GitOps.

10.4 Separation of Responsibilities

Setiap tool digunakan berdasarkan fungsi utamanya. Contohnya: Kueue mengatur admission workload, bukan menggantikan Kubernetes scheduler. MLflow mengelola experiment/model metadata, sedangkan MinIO menyimpan object artifacts dan PostgreSQL menyimpan metadata SQL. KServe menyediakan model-serving abstraction, sedangkan Triton menjalankan inference runtime.

10.5 Observable by Design

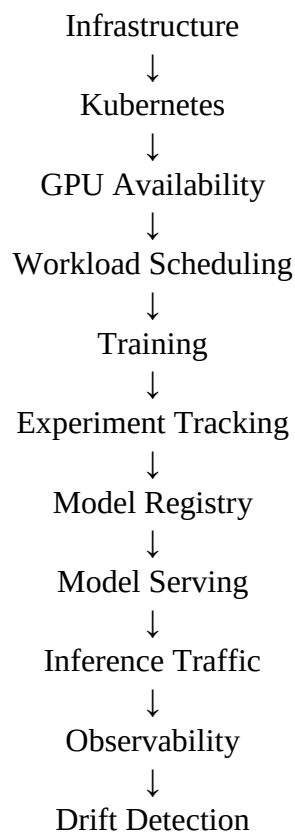
Workload tidak hanya dijalankan, tetapi juga menghasilkan metrics, logs, dan GPU telemetry yang dapat digunakan untuk monitoring dan troubleshooting.

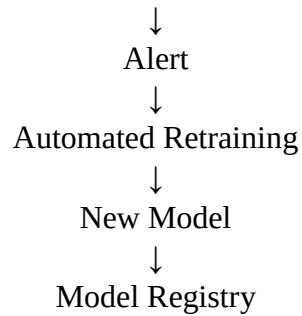
10.6 Automated ML Lifecycle

Training, model registration, deployment, monitoring, drift detection, dan retraining dihubungkan menjadi automation pipeline. Namun, pada baseline architecture ini, automated retraining belum berarti fully autonomous model deployment. Model hasil retraining masih membutuhkan tahap validation → promotion → deployment sebelum dapat menggantikan production model.

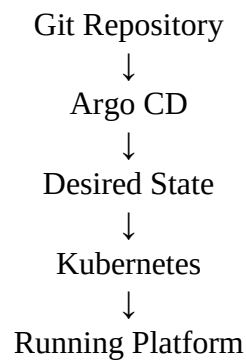
11. Platform Lifecycle

Secara keseluruhan, lifecycle platform dapat diringkas menjadi:





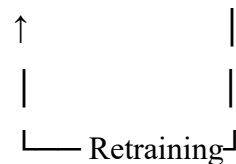
Sedangkan deployment lifecycle berjalan melalui:



Dengan demikian, NVIDIA AI Factory memiliki dua automation lifecycle utama:

Platform lifecycle : Git → Argo CD → Kubernetes → Running Infrastructure

ML lifecycle: Data → Training → MLflow → Serving → Monitoring



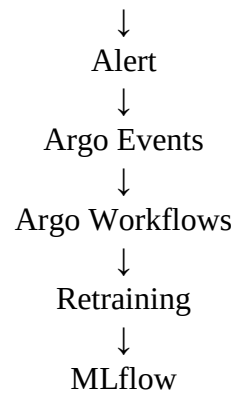
Kedua lifecycle tersebut kemudian bertemu pada Kubernetes sebagai execution platform utama.

12. Scope and Current Limitation

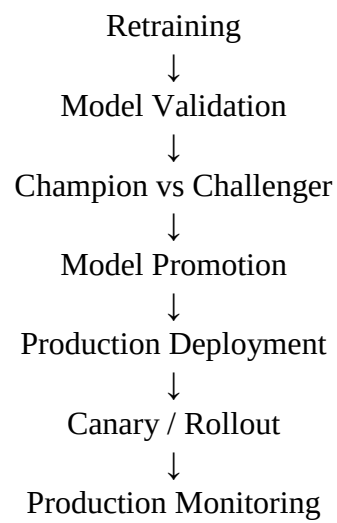
Platform ini mencakup infrastructure, Kubernetes, GPU management, workload scheduling, MLOps, model serving, observability, GitOps, multi-tenancy, automated retraining, backup, dan cost visibility. Namun, closed-loop model deployment belum sepenuhnya diimplementasikan.

Alur saat ini berhenti pada:

Drift



Belum menjadi:

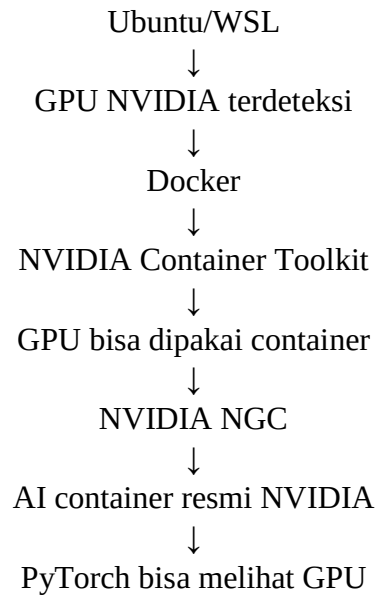


Dengan demikian, istilah yang paling tepat untuk kondisi akhir platform adalah automated retraining loop, bukan fully closed-loop autonomous MLOps.

Phase 1 — AI Infrastructure Foundation

Pada phase ini kita membangun fondasi paling bawah dari AI Factory. Ibaratnya kita mau bikin rumah: sebelum bikin Kubernetes, MLOps pipeline, scheduling GPU, atau observability, kita harus memastikan mesin tempat semuanya berjalan sudah siap dan GPU bisa dipakai oleh workload AI.

Alur besarnya:



Jadi output utama Phase 1 bukan model AI dulu, melainkan AI-capable container infrastructure yang nantinya menjadi fondasi Phase 2, yaitu Kubernetes AI Platform.

Step 0. Setup OS

Step ini bertujuan menyiapkan Ubuntu dengan package dan tools dasar yang akan dipakai sepanjang project. Kenapa harus dilakukan paling awal?

Karena step-step berikutnya membutuhkan berbagai command seperti curl, wget, git, unzip, dan tools sistem lainnya. Kalau environment dasarnya belum siap, nanti error yang muncul bisa bukan karena Docker atau NVIDIA, tetapi cuma karena dependency OS belum tersedia.

0.1 Update Ubuntu

apt update memperbarui package index dari repository yang dikonfigurasi pada sistem, sedangkan apt upgrade memperbarui package yang telah terinstal ke versi yang tersedia. Langkah ini dilakukan untuk memastikan environment berada pada kondisi yang sesuai sebelum instalasi Docker dan NVIDIA Container Toolkit.

```
sudo apt update
```

```
sudo apt upgrade -y
```

0.2 Install package dasar

Package dasar ini menyediakan utility yang digunakan sepanjang deployment, khususnya curl dan gnupg untuk konfigurasi repository, wget dan unzip untuk distribusi software, serta git untuk source-code management.

```
sudo apt install -y \  
build-essential \  
curl \  
wget \  
git \  
vim \  
unzip \  
ca-certificates \  
gnupg \  
lsb-release \  
software-properties-common
```

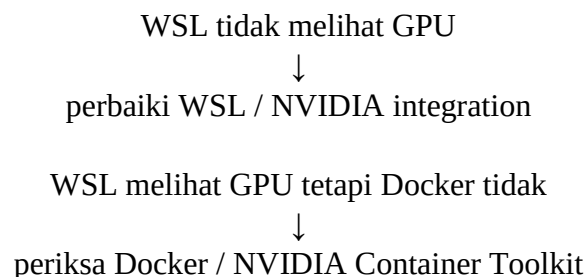
0.3 Verifikasi

Package dasar ini menyediakan utility yang digunakan sepanjang deployment, khususnya curl dan gnupg untuk konfigurasi repository, wget dan unzip untuk distribusi software, serta git untuk source-code management.

```
git --version  
curl --version
```

Step 1. GPU WSL

nvidia-smi digunakan untuk menampilkan informasi GPU NVIDIA yang tersedia, termasuk GPU model, driver, dan penggunaan GPU. Validasi ini dilakukan sebelum konfigurasi Docker karena Docker GPU passthrough bergantung pada integrasi GPU yang sudah tersedia pada WSL. Dengan demikian, troubleshooting dapat dilakukan secara berlapis:



Dokumen menggunakan nvidia-smi sebagai verifikasi awal bahwa GPU telah muncul dari WSL.

Pastikan GPU muncul dari WSL.

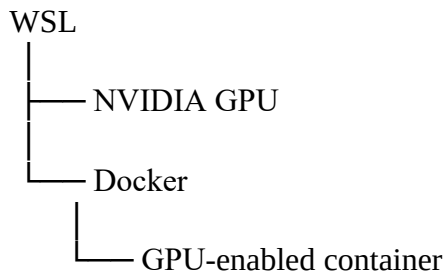
```
nvidia-smi
```

Step 2. Docker

Docker digunakan sebagai container runtime untuk menjalankan workload AI.

Pada project ini, Docker menjadi layer yang menghubungkan environment host dengan container AI. Nantinya NVIDIA Container Toolkit akan dikonfigurasi di atas Docker sehingga container dapat mengakses GPU.

Alur yang ingin dicapai adalah:



2.1 Pastikan OS, Command ini menampilkan informasi distribusi Linux yang sedang digunakan. Verifikasi ini memastikan repository dan package Docker yang digunakan sesuai dengan environment Ubuntu yang tersedia.

```
cat /etc/os-release
```

#2.2 Update package, Package index dan package sistem diperbarui sebelum instalasi Docker. Catatan: bagian ini secara fungsi sebenarnya sudah dilakukan pada Step 0. Pengulangan tidak bersifat wajib apabila Step 0 telah selesai dengan sukses.

```
sudo apt update
```

```
sudo apt upgrade -y
```

2.3 Install dependency, Dependency ini diperlukan untuk menambahkan dan memverifikasi repository Docker. ca-certificates dan curl digunakan untuk akses HTTPS, sedangkan gnupg digunakan untuk menangani signing key repository.

```
sudo apt install -y \
```

```
ca-certificates \
```

```
curl \
```

```
gnupg \
```

```
lsb-release
```

2.4 Hapus Docker lama (opsional), Package Docker/container runtime dari repository Ubuntu dapat dihapus terlebih dahulu untuk menghindari konflik dengan package Docker resmi. Langkah ini bersifat optional karena sistem mungkin belum memiliki package tersebut.

```
sudo apt remove -y docker.io containerd runc
```

2.5 Buat keyring. Direktori /etc/apt/keyrings dibuat untuk menyimpan cryptographic key yang digunakan APT ketika memverifikasi repository.

```
sudo install -m 0755 -d /etc/apt/keyrings
```

2.6 Download GPG, GPG key resmi Docker diambil dan disimpan dalam format keyring yang dapat digunakan APT. Tujuannya adalah memungkinkan APT memverifikasi package yang berasal dari repository Docker.

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg \  
| sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

2.7 Permission. Permission read diberikan agar keyring dapat dibaca oleh proses APT.

```
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

2.8 Tambah repository Docker, Repository resmi Docker ditambahkan ke konfigurasi APT. \$(dpkg --print-architecture) menentukan architecture sistem, sedangkan \$VERSION_CODENAME mengambil codename Ubuntu yang sedang digunakan. Parameter: signed-by=/etc/apt/keyrings/docker.gpg menentukan key yang digunakan untuk memverifikasi repository tersebut.

```
echo \  

```

```
"deb [arch=$(dpkg --print-architecture) signed-  
by=/etc/apt/keyrings/docker.gpg] \  

```

```
https://download.docker.com/linux/ubuntu \  

```

```
$(. /etc/os-release && echo $VERSION_CODENAME) stable" \  

```

```
| sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

2.9 Update repository, harusnya muncul repository Docker. APT membaca repository Docker yang baru ditambahkan dan memperbarui package index. Pada tahap ini repository Docker seharusnya sudah muncul dalam proses update.

```
sudo apt update
```

2.10 Install Docker, Componen Docker yang dipasang terdiri dari: docker-ce – Docker Engine docker-ce-cli – command-line interface Docker. containerd.io – container runtime. docker-buildx-plugin – image build functionality. docker-compose-plugin – Docker Compose integration.

```
sudo apt install -y \  

```

```
docker-ce \  

```

```
docker-ce-cli \  

```

```
containerd.io \  

```

```
docker-buildx-plugin \  

```

```
docker-compose-plugin
```

2.11 Cek, harus muncul kira-kira "Docker version 29.x.x". Command ini memastikan Docker CLI telah terinstal dan dapat dipanggil. Output akan menampilkan versi Docker yang terpasang.


```
docker --version
```

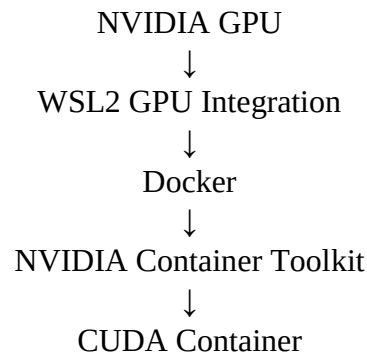
2.12 Tes, hello-world digunakan sebagai smoke test untuk Docker. Test ini memvalidasi bahwa Docker dapat: Mengakses image. Membuat container. Menjalankan container. Menampilkan output. Menghentikan container dengan normal. Jika Hello from Docker muncul, basic Docker runtime telah berfungsi. Seluruh konfigurasi Step 2 terdapat pada project source.

```
docker run hello-world
```

Step 3. Install NVIDIA Container Toolkit

NVIDIA Container Toolkit digunakan untuk mengintegrasikan NVIDIA GPU dengan container runtime Docker. Docker secara default dapat menjalankan container, tetapi agar container dapat menggunakan GPU NVIDIA diperlukan integration layer tambahan.

Arsitektur yang ingin dicapai:



3.1 Tambahkan repository, GPG key dan repository NVIDIA Container Toolkit ditambahkan ke APT.Sama seperti repository Docker, penggunaan signing key memungkinkan package manager memverifikasi sumber package

```
curl -fsSL https://nvidia.github.io/libnvidia-container/gpgkey \
| sudo gpg --dearmor -o /usr/share/keyrings/nvidia-container-toolkit-
keyring.gpg
```

```
curl -fsSL https://nvidia.github.io/libnvidia-
container/stable/deb/nvidia-container-toolkit.list \
| sed 's#deb https://#deb [signed-by=/usr/share/keyrings/nvidia-
container-toolkit-keyring.gpg] https://#g' \
| sudo tee /etc/apt/sources.list.d/nvidia-container-toolkit.list
```

3.2 Install, APT diperbarui agar mengenali repository NVIDIA yang baru ditambahkan, kemudian NVIDIA Container Toolkit dipasang.

```
sudo apt update
```

```
sudo apt install -y nvidia-container-toolkit
```

3.3 Konfigurasi runtime, nvidia-ctk digunakan untuk mengkonfigurasi integrasi NVIDIA Container Toolkit dengan Docker runtime. Konfigurasi

ini memungkinkan Docker meneruskan resource GPU kepada container yang meminta GPU.

```
sudo nvidia-ctk runtime configure --runtime=docker
```

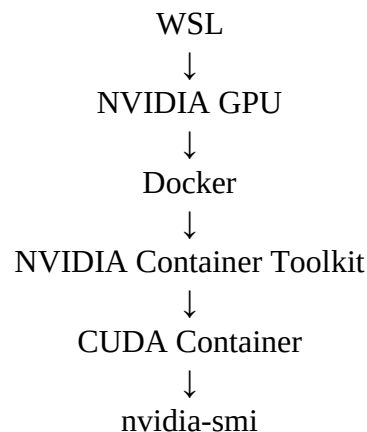
3.4 Restart Docker, Docker di-restart agar konfigurasi runtime yang baru dapat diterapkan. Keseluruhan konfigurasi NVIDIA Container Toolkit pada project berada pada Step 3

```
sudo service docker restart
```

Step 4. Tes GPU di Docker

Step ini melakukan validasi bahwa GPU dapat diakses dari dalam container, bukan hanya dari WSL. Parameter `--gpus all` memberikan akses terhadap seluruh GPU yang tersedia kepada container. Image `nvidia/cuda:12.9.1-base-ubuntu24.04` digunakan sebagai environment CUDA untuk pengujian. `nvidia-smi` kemudian dijalankan di dalam container.

Jika berhasil, berarti integrasi berikut telah berjalan:



Ini merupakan validasi yang lebih tinggi dibanding Step 1 karena GPU sekarang diuji dari container boundary. Project mendefinisikan keberhasilan step ini sebagai terhubungnya Docker, Toolkit, driver, dan GPU.

Kalau berhasil Berarti Docker → Toolkit → Driver → GPU sudah nyambung.

```
docker run --rm \
--gpus all \
nvidia/cuda:12.9.1-base-ubuntu24.04 \
nvidia-smi
```

Step 5. Install NGC CLI

NVIDIA NGC digunakan sebagai sumber container dan resource AI NVIDIA. NGC CLI menyediakan interface command-line untuk berinteraksi dengan NVIDIA NGC. Pada

step ini, NGC CLI digunakan untuk autentikasi dan validasi akses terhadap NVIDIA Container Registry sebelum image PyTorch NVIDIA ditarik.

5.1 Download, NGC CLI untuk Linux di-download dalam format ZIP.

```
wget https://ngc.nvidia.com/downloads/ngccli_linux.zip
```

5.2 Ekstrak

```
unzip ngccli_linux.zip
```

Hasilnya akan menjadi:

```
ngc-cli/
```

```
|— ngc
```

```
|— _internal/
```

Folder `_internal` dipertahankan karena merupakan bagian dari distribusi NGC CLI.

5.3 Install Seluruh folder NGC CLI dipindahkan ke `/opt/ngc-cli`.

Penggunaan `/opt` memberikan lokasi yang terpisah dari package manager sistem untuk software tambahan seperti NGC CLI.

```
sudo mv ngc-cli /opt/ngc-cli
```

5.4 Lalu buat symbolic link agar perintah “ngc” bisa dipanggil dari mana saja.

```
sudo ln -s /opt/ngc-cli/ngc /usr/local/bin/ngc
```

5.5 Verifikasi, Verifikasi memastikan NGC CLI telah terinstal dan executable dapat ditemukan melalui `$PATH`. Project kemudian menggunakan CLI tersebut pada Step 6 untuk konfigurasi authentication

```
ngc -version
```

Step 6. Login NGC

Step ini melakukan autentikasi NGC CLI menggunakan NVIDIA NGC API Key. NGC CLI yang telah dipasang pada Step 5 belum memiliki credential untuk mengakses resource yang membutuhkan authentication. Karena itu, Personal API Key dibuat terlebih dahulu melalui NVIDIA NGC.

6.1 login di <https://ngc.nvidia.com>

6.2 Klik profil (pojok kanan atas).

6.3 Pilih “Account Settings”

6.4 Keys & Secrets > API Keys > Generate API Key > Generate Personal Key

6.5 Isi Key Name, Expiration, Key Permissions, lalu “Generate Personal Key”

6.6 keluar “nvapi-***” dan Simpan key tersebut**

6.7 Login

```
ngc config set
```

Isi:

- API Key (nvapi-*****)
- Output format (ascii)
- Organization (default jika hanya punya satu)
- Team (default)

6.8 Tes, Step ini melakukan autentikasi NGC CLI menggunakan NVIDIA NGC API Key.NGC CLI yang telah dipasang pada Step 5 belum memiliki credential untuk mengakses resource yang membutuhkan authentication. Karena itu, Personal API Key dibuat terlebih dahulu melalui NVIDIA NGC.

```
ngc registry image list
```

Step 7. Pull container AI resmi

Step ini mengambil NVIDIA PyTorch container dari NVIDIA Container Registry (NGC). Container tersebut menyediakan environment yang telah dipersiapkan untuk menjalankan workload PyTorch dengan stack NVIDIA. Pada tahap ini project berpindah dari pengujian infrastructure: Docker + CUDA

menjadi environment workload: Docker + NVIDIA CUDA + PyTorch

Image yang digunakan adalah: nvcr.io/nvidia/pytorch:25.06-py3

Image kemudian digunakan pada Step 8 untuk melakukan validasi PyTorch terhadap GPU.

```
docker pull nvcr.io/nvidia/pytorch:25.06-py3
```

Step 8. Verifikasi

Step ini merupakan end-to-end validation dari seluruh konfigurasi GPU container yang telah dibangun pada Phase 1.

Sebelumnya: Step 1 memvalidasi GPU pada WSL. Step 4 memvalidasi GPU pada Docker container. Step 8 memvalidasi GPU dari PyTorch workload di dalam AI container.

Dengan demikian, Step 8 menjadi validasi terakhir sebelum infrastructure digunakan untuk phase berikutnya.

8.1 Jalankan container Container NVIDIA PyTorch dijalankan dengan akses GPU. Parameter: --rm container dihapus setelah proses selesai. -it menyediakan interactive terminal. --gpus all memberikan akses GPU kepada containe

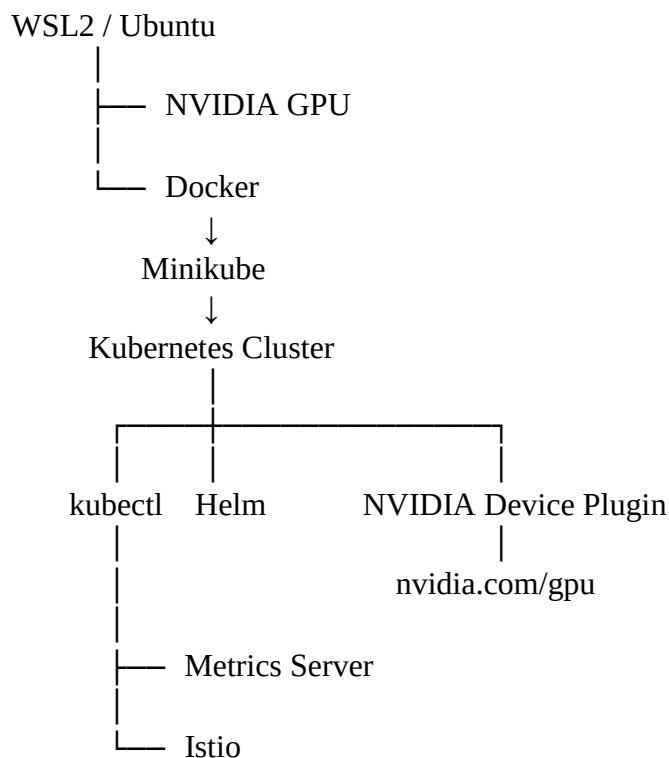
```
docker run --rm -it \
```

```
--gpus all \  
nvcr.io/nvidia/pytorch:25.06-py3  
  
# 8.2 masuk ke program python  
  
python  
  
# 8.3 Ada GPU atau tidak  
  
import torch  
  
torch.cuda.is_available()  
  
# 8.4 Seri GPU, Command ini mengambil nama GPU pada device index 0.  
Dengan demikian, selain mengetahui bahwa CUDA tersedia, kita juga  
dapat memastikan GPU yang digunakan oleh PyTorch.  
  
torch.cuda.get_device_name(0)  
  
# 8.5 Keluar python  
  
exit()  
  
# 8.6 Keluar container  
  
exit
```

Phase 2 — Kubernetes AI Platform

Phase ini membangun Kubernetes layer di atas infrastructure container yang telah disiapkan pada Phase 1. Pada Phase 1, GPU sudah divalidasi sampai level PyTorch di dalam Docker container. Phase 2 kemudian mengubah environment tersebut menjadi single-node Kubernetes cluster menggunakan Minikube, dengan Docker sebagai driver dan container runtime. Selain cluster Kubernetes dasar, phase ini menyiapkan: kubectl sebagai Kubernetes CLI. Helm sebagai package manager Kubernetes. Minikube sebagai Kubernetes distribution untuk local/single-node environment. Metrics Server untuk menyediakan resource metrics. NVIDIA Device Plugin untuk mengekspos GPU sebagai resource Kubernetes. Istio sebagai service mesh yang akan digunakan pada phase inference dan observability berikutnya.

Arsitektur yang dibangun pada phase ini secara konseptual adalah:



Dengan demikian, output Phase 2 adalah Kubernetes platform yang siap menjadi execution platform untuk workload AI, sekaligus sudah memiliki komponen dasar untuk resource monitoring, GPU scheduling, dan service-to-service networking. Struktur phase ini mengikuti project source yang melanjutkan dari Phase 1 ke Kubernetes AI Platform.

Step 9. Pastikan Docker masih normal

Sebelum Kubernetes dibangun, kita melakukan re-validation terhadap Docker dan GPU integration dari Phase 1. Ini penting karena Minikube pada project ini menggunakan Docker sebagai driver. Artinya, apabila Docker atau GPU integration bermasalah, Kubernetes yang berjalan di atas Docker juga berpotensi bermasalah.

9.1 Pastikan Docker masih normal, docker ps memastikan Docker Engine dapat diakses dan menampilkan container yang sedang berjalan. docker

run hello-world melakukan smoke test untuk memastikan Docker masih dapat membuat dan menjalankan container setelah konfigurasi Phase 1.

```
docker ps
```

```
docker run hello-world
```

9.2 GPU

```
nvidia-smi
```

9.3 Docker GPU

```
docker run --rm --gpus all \
```

```
nvidia/cuda:12.9.1-base-ubuntu24.04 \
```

```
nvidia-smi
```

Step 10. Install kubectl

10.1 Download, kubectl merupakan command-line interface utama untuk berinteraksi dengan Kubernetes API. Pada phase berikutnya, hampir seluruh operasi cluster akan dilakukan melalui kubectl

```
curl -LO "https://dl.k8s.io/release/$(curl -Ls  
https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

10.2 Install, chmod +x memberikan executable permission kepada binary. Binary kemudian dipindahkan ke /usr/local/bin agar dapat dipanggil langsung dari shell

```
chmod +x kubectl
```

```
sudo mv kubectl /usr/local/bin/
```

10.3 Cek, Command ini memeriksa versi client kubectl tanpa membutuhkan koneksi ke Kubernetes cluster. Pada titik ini Kubernetes cluster belum dibuat. Jadi yang diverifikasi baru Kubernetes CLI, bukan cluster.

```
kubectl version -client
```

Step 11. Install Helm

Helm digunakan sebagai package manager untuk Kubernetes. Dalam project ini Helm nantinya digunakan untuk memasang komponen Kubernetes yang didistribusikan sebagai Helm chart, salah satunya NVIDIA Device Plugin.

11.1 Download, Script instalasi Helm dijalankan untuk memasang Helm 3 pada sistem. Setelah instalasi, executable helm tersedia dari shell.

```
curl https://raw.githubusercontent.com/helm/helm/main/scripts/get-  
helm-3 | bash
```

11.2 Tes

```
helm version
```

Step 12. Install Minikube

12.1 Download, Minikube digunakan untuk membuat Kubernetes cluster single-node pada environment lokal. Project ini menggunakan Minikube karena target environment merupakan local lab, bukan multi-node production cluster.

```
curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube-
linux-amd64
```

12.2 Install, Binary dipasang ke /usr/local/bin agar command minikube tersedia secara global.

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

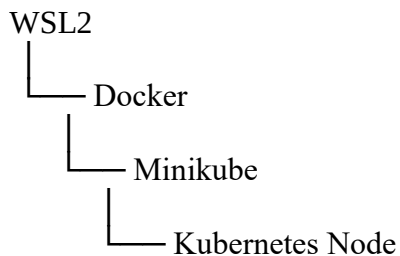
12.3 Tes

```
minikube version
```

Step 13. jalankan Cluster

Step ini merupakan titik di mana Kubernetes cluster benar-benar dibuat.

Minikube akan dijalankan menggunakan Docker sebagai driver sehingga architecture-nya menjadi:



13.1 Jalankan minikube, --driver=docker Menentukan Docker sebagai driver Minikube. Dengan konfigurasi ini Minikube tidak membuat VM terpisah untuk menjalankan cluster, tetapi menggunakan container Docker sebagai environment node. --container-runtime=docker Menentukan Docker sebagai container runtime yang digunakan oleh cluster. --gpus=all Meminta Minikube menggunakan GPU yang tersedia. Ini merupakan bagian dari persiapan GPU-enabled Kubernetes environment. --cpus=4 Memberikan 4 CPU kepada Minikube. --memory=10000 Memberikan sekitar 10 GB memory kepada Minikube. Resource allocation ini penting karena Kubernetes sendiri membutuhkan resource untuk control-plane components dan workload yang akan dijalankan.

```
minikube start \
    --driver=docker \
    --container-runtime=docker \
    --gpus=all \
```



```
--cpus=4 \  
--memory=10000  
  
# 13.2 Cek  
  
kubectl get nodes  
minikube status  
minikube ssh -- nvidia-smi
```

Step 14. Install Metrics Server (Optional buat testing)

```
# 14.1 Aktifkan addon.  
  
# minikube addons enable metrics-server  
  
# 14.2 Cek apakah sudah terbuat  
  
# kubectl get pods -n kube-system  
  
# 14.3 Cek apakah CPU dan RAM bisa terbaca  
  
# kubectl top nodes
```

Step 15. (Optional Linux Native Only) Enable NVIDIA GPU for Kubernetes

Tujuan: Membuat Kubernetes mengenali GPU NVIDIA sebagai resource nvidia.com/gpu

Catatan: Pada WSL2, NVIDIA GPU Operator tidak digunakan karena driver GPU dikelola oleh Windows dan integrasi GPU sudah disediakan oleh WSL2. Untuk lab single-node, cukup menggunakan NVIDIA Device Plugin.

```
# 15.1 Tambahkan Helm repository NVIDIA  
  
helm repo add nvidia https://helm.ngc.nvidia.com/nvidia  
helm repo update  
  
# 15.2 Install NVIDIA Device Plugin  
  
helm install nvdp nvidia/nvidia-device-plugin  
  
# 15.3 Pastikan pod berjalan  
  
kubectl get pods -A  
  
4. Verifikasi apakah Kubernetes sudah melihat GPU  
  
kubectl describe node minikube
```

Step 16. (Optional Linux Native Only) GPU Verification

```
# 16.1 Tes GPU
```

16.2 Buat file gpu-test.yaml

```
cat <<EOF > gpu-test.yaml
apiVersion: v1
kind: Pod
metadata:
  name: gpu-test
spec:
  restartPolicy: Never
  containers:
  - name: cuda
    image: nvidia/cuda:12.9.1-base-ubuntu24.04
    command:
    - nvidia-smi
    resources:
      limits:
        nvidia.com/gpu: 1
EOF
```

16.3 Apply

```
kubectl apply -f gpu-test.yaml
```

16.4 Lihat log, harus muncul 'NVIDIA-SMI'

```
kubectl logs gpu-test
```

Step 17 Install Istio

Istio digunakan sebagai service mesh untuk menyediakan functionality pada komunikasi antar-service di Kubernetes. Komponen ini akan menjadi fondasi untuk phase berikutnya yang mulai menjalankan inference workload dan membutuhkan networking, traffic management, serta observability.

Secara konseptual:

Application Pod



Istio Sidecar



Istio Service Mesh



17.1 Download

```
curl -L https://istio.io/downloadIstio | sh -
```

17.2 Masuk

```
cd istio-*
```

17.3 Tambah PATH

```
export PATH=$PWD/bin:$PATH
```

17.4 Install

```
istioctl install --set profile=demo -y
```

17.5 Enable istio in Minikube

```
# minikube addons enable istio
```

```
# minikube addons enable istio-provisioner
```

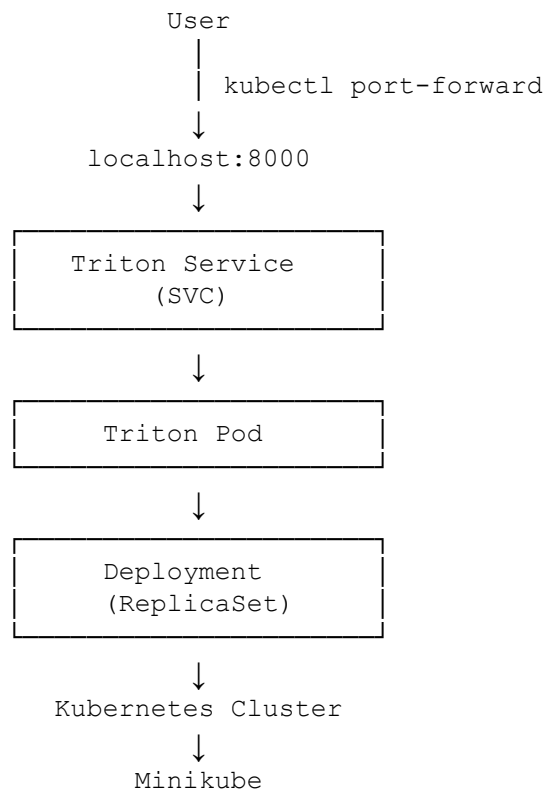
17.6 Enable injection

```
# kubectl label namespace default istio-injection=enabled
```

Phase 3 — AI Inference Platform (CPU Validation)

Tim AI mulai membutuhkan platform untuk menyediakan model machine learning sebagai inference API yang dapat diakses oleh aplikasi lain. Kubernetes yang telah dibangun pada phase sebelumnya digunakan sebagai platform deployment, sedangkan GPU scheduling belum menjadi bagian dari implementasi saat ini. Oleh karena itu, inference platform terlebih dahulu dijalankan menggunakan CPU. Fokus phase ini bukan pada performa inference, melainkan pada validasi alur dasar model serving, Kubernetes deployment, service networking, health checking, dan observability sebelum GPU inference diimplementasikan. Triton Inference Server menjadi komponen utama untuk menyediakan model sebagai service. Deployment memastikan Pod Triton dikelola oleh Kubernetes, sedangkan Service menyediakan endpoint jaringan yang stabil untuk mengakses Pod tersebut. Port 8000, 8001, dan 8002 masing-masing digunakan untuk HTTP inference API, gRPC inference API, dan metrics endpoint.

Arsitektur



Step 18. Buat Namespace AI

Namespace digunakan untuk memisahkan resource inference platform dari workload Kubernetes lainnya. Dengan namespace ai, Deployment, Pod, dan Service yang berkaitan dengan platform inference dapat dikelola secara terisolasi dan lebih mudah diidentifikasi.

```
kubectl create namespace ai
```

Step 19. Buat Folder Project, nanti seluruh manifest Kubernetes Phase 3 disimpan di sini

```
mkdir -p ~/ai-platform/triton
cd ~/ai-platform/triton
```

Step 20. Deploy Triton (deployment.yaml)

NVIDIA Triton Inference Server digunakan sebagai inference server yang bertugas menerima request dari aplikasi dan menjalankan model machine learning. Pada tahap ini Triton dijalankan sebagai Kubernetes Deployment dengan satu replica. Model repository menggunakan emptyDir karena phase ini berfokus pada validasi infrastructure layer terlebih dahulu, bukan pada persistent model storage.

- replicas: 1 menjalankan satu instance Triton Pod.
- selector.matchLabels menentukan Pod yang dikelola oleh Deployment.
- app: triton digunakan sebagai label untuk menghubungkan Deployment dengan Pod dan Service.
- image menggunakan NVIDIA Triton Inference Server.
- --model-repository=/models menentukan lokasi model repository di dalam container.
- --model-control-mode=POLL membuat Triton melakukan polling terhadap model repository.
- --repository-poll-secs=5 menentukan interval polling setiap 5 detik.
- Port 8000 digunakan untuk HTTP/REST.
- Port 8001 digunakan untuk gRPC.
- Port 8002 digunakan untuk metrics.
- emptyDir menyediakan storage sementara untuk /models.

```
cat << 'EOF' > deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: triton
  namespace: ai
spec:
  replicas: 1
  selector:
    matchLabels:
      app: triton
  template:
    metadata:
      labels:
        app: triton
    spec:
      containers:
        - name: triton
          image: nvcr.io/nvidia/tritonserver:25.06-py3
```

```

        args: ["tritonserver", "--model-repository=/models", "--
strict-readiness=false", "--model-control-mode=POLL", "--repository-
poll-secs=5"]
        ports:
        - containerPort: 8000
        - containerPort: 8001
        - containerPort: 8002
        volumeMounts:
        - name: models
          mountPath: /models
        volumes:
        - name: models
          emptyDir: {}
EOF

```

Step 21. Deploy

```
kubectl apply -f deployment.yaml
```

Step 22. Cek Deployment

```
kubectl get deployment -n ai
```

Step 22.1 Cek ReplicaSet

```
kubectl get rs -n ai
```

Step 22.2 Cek Pod (Harus Running), kalo gagal kubectl describe pod <nama-pod> -n ai ATAU kubectl logs <nama-pod> -n ai

```
kubectl get pods -n ai
```

Step 23. Expose Service (service.yaml)

Pod Kubernetes memiliki lifecycle yang bersifat ephemeral sehingga alamat IP Pod tidak cocok digunakan sebagai endpoint aplikasi secara langsung. Kubernetes Service digunakan untuk menyediakan network endpoint yang stabil dan meneruskan traffic menuju Pod Triton berdasarkan label app: triton

```

cat <<EOF > service.yaml
apiVersion: v1
kind: Service
metadata:
  name: triton
  namespace: ai
spec:
  selector:
    app: triton
  ports:
  - name: http
    port: 8000
    targetPort: 8000

```

```
- name: grpc
  port: 8001
  targetPort: 8001
- name: metrics
  port: 8002
  targetPort: 8002
EOF
```

Step 24. Deploy

```
kubectl apply -f service.yaml
```

Step 25. Verifikasi Service

```
kubectl get svc -n ai
```

Step 26. Port Forward (Terminal ini dibiarkan tetap berjalan.)

```
kubectl port-forward svc/triton -n ai 8000:8000
```

Step 27. Health Check (Terminal baru, harus OK duaduanya)

```
curl localhost:8000/v2/health/live
```

```
curl localhost:8000/v2/health/ready
```

Step 28. Metrics (Terminal baru)

```
kubectl port-forward svc/triton -n ai 8002:8002
```

Lalu:

```
curl localhost:8002/metrics
```

Harus muncul ratusan metrik Prometheus, misalnya: `nv_inference_request_success`

Phase 4 — AI Training & MLOps Pipeline

Phase ini membangun training pipeline dan MLOps workflow di atas Kubernetes yang telah tersedia. Jika Phase 3 berfokus pada model serving/inference, maka Phase 4 berfokus pada bagaimana model tersebut dibuat, dilacak, disimpan, dan direproduksi. Pipeline menggabungkan Git, DVC, Argo Workflows, MLflow, PostgreSQL, dan MinIO. Masing-masing komponen memiliki tanggung jawab berbeda:

Git → versioning source code dan konfigurasi pipeline.

DVC → versioning dataset dan dependency data.

Argo Workflows → orchestration training pipeline di Kubernetes.

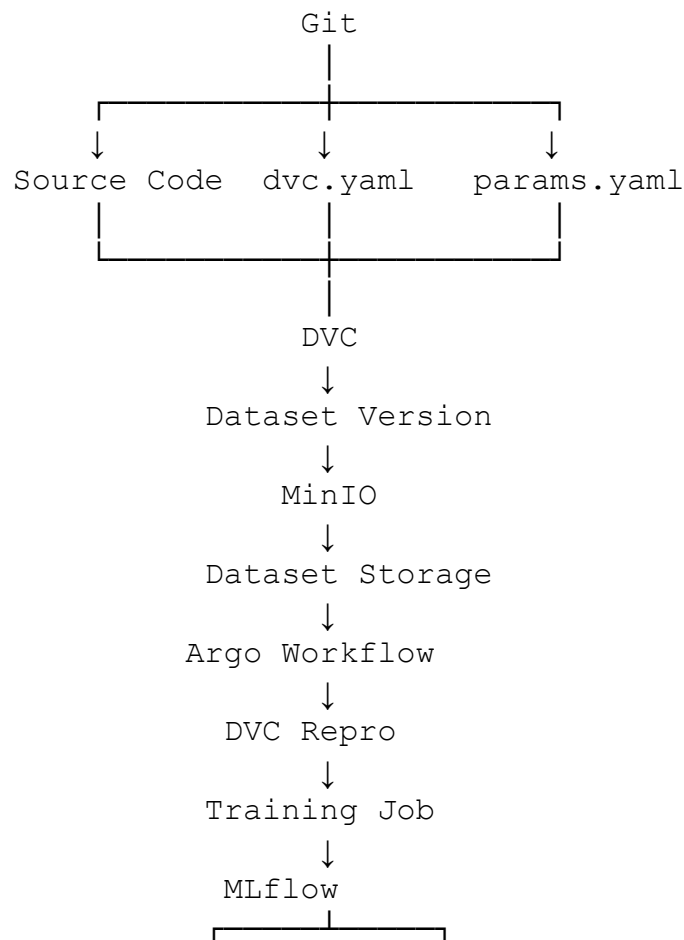
MLflow → experiment tracking dan model artifact management.

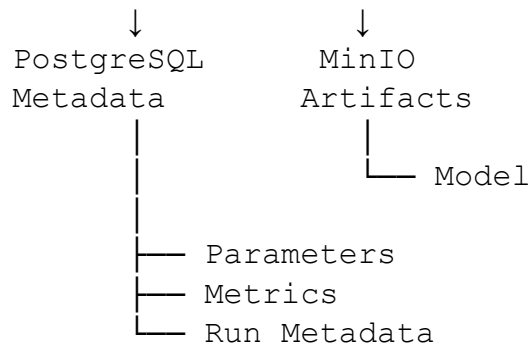
PostgreSQL → penyimpanan metadata MLflow.

MinIO → object storage untuk dataset dan model artifacts.

Target akhirnya adalah setiap training run dapat dikaitkan dengan versi code, versi dataset, hyperparameter, metrics, dan model artifact, sehingga training dapat diulang secara reproducible.

Target Architecture





Dengan workflow tersebut, sebuah training run tidak hanya menghasilkan model, tetapi juga metadata yang menjelaskan bagaimana model tersebut dibuat.

Step 29. Install MinIO

MinIO digunakan sebagai **object storage layer** untuk platform MLOps. Storage ini dibutuhkan karena workflow nantinya akan menghasilkan berbagai file berukuran relatif besar, seperti dataset, model artifact, dan artifact hasil experiment. PostgreSQL tidak digunakan untuk menyimpan file-file tersebut; PostgreSQL digunakan untuk metadata MLflow. Namespace khusus mlops dibuat agar seluruh komponen MLOps dapat dikelompokkan dalam satu logical boundary.

Buat namespace

```
kubectl create namespace mlops
```

Cek

```
kubectl get ns
```

29.1 Tambahkan Helm repository

```
helm repo add minio https://charts.min.io/
```

```
helm repo update
```

Cek:

```
helm search repo minio
```

29.2 Buat values, Konfigurasi tersebut menggunakan mode standalone dengan satu replica sehingga sesuai untuk environment lab. Persistence diaktifkan dengan storage sebesar 10Gi. Hal ini penting karena object storage tidak seharusnya bergantung pada filesystem container yang bersifat ephemeral. Resource request dan limit juga diberikan agar penggunaan resource MinIO tetap terkendali pada environment lokal.

```
mkdir -p ~/ai-factory/phase4
```

```
cd ~/ai-factory/phase4
```

```
cat <<EOF >minio-values.yaml
```

```
mode: standalone
```

```
replicas: 1
```

```
rootUser: minioadmin
```

```
rootPassword: minioadmin123
```

```
persistence:
```

```
    enabled: true
    size: 10Gi
resources:
  requests:
    cpu: 100m
    memory: 256Mi
  limits:
    cpu: 500m
    memory: 512Mi
EOF
```

Install:

```
helm install minio minio/minio \
  -n mlops \
  -f minio-values.yaml
```

29.3 Cek MinIO dan service

```
kubectl get pods -n mlops
kubectl get svc -n mlops
```

Step 30. Akses MinIO dari Windows

30.1 port-forward:

```
kubectl port-forward -n mlops svc/minio-console 9001:9001
```

30.2 Buka browser Windows:

<http://localhost:9001>

30.3 Login, Production note: credential seperti ini hanya cocok untuk local lab. Pada environment production, credential seharusnya disimpan menggunakan Kubernetes Secret atau secret-management system dan tidak ditulis secara plaintext dalam configuration file.

Username: minioadmin

Password: minioadmin123

Step 31. Buat Bucket MLOps

Di MinIO buat bucket, versioning dinyalain, sisanya gausah:

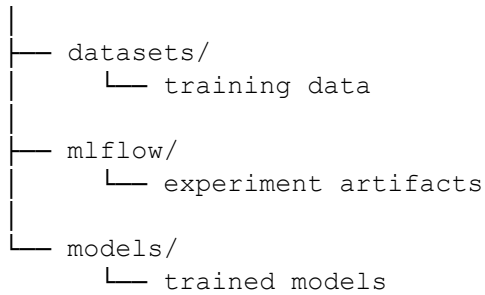
`datasets` → data yang digunakan dalam training.

`mlflow` → artifact yang dikelola oleh MLflow.

`models` → model artifact yang nantinya dapat digunakan oleh inference platform.

Secara konsep:

MinIO

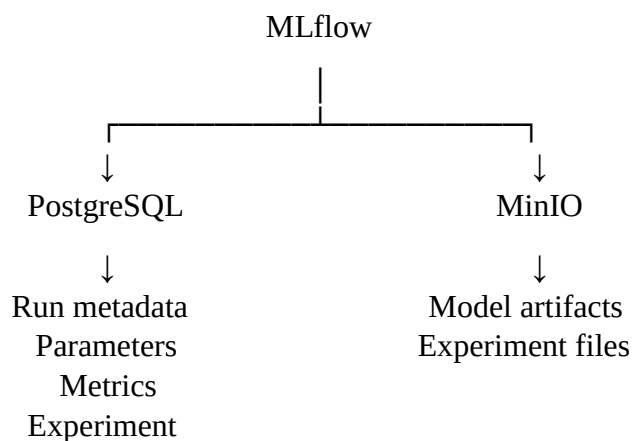


Step 32. Install PostgreSQL untuk MLflow

MLflow membutuhkan storage untuk dua kategori data yang berbeda:

1. Backend Store: Menyimpan metadata experiment dan training run, seperti: Experiment, Run, Parameters, Metrics, Model metadata, Registry metadata
2. Artifact Store: Menyimpan file hasil training, seperti: Model artifact, Plot, Checkpoint, File pendukung lainnya

Arsitektur storage yang digunakan:



PostgreSQL digunakan sebagai backend store, sedangkan MinIO digunakan sebagai artifact store.

32.1 Install PostgreSQL:

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update
helm install postgresql bitnami/postgresql \
  -n mlops \
  --set auth.username=mlflow \
  --set auth.password=mlflow123 \
  --set auth.database=mlflow \
  --set primary.persistence.size=5Gi
```

32.2 Cek:

```
kubectl get pods -n mlops
```

Step 33. Install Mlflow (Kita jalankan MLflow sebagai Kubernetes Deployment)

MLflow digunakan sebagai experiment tracking platform.

MLflow akan menerima informasi dari proses training berupa: Parameters, Metrics, Experiment, Run, Model Artifact

MLflow dijalankan sebagai Kubernetes Deployment sehingga dapat diakses oleh workload lain melalui Kubernetes Service.

33. 1 Manifest tersebut menginstruksikan MLflow server untuk: listen pada port 5000; menggunakan PostgreSQL sebagai backend store; menggunakan MinIO melalui S3-compatible API sebagai artifact store; mengambil credential MinIO dari Kubernetes Secret. Blind spot penting: Secret mlflow-s3 harus sudah tersedia sebelum Pod MLflow dijalankan, sedangkan pada urutan awal lu Secret tersebut baru dibuat di Step 35. Jadi secara implementasi, lebih aman melakukan Step 35 sebelum apply Deployment MLflow, atau apply ulang Deployment setelah Secret dibuat.

```
cat <<EOF > mlflow.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mlflow
  namespace: mlops
spec:
  replicas: 1
  selector:
    matchLabels:
      app: mlflow
  template:
    metadata:
      labels:
        app: mlflow
    spec:
      containers:
        - name: mlflow
          image: ghcr.io/mlflow/mlflow:latest
          command:
            - mlflow
            - server
            - --host=0.0.0.0
            - --port=5000
            - --backend-store-
uri=postgresql://mlflow:mlflow123@postgresql:5432/mlflow
            - --default-artifact-root=s3://mlflow/
          ports:
            - containerPort: 5000
          env:
```

```

- name: MLFLOW_S3_ENDPOINT_URL
  value: http://minio:9000
- name: AWS_ACCESS_KEY_ID
  valueFrom:
    secretKeyRef:
      name: mlflow-s3
      key: AWS_ACCESS_KEY_ID
- name: AWS_SECRET_ACCESS_KEY
  valueFrom:
    secretKeyRef:
      name: mlflow-s3
      key: AWS_SECRET_ACCESS_KEY

```

EOF

Apply:

```
kubectl apply -f mlflow.yaml
```

Step 34. MLflow Service (mlflow-service.yaml)

```

cat <<EOF > mlflow-service.yaml
apiVersion: v1
kind: Service
metadata:
  name: mlflow
  namespace: mlops
spec:
  selector:
    app: mlflow
  ports:
    - port: 5000
      targetPort: 5000
EOF

```

Apply:

```
kubectl apply -f mlflow-service.yaml
```

Step 35. Konfigurasi MinIO untuk MLflow

MLflow perlu tahu: S3 endpoint, Access key, Secret key

Karena MinIO kompatibel dengan S3 API.

25.1 Buat secret:

```

kubectl create secret generic mlflow-s3 \
  -n mlops \
  --from-literal=AWS_ACCESS_KEY_ID=minioadmin \
  --from-literal=AWS_SECRET_ACCESS_KEY=minioadmin123

```

Nanti Deployment MLflow perlu environment:

```
env:
- name: MLFLOW_S3_ENDPOINT_URL
  value: http://minio:9000
- name: AWS_ACCESS_KEY_ID
  valueFrom:
    secretKeyRef:
      name: mlflow-s3
      key: AWS_ACCESS_KEY_ID

- name: AWS_SECRET_ACCESS_KEY
  valueFrom:
    secretKeyRef:
      name: mlflow-s3
      key: AWS_SECRET_ACCESS_KEY
```

Catatan: hostname Service MinIO harus kita cek dengan

```
kubectl get svc -n mlops
```

Karena nama service bisa berbeda tergantung chart/version.

Step 36. Akses MLflow

Port-forward:

```
kubectl port-forward \
-n mlops \
svc/mlflow \
5000:5000
```

Windows browser:

<http://localhost:5000>

Sekarang lu punya:

```
Windows
|
├── :9001 → MinIO
|
└── :5000 → MLflow
```

Step 37. Install DVC

DVC (Data Version Control) digunakan untuk melakukan versioning terhadap dataset dan dependency data tanpa menyimpan file dataset berukuran besar langsung di Git. Git tetap digunakan untuk menyimpan: Code, Configuration, DVC metadata. sedangkan DVC digunakan untuk mengelola: Dataset Dengan demikian Git commit dapat menunjuk pada versi dataset tertentu.

37.1 download python dan membuat env

```
sudo apt update
```

```

sudo apt install -y python3-venv

cd ~/ai-factory/phase4

python3 -m venv .venv

source .venv/bin/activate

# 37.2 Download DVC, Extra [s3] diperlukan karena storage backend yang
digunakan adalah MinIO melalui S3-compatible API.

python -m pip install --upgrade pip

pip install "dvc[s3]"

```

Step 38. Buat Repository ML

Repository ini menjadi source-of-truth untuk training project.

```

cd ~/ai-factory/phase4/ml-project

# 38.1 Git, Git digunakan untuk versioning source code, konfigurasi
training, dan metadata pipeline.

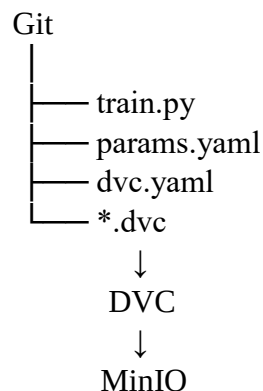
git init

# 38.2 DVC, DVC kemudian membuat konfigurasi internal yang
memungkinkan dataset dikelola menggunakan mekanisme versioning DVC.

dvc init

```

Hubungan keduanya:



Step 39. Dataset + DVC Pipeline Definition

Dataset kecil digunakan agar training dapat dijalankan dengan resource terbatas pada environment lokal. Dataset yang digunakan adalah California Housing.

Buat struktur:

```

ml-project/
├── data/

```

```
|   └─ raw/
|
|   └─ src/
|       └─ train.py
|
|   └─ models/
|
|   └─ dvc.yaml
|   └─ params.yaml
|   └─ requirements.txt
```

39.1 Pengelompokan file

```
cd ~/ai-factory/phase4
mkdir -p ml-project/data/raw
mkdir -p ml-project/src
mkdir -p ml-project/models
touch ml-project/src/train.py
touch ml-project/dvc.yaml
touch ml-project/params.yaml
```

39.2 Masuk ke project

```
cd ~/ai-factory/phase4/ml-project
```

39.3 Download dataset

```
sudo apt update
sudo apt install -y unzip
curl -L -o /tmp/california-housing-prices.zip \
https://www.kaggle.com/api/v1/datasets/download/camnugent/california-
housing-prices
```

39.4 Extract ke raw dataset

```
unzip /tmp/california-housing-prices.zip -d data/raw/
```

39.5 Hapus ZIP sementara

```
rm /tmp/california-housing-prices.zip
```

39.6 Validasi

```
ls -lh data/raw/
head data/raw/housing.csv
```

39.7 `vim ~/ai-factory/phase4/ml-project/dvc.yaml`, `dvc.yaml` mendefinisikan pipeline stage. Dalam kasus ini terdapat satu stage: `train`, DVC mengetahui bahwa training bergantung pada: `housing.csv`, `train.py`, `params.yaml` dan menghasilkan: `random_forest.joblib`, `metrics.json`. Ini

menjadi dasar reproducibility pipeline karena DVC dapat menentukan apakah input training berubah dan apakah stage perlu dijalankan kembali.

```
stages:
  train:
    cmd: python src/train.py
    deps:
      - data/raw/housing.csv
      - src/train.py
    params:
      - n_estimators
      - random_state
      - n_jobs
    outs:
      - models/random_forest.joblib
    metrics:
      - metrics.json:
        cache: false
```

39.8 `vim ~/ai-factory/phase4/ml-project/params.yaml`, Hyperparameter dipisahkan dari source code agar eksperimen dapat dilakukan tanpa mengubah `train.py`.

Contohnya:

n_estimators: 100 → training run #1

kemudian:

n_estimators: 200 → training run #2

Perubahan tersebut dapat dilacak melalui Git dan DVC.

```
n_estimators: 100
random_state: 42
n_jobs: -1
```

Step 40. DVC Tracking

Notes : harus masuk ke venv dulu

40.1 Masuk ke repository ML yang sudah dibuat pada Step 38

```
cd ~/ai-factory/phase4/ml-project
```

40.2 Set remote DVC ke MinIO, Command tersebut membuat remote storage bernama minio yang menunjuk ke bucket datasets.

```
dvc remote add -d minio s3://datasets
```

40.3 Endpoint MinIO, Karena DVC berjalan dari WSL dan MinIO berada di Kubernetes, endpoint localhost:9000 hanya dapat digunakan jika port-forward MinIO API sedang aktif.

```
dvc remote modify minio endpointurl http://localhost:9000
```

40.4 Credential MinIO, Flag --local penting karena credential tidak dimasukkan ke konfigurasi DVC yang akan di-commit ke Git

```

dvc remote modify --local minio access_key_id minioadmin
dvc remote modify --local minio secret_access_key minioadmin123

# 40.5 Validasi remote

dvc remote list

# 40.6 Commit kode dulu (SEBELUM dvc add), Source code dan pipeline
definition disimpan terlebih dahulu di Git. Ini membuat Git memiliki
baseline project sebelum dataset mulai di-version menggunakan DVC.

cd ~/ai-factory/phase4/ml-project

git add src/ dvc.yaml params.yaml

git commit -m "init: project structure, dvc.yaml, params.yaml"

# 40.7 Baru track dataset dengan DVC, DVC tidak memasukkan file
dataset besar tersebut ke Git secara langsung. Sebaliknya, DVC
menghasilkan metadata file: housing.csv.dvc yang berisi referensi
terhadap object dataset yang disimpan pada DVC remote.

dvc add data/raw/housing.csv

# 40.8 Commit dataset version (terpisah), Git sekarang menyimpan
pointer/version metadata dataset, bukan dataset binary-nya.

git add data/raw/housing.csv.dvc data/raw/.gitignore

git commit -m "dataset v1: california housing raw"

# 40.9 Push dataset ke MinIO, Git (housing.csv.dvc) → DVC → MinIO
(dataset object)

dvc push

# 40.10 Validasi

dvc status

git log --oneline

```

Step 41. Training Model + MLflow Tracking

Pada tahap ini dataset yang telah di-version menggunakan DVC digunakan untuk melakukan training model. Algoritma yang digunakan adalah Random Forest Regression dari Scikit-learn. Training akan menghasilkan dua bentuk output:

```

Local artifact
└── models/random_forest.joblib
MLflow
├── Parameters
├── Metrics
└── Model artifact

```

41.1 Masuk ke environment

```
cd ~/ai-factory/phase4
```

```
source .venv/bin/activate
```

```
cd ml-project
```

41.2 Install dependency

```
pip install pandas scikit-learn joblib mlflow boto3
```

41.3 Cek:

```
python -c "import pandas, sklearn, joblib, mlflow, boto3; print('OK')"
```

41.4 Pastikan MLflow Server bisa diakses

Karena MLflow Server lu berjalan di Kubernetes, dari WSL kita expose sementara, Biarkan terminal ini tetap hidup:

```
kubect1 port-forward -n mlops svc/mlflow 5000:5000
```

Di windows browser buka :

```
http://localhost:5000
```

Notes : Untuk kondisi lu sekarang, gunakan `http://localhost:5000`, bukan `http://mlflow:5000`, karena `train.py` dijalankan langsung dari WSL. `mlflow:5000` baru digunakan kalau training juga berjalan di dalam Kubernetes dan bisa resolve Service `mlflow`.

41.5 Expose MinIO ke WSL (Buka tab baru)

```
kubect1 port-forward -n mlops svc/minio 9000:9000
```

41.6 Set credential MinIO → Credential MinIO lu tidak ditulis langsung di source code. Environment variable tersebut kemudian digunakan oleh boto3/MLflow ketika berkomunikasi dengan S3-compatible API MinIO.

```
export AWS_ACCESS_KEY_ID=$(kubect1 get secret minio -n mlops -o  
jsonpath='{.data.rootUser}' | base64 -d)
```

```
export AWS_SECRET_ACCESS_KEY=$(kubect1 get secret minio -n mlops -o  
jsonpath='{.data.rootPassword}' | base64 -d)
```

```
test -n "$AWS_ACCESS_KEY_ID" && echo "ACCESS KEY: OK"
```

```
test -n "$AWS_SECRET_ACCESS_KEY" && echo "SECRET KEY: OK"
```

41.7 Set MinIO endpoint

```
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
```

```
echo $MLFLOW_S3_ENDPOINT_URL
```

41.8 Pastikan bucket mlflow tersedia (Buka tab baru)

```
kubect1 port-forward -n mlops svc/minio-console 9001:9001
```

41.9 Implement Training Script, Script bertanggung jawab terhadap seluruh training lifecycle: Load Dataset → Preprocessing → Train/Test Split → Load Parameters → Train Random Forest → Prediction → Calculate Metrics → Log to Mlflow → Save Local Model → Log Model Artifact

```

vim ~/ai-factory/phase4/ml-project/src/train.py
import os
import json
import yaml
from pathlib import Path
import joblib
import mlflow
import mlflow.sklearn
import pandas as pd
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error, r2_score
from sklearn.model_selection import train_test_split
BASE_DIR = Path(__file__).resolve().parent.parent
DATA_PATH = BASE_DIR / "data" / "raw" / "housing.csv"
PARAMS_PATH = BASE_DIR / "params.yaml"
METRICS_PATH = BASE_DIR / "metrics.json"
MODEL_PATH = BASE_DIR / "models" / "random_forest.joblib"
def main():
    # MLflow configuration
    mlflow.set_tracking_uri(os.environ.get("MLFLOW_TRACKING_URI",
"http://localhost:5000"))
    mlflow.set_experiment("random-forest-training")
    with mlflow.start_run():
        # Load dataset
        print("Loading dataset...")
        df = pd.read_csv(DATA_PATH)
        print(f"Dataset shape: {df.shape}")
        # Target
        target = "median_house_value"
        X = df.drop(columns=[target])
        y = df[target]
        # Handle categorical column
        X = pd.get_dummies(X, drop_first=True)
        # Train / test split
        X_train, X_test, y_train, y_test = train_test_split(
            X,
            y,
            test_size=0.2,
            random_state=42
        )
        print(f"Training samples: {len(X_train)}")
        print(f"Testing samples: {len(X_test)}")
        # Model parameters (dibaca dari params.yaml, bukan hardcode)
        with open(PARAMS_PATH) as f:
            params = yaml.safe_load(f)
        n_estimators = params["n_estimators"]
        random_state = params["random_state"]
        n_jobs = params["n_jobs"]
        # Training
        model = RandomForestRegressor(
            n_estimators=n_estimators,
            random_state=random_state,
            n_jobs=n_jobs
        )
        print("Training model...")

```

```

model.fit(X_train, y_train)
# Prediction
predictions = model.predict(X_test)
# Evaluation
rmse = mean_squared_error(
    y_test,
    predictions
) ** 0.5
r2 = r2_score(
    y_test,
    predictions
)
print(f"RMSE: {rmse:.2f}")
print(f"R2: {r2:.4f}")
# MLflow Parameters
mlflow.log_param(
    "n_estimators",
    n_estimators
)
mlflow.log_param(
    "random_state",
    random_state
)
mlflow.log_param(
    "n_jobs",
    n_jobs
)
# MLflow Metrics
mlflow.log_metric(
    "rmse",
    rmse
)
mlflow.log_metric(
    "r2",
    r2
)
# Tulis metrics.json (dibutuhkan dvc.yaml stage 'train')
with open(METRICS_PATH, "w") as f:
    json.dump({"rmse": rmse, "r2": r2}, f, indent=2)
print(f"Metrics saved to: {METRICS_PATH}")
# Save local model
os.makedirs(
    MODEL_PATH.parent,
    exist_ok=True
)
joblib.dump(
    model,
    MODEL_PATH
)
print(
    f"Model saved to: {MODEL_PATH}"
)
# Log model to MLflow
mlflow.sklearn.log_model(
    model,

```

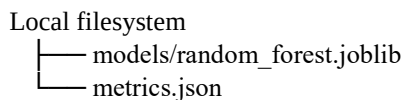
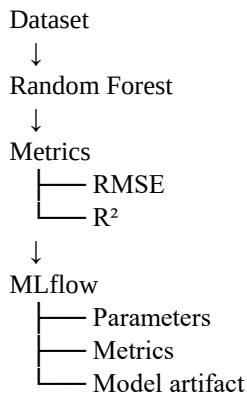
```

        "model"
    )
    print("Model logged to MLflow")
if __name__ == "__main__":
    main()

```

41.10 Jalankan training dengan port forwarding 5000, 9000, 9001 sedang menyala

Training akan:



```
python ~/ai-factory/phase4/ml-project/src/train.py
```

41.11 Validasi local model, File yang diharapkan:
random_forest.joblib. File tersebut merupakan serialized Random Forest model yang dapat digunakan kembali tanpa melakukan training ulang.

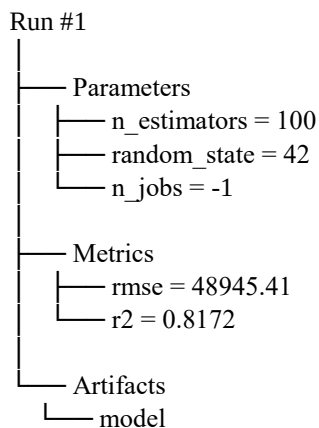
```
ls -lh ~/ai-factory/phase4/ml-project/models
```

41.12 Validasi Mlflow, Dengan demikian MLflow sekarang menyimpan context dari training run, bukan sekadar file model.

Buka: <http://localhost:5000>

Experiment: random-forest-training

Run:

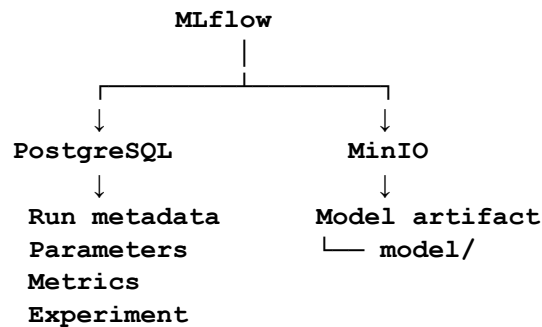


41.13 Validasi MinIO

Buka: `http://localhost:9001`

Di bucket: `mlflow`

harus terdapat artifact yang dibuat oleh MLflow. Jadi akhirnya storage kita:



Sementara local filesystem masih punya:

```
ml-project/
├── data/
│   ├── raw/
│   │   └── housing.csv
├── models/
│   └── random_forest.joblib
└── src/
    └── train.py
```

Step 42. Install Argo Workflows

Sampai Step 41, training masih dijalankan secara manual: `python src/train.py`

Masalahnya adalah training belum menjadi orchestrated Kubernetes workflow. Argo Workflows digunakan untuk mengubah training process tersebut menjadi workflow yang dapat dijalankan oleh Kubernetes.

Secara konsep: Manual python train.py → Training → Argo Workflow → Container → DVC → Training

42.1 Namespace ini digunakan untuk resource Argo Workflows.

```
kubectl create namespace argo
```

42.2

```
kubectl apply -n argo -f https://github.com/argoproj/argo-
workflows/releases/download/v3.6.2/install.yaml kubectl get pods -n
argo
```

42.3 Binding tersebut memberikan cluster-admin permission kepada default ServiceAccount namespace mlops. Catatan keamanan penting: ini sangat permisif dan hanya cocok untuk lab. cluster-admin memberikan

akses hampir penuh terhadap cluster. Untuk environment production, sebaiknya dibuat ServiceAccount khusus Argo dengan RBAC permission minimum (least privilege) sesuai resource yang memang dibutuhkan workflow.

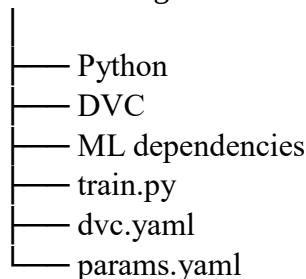
```
kubectl create clusterrolebinding default-cluster-admin-mlops \
  --clusterrole=cluster-admin \
  --serviceaccount=mlops:default 2>/dev/null || true
```

Step 43. Integrate Argo with Training Pipeline

Pada tahap ini training environment dikemas menjadi container image sehingga Argo tidak bergantung pada Python environment host.

Konsepnya:

Docker Image



Argo kemudian menjalankan image tersebut sebagai Kubernetes workload.

43.1 Create Dockerfile, Dockerfile membuat environment training yang reproducible. Perlu dipastikan requirements.txt memang berisi dependency yang digunakan pipeline, minimal: dvc[s3], pandas, scikit-learn, joblib, mlflow, boto3, pyyaml

```
cd ~/ai-factory/phase4/ml-project
# Buat/Update Dockerfile
cat <<'EOF' > Dockerfile
FROM python:3.11-slim
WORKDIR /workspace/ml-project
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EOF
```

43.2 Build image & load langsung ke engine minikube

```
docker build -t ml-project:v1 .
minikube image load ml-project:v1
```


43.3 Create Argo WorkflowTemplate, Workflow ini menjalankan: Argo Workflow → ml-project:v1 → dvc repro -f → DVC Pipeline → train stage → python src/train.py. Perhatikan perbedaan penting dengan Step 41. Step 41: train.py → localhost:5000 / localhost:9000 sedangkan Step 43: train.py → mlflow:5000 / minio:9000. Karena sekarang training container berjalan di dalam Kubernetes, Kubernetes DNS dapat digunakan untuk menemukan Service

```
cat <<EOF > argo-training-workflow.yaml
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: housing-training-pipeline
  namespace: mlops
spec:
  entrypoint: run-pipeline
  templates:
  - name: run-pipeline
    container:
      image: ml-project:v1
      imagePullPolicy: Never
      command: ["dvc", "repro", "-f"]
      env:
        - name: MLFLOW_TRACKING_URI
          value: http://mlflow:5000
        - name: MLFLOW_S3_ENDPOINT_URL
          value: http://minio:9000
      envFrom:
        - secretRef:
            name: mlflow-s3
EOF
```

43.4 Install Argo CLI

```
curl -sLO https://github.com/argoproj/argo-
workflows/releases/download/v3.6.2/argo-linux-amd64.gz

gunzip argo-linux-amd64.gz
chmod +x argo-linux-amd64

sudo mv argo-linux-amd64 /usr/local/bin/argo

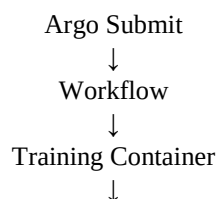
argo version
```

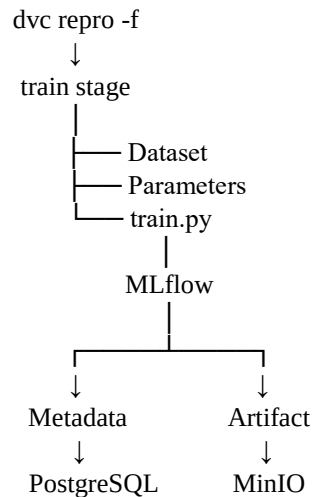
43.5 Register WorkflowTemplate, WorkflowTemplate sekarang tersimpan sebagai Kubernetes custom resource dan dapat digunakan berulang kali.

```
kubectl apply -n mlops -f argo-training-workflow.yaml
```

43.6 Argo akan membuat Workflow berdasarkan WorkflowTemplate.

Pipeline:





```
argo submit --from workflowtemplate/housing-training-pipeline -n mlops  
--watch
```

43.7 validasi

```
cd ~/ai-factory/phase4/ml-project
```

```
cat <<'EOF' > params.yaml
```

```
n_estimators: 200
```

```
random_state: 42
```

```
n_jobs: -1
```

```
EOF
```

43.8 Commit Perubahan

```
git add params.yaml
```

```
git commit -m "experiment: n_estimators=200"
```

43.9 Jalankan pipeline kembali dan validasi

```
argo submit --from workflowtemplate/housing-training-pipeline -n mlops  
--watch
```

```
argo get @latest -n mlops
```

```
git add dvc.lock 2>/dev/null || true
```

```
git commit -m "pipeline run: n_estimators=200 completed" 2>/dev/null  
|| true
```

43.10 Masukkan semua file konfigurasi pipeline ke Git tracking

```
git add Dockerfile argo-training-workflow.yaml dvc.yaml src/  
.dvc/config
```

43.11 Simpan commit penutup Phase 4

```
git commit -m "feat(mlops): complete phase 4 argo workflow integration  
and reproducibility validation"
```

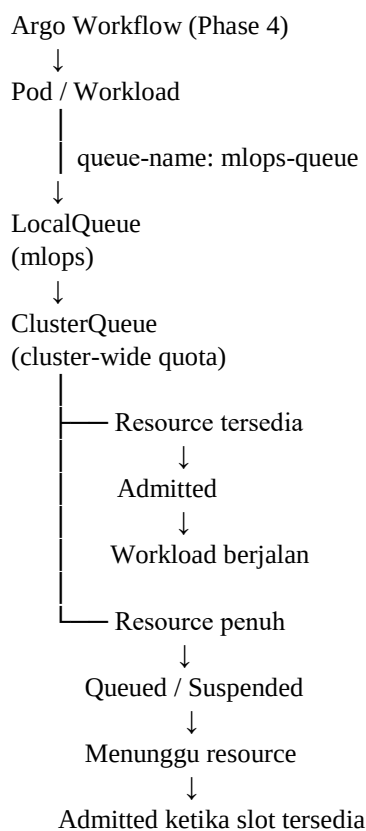
43.12 Cek ulang status git (Pastikan clean)

```
git status
```

Phase 5 — AI Workload Scheduling

Phase ini memperkenalkan workload scheduling dan GPU resource management pada Kubernetes. Pada Phase 4, training pipeline sudah dapat dijalankan melalui Argo Workflow. Ketika beberapa training workload dijalankan secara bersamaan, setiap workload dapat meminta resource seperti GPU, CPU, dan memory. Dengan kapasitas cluster yang terbatas, kondisi ini dapat menyebabkan workload berikutnya masuk ke status Pending atau resource cluster tidak teralokasi secara teratur. Untuk mengatasi masalah tersebut, Phase 5 menggunakan Kueue, workload queueing system dari Kubernetes SIG-Batch. Kueue menambahkan mekanisme antrean, prioritas, dan quota management sebelum workload memperoleh resource dari cluster.

Alur yang dibangun pada phase ini adalah:



Output: training job dari Argo (Phase 4) nggak lagi rebutan GPU secara liar. semuanya lewat antrean yang adil dan bisa dipantau (kubectl get workloads).

Step 44. Menentukan Versi Kueue

Kueue berkembang cepat. API-nya baru saja naik generasi dari v1beta1 ke v1beta2 di rilis terbaru (v0.19.0), dan versi itu mensyaratkan Kubernetes 1.34+. Cek dulu versi cluster lu sebelum pilih. Step ini dilakukan sebelum instalasi Kueue untuk memastikan versi Kubernetes cluster sesuai dengan versi Kueue yang akan digunakan. KUEUE_VERSION kemudian digunakan oleh step berikutnya ketika mengambil manifest Kueue, sedangkan KUEUE_API digunakan dalam manifest resource Kueue seperti ResourceFlavor, ClusterQueue, dan LocalQueue. Catatan: versi Kueue dan API

pada dokumentasi ini mengikuti konfigurasi project. Jangan mengganti versi secara terpisah tanpa memastikan compatibility antara versi Kubernetes, Kueue, dan API yang digunakan.

```
kubectl version  
  
export KUEUE_VERSION=v0.19.0  
  
export KUEUE_API=kueue.x-k8s.io/v1beta2
```

Step 45. Install Kueue

Step ini memasang Kueue controller ke dalam Kubernetes cluster. Kueue bertindak sebagai layer yang mengatur admission workload berdasarkan resource quota dan queueing policy yang nantinya didefinisikan pada Step 46-48.

```
mkdir ~/ai-factory/phase5/kueue  
  
cd ~/ai-factory/phase5/kueue  
  
# 45.1 Manifest diambil berdasarkan versi yang telah ditentukan pada  
Step 44. Dengan menyimpan manifest secara lokal, konfigurasi Kueue  
yang digunakan oleh project dapat disimpan bersama source project  
sehingga lebih mudah direproduksi.  
  
curl -L -o kueue-manifests.yaml \  
    https://github.com/kubernetes-  
sigs/kueue/releases/download/${KUEUE_VERSION}/manifests.yaml  
  
# 45.2 Apply Manifest, Manifest Kueue diterapkan ke Kubernetes  
menggunakan server-side apply. Kubernetes kemudian membuat resource  
yang dibutuhkan Kueue, termasuk controller dan konfigurasi  
pendukungnya.  
  
kubectl apply --server-side -f kueue-manifests.yaml  
  
# 45.3 Restart Controller, Controller Manager Kueue di-restart untuk  
memastikan controller berjalan menggunakan konfigurasi yang telah  
diterapkan.  
  
kubectl rollout restart deployment/kueue-controller-manager -n kueue-  
system  
  
# 45.4 validasi, Command ini digunakan untuk memastikan Pod Kueue  
telah dibuat dan berjalan. Kemudian periksa konfigurasi framework,  
Validasi ini digunakan untuk memeriksa konfigurasi Kueue yang  
tersimpan pada ConfigMap.  
  
kubectl get pods -n kueue-system  
  
kubectl get configmap kueue-manager-config -n kueue-system -o yaml |  
grep -A3 frameworks
```

Step 46. Membuat ResourceFlavor

ResourceFlavor digunakan untuk mendeskripsikan variasi atau kategori resource yang dapat digunakan oleh Kueue. Pada project ini dibuat satu flavor bernama: gpu-flavor. Flavor tersebut kemudian direferensikan oleh ClusterQueue pada Step 47.

46.1 Install gettext-base, Package ini menyediakan envsubst, yang digunakan untuk melakukan variable substitution pada manifest YAML.

```
sudo apt install gettext-base
```

46.2 Buat 01-resource-flavor.yaml, Manifest menggunakan variable: \${KUEUE_API} daripada menuliskan API version secara langsung. Nilai tersebut akan diisi dari environment variable yang telah dibuat pada Step 44. ResourceFlavor yang dibuat bernama: gpu-flavor

```
cat <<EOF > 01-resource-flavor.yaml
apiVersion: ${KUEUE_API}
kind: ResourceFlavor
metadata:
  name: gpu-flavor
EOF
```

46.3 Applyenvsubst mengganti \${KUEUE_API} dengan nilai environment variable yang telah ditentukan. Hasil YAML kemudian langsung diberikan kepada kubectl apply. Alurnya: 01-resource-flavor.yaml → envsubst → API version terisi → kubectl apply → ResourceFlavor (gpu-flavor)

```
envsubst < 01-resource-flavor.yaml | kubectl apply -f -
```

Step 47. Membuat ClusterQueue

ClusterQueue merupakan komponen yang menentukan quota resource pada level cluster. Pada project ini, quota disesuaikan dengan kapasitas Minikube yang digunakan pada phase sebelumnya: CPU: 4, Memory : 10 Gi, GPU: 1. Dengan demikian, Kueue tidak akan menganggap cluster memiliki resource lebih besar daripada kapasitas yang sebenarnya. Ini penting terutama untuk GPU karena environment hanya menyediakan satu GPU yang dapat digunakan oleh workload

47.1 Buat 02-cluster-queue.yaml

Resource yang dikelola oleh ClusterQueue adalah: cpu, memory, nvidia.com/gpu

Quota yang ditetapkan:

Resource	Nominal Quota
CPU	4
Memory	10 Gi
GPU	1

queueingStrategy: StrictFIFO

Queue menggunakan strategi First-In-First-Out. Artinya workload diproses berdasarkan urutan masuk queue sesuai strategi tersebut.

namespaceSelector: {}

Konfigurasi ini memungkinkan ClusterQueue digunakan oleh queue dari namespace yang sesuai tanpa membatasi selector namespace secara spesifik pada manifest tersebut.

nvidia.com/gpu: 1

Ini merupakan bagian paling penting untuk simulasi antrean GPU. Karena quota GPU hanya: 1 GPU maka hanya workload yang kebutuhan GPU-nya dapat dipenuhi oleh quota tersebut yang dapat di-admit secara bersamaan.

```
cat <<EOF > 02-cluster-queue.yaml
apiVersion: ${KUEUE_API}
kind: ClusterQueue
metadata:
  name: cluster-gpu-queue
spec:
  namespaceSelector: {}
  queueingStrategy: StrictFIFO
  resourceGroups:
  - coveredResources: ["cpu", "memory", "nvidia.com/gpu"]
    flavors:
    - name: gpu-flavor
      resources:
      - name: "cpu"
        nominalQuota: 4
      - name: "memory"
        nominalQuota: 10Gi
      - name: "nvidia.com/gpu"
        nominalQuota: 1
```

47.2 ApplyManifest diproses oleh envsubst, kemudian ClusterQueue dibuat atau diperbarui pada Kubernetes. Hasilnya: Cluster → ClusterQueue → cluster-gpu-queue (4 CPU, 10 Gi Memory, 1 GPU). Catatan penting: quota 1 GPU sengaja disesuaikan dengan kapasitas environment project. Menggunakan quota GPU 2 pada cluster dengan hanya satu GPU fisik tidak akan membuat Kubernetes memiliki GPU tambahan.

```
envsubst < 02-cluster-queue.yaml | kubectl apply -f -
```

Step 48. Membuat LocalQueue

LocalQueue menyediakan queue pada level namespace yang kemudian diarahkan ke ClusterQueue. Pada project ini dibuat: Namespace mlops, LocalQueue mlops-queue, ClusterQueue cluster-gpu-queue

48.1 Buat 03-local-queue.yaml, Dengan konfigurasi ini, workload yang masuk ke: mlops-queue akan menggunakan quota yang tersedia pada: cluster-gpu-queue. Hubungannya: Workload → LocalQueue → mlops-queue → ClusterQueue cluster-gpu-queue (CPU quota, Memory quota, GPU quota)

```
cat <<EOF > 03-local-queue.yaml
apiVersion: ${KUEUE_API}
kind: LocalQueue
metadata:
  namespace: mlops
  name: mlops-queue
spec:
```

```

    clusterQueue: cluster-gpu-queue
EOF

# 48.2 Apply

envsubst < 03-local-queue.yaml | kubectl apply -f -

kubectl get localqueue -n mlops

```

Step 49. Integrasi Argo Workflow dengan Kueue

Step ini menghubungkan Argo Workflow dari Phase 4 dengan queue Kueue yang telah dibuat pada Step 48. Sebelum integrasi ini, Argo dapat menjalankan training workload tanpa melewati queue Kueue. Pada step ini ditambahkan label: `kueue.x-k8s.io/queue-name: mlops-queue` sehingga workload dari Argo diarahkan ke LocalQueue tersebut.

49.1 Edit Argo Workflow tambahin podMetadata, abel tersebut mengidentifikasi queue yang harus digunakan workload. Workflow tetap menggunakan infrastructure dari Phase 4: Argo Workflow (DVC, Mlflow, MinIO). tetapi sekarang workload-nya diarahkan ke Kueue: Argo Workflow → mlops-queue → cluster-gpu-queue)

```

vim ~/ai-factory/phase4/ml-project/argo-training-workflow.yaml
apiVersion: argoproj.io/v1alpha1
kind: WorkflowTemplate
metadata:
  name: housing-training-pipeline
  namespace: mlops
spec:
  entrypoint: run-pipeline
  podMetadata:
    labels:
      kueue.x-k8s.io/queue-name: mlops-queue
  templates:
    - name: run-pipeline
      container:
        image: ml-project:v1
        imagePullPolicy: Never
        command: ["dvc", "repro", "-f"]
        env:
          - name: MLFLOW_TRACKING_URI
            value: http://mlflow:5000
          - name: MLFLOW_S3_ENDPOINT_URL
            value: http://minio:9000
        envFrom:
          - secretRef:
              name: mlflow-s3

```

49.2 Apply, WorkflowTemplate diperbarui pada namespace mlops. Dengan demikian, Phase 4 dan Phase 5 mulai terintegrasi: Phase 4 Argo Workflow → Phase 5 Kueue Queue → GPU/CPU/Memory Admission

```

kubectl apply -n mlops -f argo-training-workflow.yaml

```

Step 50. GPU Job Test

Sebelum melakukan pengujian integrasi Argo, dilakukan test menggunakan Kubernetes Job biasa. Tujuannya adalah memisahkan pengujian Kueue dari kompleksitas Argo Workflow. Jika test ini gagal, kita dapat mengetahui bahwa masalah berada pada Kueue atau konfigurasi resource, bukan pada integrasi Argo. Image yang digunakan: nvidia/cuda:12.9.1-base-ubuntu24.04 Image tersebut dibuat sama dengan image CUDA yang digunakan pada Phase 1 agar environment test tetap konsisten.

50.1 Buat 04-gpu-job-test.yaml, Job meminta 1 GPU, 2 CPU, 2Gi Memory dan diarahkan ke mlops-queue. spec.suspend: true digunakan agar Job berada dalam kondisi suspended sampai Kueue melakukan admission. Dengan quota: 1 GPU Job tersebut membutuhkan seluruh quota GPU yang tersedia.

```
cat <<EOF > 04-gpu-job-test.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: training-job-1
  namespace: mlops
  labels:
    kueue.x-k8s.io/queue-name: mlops-queue
spec:
  suspend: true
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: train-container
        image: nvidia/cuda:12.9.1-base-ubuntu24.04
        command: ["nvidia-smi"]
        resources:
          limits:
            nvidia.com/gpu: 1
            cpu: "2"
            memory: "2Gi"
          requests:
            nvidia.com/gpu: 1
            cpu: "2"
            memory: "2Gi"
EOF
```

50.2 Apply, kubectl get workloads digunakan untuk melihat objek Workload yang dikelola oleh Kueue. Ini merupakan perbedaan penting dibanding hanya menjalankan: kubectl get jobs karena Kueue menggunakan resource Workload untuk merepresentasikan workload yang sedang diproses oleh queueing system.

```
kubectl apply -f 04-gpu-job-test.yaml
kubectl get workloads -n mlops
```

Step 51. Stress Test Simulasi Antrean GPU Penuh

Step ini menguji skenario utama Phase 5: Bagaimana Kubernetes menangani dua workload GPU ketika cluster hanya memiliki quota satu GPU? ClusterQueue sebelumnya memiliki: GPU quota = 1, Maka dua workload yang masing-masing meminta: GPU = 1 tidak dapat di-admit secara bersamaan.

51.1 Submit dua job

51.1.1 Job Pertama

```
kubectl apply -f 04-gpu-job-test.yaml
```

51.1.2 Job kedua dibuat dengan mengganti nama Job

```
sed 's/training-job-1/training-job-2/g' 04-gpu-job-test.yaml | kubectl apply -f -
```

51.2 Kemudian cek

```
kubectl get jobs -n mlops
```

```
kubectl get workloads -n mlops
```

Output yang diharapkan:

NAME	COMPLETIONS	AGE
training-job-1	0/1	3s (RUNNING — pakai GPU)
training-job-2	0/1	3s (SUSPENDED — nunggu di antrean)

Begitu training-job-1 selesai, cek lagi:

```
kubectl get workloads -n mlops
```

```
kubectl get jobs -n mlops
```

training-job-2 harus otomatis ke-ADMITTED tanpa campur tangan manual.

Step 52. Validasi Integrasi Argo ↔ Kueue

Step ini merupakan validasi utama Phase 5. Sebelumnya kita sudah membuktikan bahwa Kueue dapat mengelola Kubernetes Job biasa. Sekarang kita perlu memastikan bahwa training pipeline dari Argo Workflow pada Phase 4 juga benar-benar melewati Kueue.

52.1 Submit Argo Workflow, Command tersebut menjalankan WorkflowTemplate housing-training-pipeline yang telah diberi label: kueue.x-k8s.io/queue-name: mlops-queue. Argo kemudian membuat workload/pod berdasarkan workflow tersebut

```
argo submit --from workflowtemplate/housing-training-pipeline -n mlops --watch
```

52.2 Di terminal lain, sambil Argo jalan, cek:

```
kubectl get workloads -n mlops
```

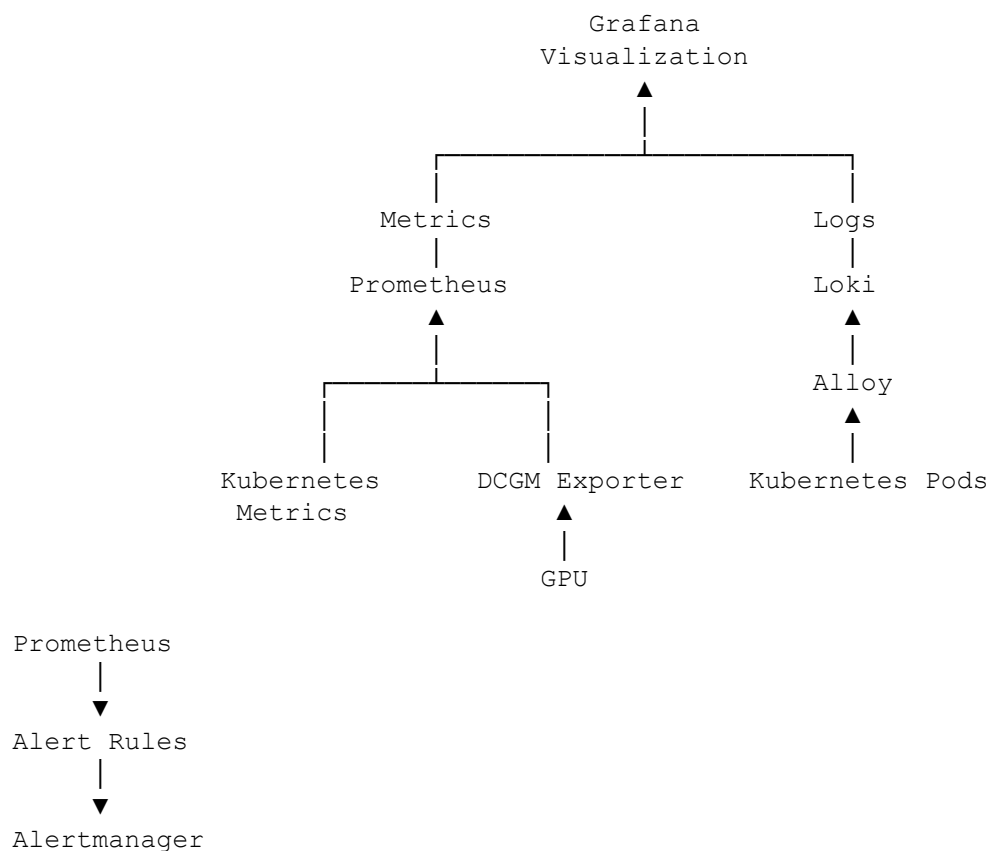
```
kubectl get pods -n mlops -o jsonpath='{.items[*].metadata.labels}' | grep queue-name
```

Kalau ada Workload baru muncul di kubectl get workloads bertepatan dengan argo submit , berarti integrasinya beneran nyambung Pod dari Argo memang dicegat Kueue, sama seperti Pod dari batch/v1 Job biasa.

Phase 6 — AI Observability Platform

Pada phase ini dibangun observability layer untuk AI infrastructure yang telah dibuat pada Phase 1–5. Sebelumnya, sistem sudah memiliki GPU infrastructure, Kubernetes cluster, AI workload, Argo Workflow, dan Kueue. Namun, tanpa observability, kita sulit mengetahui apakah workload berjalan dengan sehat, seberapa besar resource yang digunakan, bagaimana kondisi GPU, serta apa yang terjadi ketika Pod mengalami error. Phase 6 menambahkan tiga pilar utama observability: Metrics → Prometheus, Logs → Loki, Visualization → Grafana, Alerting → Alertmanager, GPU Metrics → DCGM Exporter

Alur observability yang dibangun:



Selain monitoring Kubernetes resource, phase ini juga menambahkan GPU observability menggunakan NVIDIA DCGM Exporter sehingga utilisasi GPU dan metric GPU lainnya dapat dikumpulkan oleh Prometheus.

step 53. Membuat Namespace Monitoring

Observability components dipisahkan ke namespace khusus agar deployment monitoring tidak bercampur dengan workload aplikasi. Dibuat dua namespace yaitu monitoring dan gpu-monitoring. monitoring digunakan untuk komponen utama seperti Prometheus, Grafana, Alertmanager, Loki, dan Alloy. gpu-monitoring digunakan untuk DCGM Exporter.

```
kubectl create namespace monitoring
```

```
kubectl create namespace gpu-monitoring
```

```
kubectl get ns
```

53.1 Validasi

```
kubectl get ns monitoring gpu-monitoring
```

Step 54. Install kube-prometheus-stack (Prometheus + Grafana + Alertmanager + node-exporter)

kube-prometheus-stack digunakan sebagai fondasi monitoring metrics pada cluster. Stack ini menyediakan beberapa komponen sekaligus:

- Prometheus: mengumpulkan dan menyimpan metrics.
- Grafana: memvisualisasikan metrics.
- Alertmanager: menangani alert dari Prometheus.
- node-exporter: menyediakan metrics dari node.
- kube-state-metrics: menyediakan metrics mengenai state resource Kubernetes.

Dengan satu Helm chart, komponen utama monitoring Kubernetes dapat dideploy secara terintegrasi.

54.1 Repository tersebut menyediakan Helm chart kube-prometheus-stack.

```
helm repo add prometheus-community https://prometheus-  
community.github.io/helm-charts
```

```
helm repo update
```

54.2 Membuat konfigurasi

Konfigurasi ini sengaja dibuat relatif ringan untuk environment Minikube single-node.

- **Prometheus** menggunakan: Retention 3 hari, Storage 5 Gi. Resource request dan limit juga ditentukan agar Prometheus tidak mengambil resource secara berlebihan dari cluster.
- **Grafana** diberikan persistence sebesar 2 Gi sehingga data yang disimpan oleh Grafana tidak hanya berada pada ephemeral container filesystem.
- **Alertmanager** diberikan resource limit kecil karena environment yang digunakan merupakan lab single-node.
- **node-exporter dan kube-state-metrics.** Keduanya diaktifkan karena dibutuhkan untuk memperoleh metrics mengenai: kondisi node; CPU; memory; filesystem; state Pod; state Deployment; state resource Kubernetes lainnya.

```
mkdir ~/ai-factory/phase6/monitoring
```

```
cd ~/ai-factory/phase6/monitoring
```

```
cat <<EOF > kube-prometheus-values.yaml
```

```
prometheus:
```

```
  prometheusSpec:
```

```
    retention: 3d
```

```
    resources:
```

```
      requests:
```

```
        cpu: 150m
```

```
        memory: 400Mi
```

```

        limits:
          cpu: 500m
          memory: 800Mi
      storageSpec:
        volumeClaimTemplate:
          spec:
            accessModes: ["ReadWriteOnce"]
            resources:
              requests:
                storage: 5Gi
grafana:
  adminPassword: admin123
  resources:
    requests:
      cpu: 50m
      memory: 128Mi
    limits:
      cpu: 200m
      memory: 256Mi
  persistence:
    enabled: true
    size: 2Gi
alertmanager:
  alertmanagerSpec:
    resources:
      requests:
        cpu: 20m
        memory: 64Mi
      limits:
        cpu: 100m
        memory: 128Mi
nodeExporter:
  enabled: true
kubeStateMetrics:
  enabled: true
EOF

```

54.3 Install Helm Chart

```
helm install kube-prometheus prometheus-community/kube-prometheus-
stack \
```

```

  -n monitoring \
  -f kube-prometheus-values.yaml

```

54.4 Validasi

```

kubectl get pods -n monitoring
kubectl get pvc -n monitoring

```

Step 55. Akses Prometheus & Grafana

Terminal 1 - Prometheus UI

```
kubectl port-forward -n monitoring svc/kube-prometheus-kube-prometheus 9090:9090
```

Terminal 2 - Grafana Dashboard

```
kubectl port-forward -n monitoring svc/kube-prometheus-grafana 3000:80
```

Buka browser ke <http://localhost:9090> (Prometheus) dan <http://localhost:3000> (Grafana, login admin / admin123).

Validasi:

- Prometheus: Buka Status → Targets, minimal target node-exporter, kube-state-metrics, dan apiserver berstatus UP.
- Grafana: Masuk Connections → Data sources, sudah ada Prometheus ter-konfigurasi otomatis oleh chart.

Step 56. Install DCGM Exporter (metrik GPU)

Prometheus pada Step 54 terutama memberikan metrics mengenai Kubernetes dan node. Namun, monitoring AI infrastructure membutuhkan informasi yang lebih spesifik mengenai GPU. Untuk itu digunakan NVIDIA DCGM Exporter. DCGM Exporter mengambil GPU telemetry melalui NVIDIA Data Center GPU Manager (DCGM) dan mengeksposnya dalam format metrics yang dapat di-scrape oleh Prometheus. Contoh metric yang digunakan pada project ini adalah DCGM_FI_DEV_GPU_UTIL, metric tersebut menunjukkan utilisasi GPU.

56.1 Tambahkan Helm Repo

```
helm repo add gpu-helm-charts https://nvidia.github.io/dcgm-exporter/helm-charts
```

```
helm repo update
```

56.2 Buat file dcgm-values.yaml (Di-tuning khusus WSL2 & GPU Pass-Through biar gak OOMKilled dan kebaca K8s):

```
cat <<EOF > dcgm-values.yaml
serviceMonitor:
  enabled: true
  namespace: monitoring
resources:
  requests:
    cpu: 50m
    memory: 128Mi
  limits:
    cpu: 300m
    memory: 512Mi
EOF
```

56.3 Install / Upgrade Helm Chart:

```
helm upgrade --install dcgm-exporter gpu-helm-charts/dcgm-exporter \
  -n gpu-monitoring \
```

```

-f dcgm-values.yaml

# 56.4 Validasi Cek status Pod: Berhasil jika Status pod dcgm-exporter-
* harus 1/1 Running (tidak OOMKilled atau CrashLoopBackOff).

kubectl get pods -n gpu-monitoring

# 56.5 Cek nvidia-smi di dalam Pod exporter (pakai selector label Helm
modern): Lolos kalau nvidia-smi menampilkan info GPU fisiknya (misal:
NVIDIA GeForce GTX 1660 SUPER / RTX series).

POD_NAME=$(kubectl get pod -n gpu-monitoring -l
app.kubernetes.io/name=dcgm-exporter -o
jsonpath='{.items[0].metadata.name}')

kubectl exec -n gpu-monitoring -it $POD_NAME -- nvidia-smi

# 56.6 Cek metrik langsung dari Exporter: Lolos kalau muncul output baris
metrik utilisasi GPU yang menyertakan model GPU dan nilai utilisasinya
(contoh: DCGM_FI_DEV_GPU_UTIL{...modelName="NVIDIA GeForce GTX 1660
SUPER"...} 0).

kubectl port-forward -n gpu-monitoring svc/dcgm-exporter 9400:9400

Buka terminal lain/cURL:

curl -s localhost:9400/metrics | grep DCGM_FI_DEV_GPU_UTIL

```

Step 56. Integrasi DCGM Exporter ke Prometheus

Setelah DCGM Exporter berjalan, metrics-nya masih perlu ditemukan oleh Prometheus.

56.1 Agar Prometheus otomatis men-scrape target dari gpu-monitoring, berikan label release=kube-prometheus pada ServiceMonitor bawaan DCGM. Label tersebut digunakan agar Prometheus Operator menemukan ServiceMonitor DCGM Exporter. Alurnya menjadi: GPU → DCGM Exporter → ServiceMonitor → Prometheus → GPU Metrics

```

kubectl label servicemonitor dcgm-exporter -n gpu-monitoring
release=kube-prometheus --overwrite

```

56.2 Validasi Port-Forward Prometheus:

```

kubectl port-forward -n monitoring svc/kube-prometheus-kube-prometheus
prometheus 9090:9090

```

- Buka **http://localhost:9090/targets** di browser, cari target serviceMonitor/gpu-monitoring/dcgm-exporter/0 — statusnya harus UP.
- Di tab Graph, ketik query `DCGM_FI_DEV_GPU_UTIL` lalu klik Execute. harus memunculkan data metrik GPU secara real-time.

Step 57. Install Loki (log storage)

Prometheus digunakan untuk metrics, sedangkan observability juga membutuhkan log collection. Untuk log storage digunakan Loki. Loki bertugas menyimpan log yang dikirim oleh log collector. Pada project ini Loki digunakan sebagai backend log untuk Grafana.

57.1 Tambahkan Grafana Helm repository

```
helm repo add grafana https://grafana.github.io/helm-charts
```

```
helm repo update
```

57.2 Konfigurasi Loki, Loki dikonfigurasi menggunakan mode SingleBinary karena project menggunakan single-node Minikube dan bukan deployment production multi-node. Storage juga menggunakan filesystem sehingga tidak membutuhkan object storage tambahan untuk log pada phase ini.

```
cat <<EOF > loki-values.yaml
deploymentMode: SingleBinary
loki:
  auth_enabled: false
  commonConfig:
    replication_factor: 1
  storage:
    type: 'filesystem'
  schemaConfig:
    configs:
      - from: "2024-01-01"
        store: tsdb
        object_store: filesystem
        schema: v13
        index:
          prefix: index_
          period: 24h
singleBinary:
  replicas: 1
  resources:
    requests:
      cpu: 50m
      memory: 100Mi
    limits:
      cpu: 200m
      memory: 256Mi
lokiCanary:
  enabled: true
chunksCache:
  enabled: false
resultsCache:
  enabled: false
backend:
  replicas: 0
read:
  replicas: 0
write:
  replicas: 0
EOF
```

57.3 Apply


```
helm install loki grafana/loki -n monitoring -f loki-values.yaml
```

57.4 Validasi

terminal 1

```
kubectl get pods -n monitoring -l app.kubernetes.io/name=loki
```

```
kubectl port-forward -n monitoring svc/loki 3100:3100
```

terminal 2 (harus nge return READY)

```
curl localhost:3100/ready
```

Step 58. Install Grafana Alloy (pengganti Promtail log collector)

Loki membutuhkan log collector untuk mengambil log dari Kubernetes. Pada project ini digunakan Grafana Alloy sebagai pengganti Promtail. Alloy bertugas: menemukan Pod Kubernetes; mengambil metadata Pod; membaca container logs; menambahkan metadata seperti namespace, Pod, container, dan node; mengirim log ke Loki. Arsitekturnya: Kubernetes Pod → Container Logs → Grafana Alloy → Loki → Grafana

58.1 Konfigurasi Alloy

Konfigurasi Alloy menggunakan Kubernetes discovery discovery.kubernetes untuk menemukan Pod. Metadata kemudian dipetakan menjadi label namespace, pod, container, node. Log dibaca menggunakan loki.source.kubernetes dan dikirim ke loki.write, endpoint `http://loki-gateway.monitoring.svc.cluster.local/loki/api/v1/push` RBAC juga diaktifkan `rbac.create: true` agar Alloy memiliki permission yang dibutuhkan untuk membaca Pod logs. Resource Alloy dibatasi pada CPU request 50m limit 150m; Memory request 64Mi limit 128Mi. DaemonSet Konfigurasi `controller.type: daemonset`, berarti Alloy dijalankan sebagai DaemonSet, sehingga satu Pod Alloy ditempatkan pada setiap node. Pada cluster single-node project ini 1 node menggunakan 1 alloy pod. Jika cluster memiliki beberapa node: Node 1 → Alloy, Node 2 → Alloy, Node 3 → Alloy. Dengan model tersebut setiap node memiliki Alloy instance untuk mengumpulkan log dari workload pada node tersebut.

```
cat <<EOF > alloy-values.yaml
alloy:
  configMap:
    content: |
      // 1. Discovery Kubernetes Pods
      discovery.kubernetes "pod" {
        role = "pod"
      }
      // 2. Relabeling Metadata (Namespace, Pod Name, Container Name)
      discovery.relabel "pod_logs" {
        targets = discovery.kubernetes.pod.targets
        rule {
          source_labels = ["__meta_kubernetes_namespace"]
          action         = "replace"
          target_label   = "namespace"
        }
        rule {
          source_labels = ["__meta_kubernetes_pod_name"]
          action         = "replace"
          target_label   = "pod"
        }
      }
```

```

    rule {
      source_labels = ["__meta_kubernetes_pod_container_name"]
      action        = "replace"
      target_label  = "container"
    }
    rule {
      source_labels = ["__meta_kubernetes_pod_node_name"]
      action        = "replace"
      target_label  = "node"
    }
  }
  // 3. Read Container Logs from disk
  loki.source.kubernetes "pod_logs" {
    targets      = discovery.relabel.pod_logs.output
    forward_to   = [loki.write.local.receiver]
  }
  // 4. Send to Loki Service
  loki.write "local" {
    endpoint {
      url = "http://loki-
gateway.monitoring.svc.cluster.local/loki/api/v1/push"
    }
  }
}

# Aktifkan RBAC agar Alloy diizinkan membaca Pod logs di Cluster
rbac:
  create: true
controller:
  type: daemonset
resources:
  requests:
    cpu: 50m
    memory: 64Mi
  limits:
    cpu: 150m
    memory: 128Mi
EOF

```

58.2 "Simple: controller.type: daemonset" itu artinya 1 Alloy pod jalan di setiap node – buat minikube single-node lu sekarang cuma jadi 1 pod, tapi kalau nanti tambah node, otomatis tambah juga.

```
helm install alloy grafana/alloy -n monitoring -f alloy-values.yaml
```

58.3 Validasi Lolos kalau pod alloy-* Running.

```
kubectl get pods -n monitoring -l app.kubernetes.io/name=alloy
```

58.4 Cek logs beneran ngalir ke Loki. Lolos kalau mlops muncul di daftar namespace – artinya logs dari namespace mlops (MLflow, Argo, dll) udah dikirim ke Loki.

Terminal 1

```
kubectl port-forward -n monitoring svc/loki 3100:3100
```

Terminal 2 (harus keluar '{"status": "success", "data": ["argo", "kubernetes", "mlops", "monitoring"]}')

```
curl -s "http://localhost:3100/loki/api/v1/label/namespace/values" |  
grep mlops
```

Step 59. Alertmanager (Contoh Alert Rule)

Monitoring akan menjadi lebih berguna apabila sistem tidak hanya menampilkan metrics, tetapi juga dapat memberikan indikasi ketika kondisi tertentu terjadi. Pada step ini dibuat dua alert yaitu GPUHighUtilization dan PodCrashLooping. Alert dibuat menggunakan PrometheusRule.

59.1 Membuat Alert rule, GPUHighUtilization Rule:

DCGM_FI_DEV_GPU_UTIL > 90 dan kondisi harus berlangsung 5 menit sebelum alert aktif. Tujuannya agar utilisasi GPU yang hanya naik sesaat tidak langsung dianggap sebagai kondisi bermasalah.

PodCrashLooping Rule menggunakan

kube_pod_container_status_restarts_total untuk mendeteksi adanya restart container. Alert diberi severity: critical sedangkan GPU utilization diberi: warning

```
cat <<EOF > alert-rules.yaml  
apiVersion: monitoring.coreos.com/v1  
kind: PrometheusRule  
metadata:  
  name: ai-factory-alerts  
  namespace: monitoring  
  labels:  
    release: kube-prometheus  
spec:  
  groups:  
    - name: gpu-and-pod-health  
      rules:  
        - alert: GPUHighUtilization  
          expr: DCGM_FI_DEV_GPU_UTIL > 90  
          for: 5m  
          labels:  
            severity: warning  
          annotations:  
            summary: "GPU utilisasi tinggi (>90% selama 5 menit)"  
        - alert: PodCrashLooping  
          expr: rate(kube_pod_container_status_restarts_total[15m]) > 0  
          for: 5m  
          labels:  
            severity: critical  
          annotations:  
            summary: "Pod {{ $labels.pod }} di namespace {{  
$labels.namespace }} CrashLoopBackOff"  
EOF
```

59.2 Apply

```
kubectl apply -f alert-rules.yaml
```

59.3 Validasi

```
kubectl port-forward -n monitoring svc/kube-prometheus-kube-prometheus 9090:9090
```

Buka <http://localhost:9090/alerts>, harus muncul GPUHighUtilization dan PodCrashLooping dengan status inactive (normal, karena kondisinya belum terpenuhi). Coba trigger manual (opsional): jalankan training job berat lewat Argo (Phase 4) sambil pantau status alert harus berubah jadi pending lalu firing kalau GPU beneran di atas 90% selama 5 menit.

Step 60. Dashboard Grafana (Import Dashboard Siap Pakai)

Daripada bikin dashboard dari nol, pakai dashboard komunitas yang udah teruji:

Dashboard	ID	Isi
	Grafana.com	
NVIDIA DCGM Exporter Dashboard	12239	GPU utilization, VRAM, temperature, power
Loki Logs / Log App	13639	Log volume & error rate per namespace

60.1 diatas bagian kanan search bar ada tanda “+”, klik dan pilih “Import dashboard lagi

60.2 isi bagian “Find and import dashboards for common applications at grafana.com/dashboards” dengan 12239.

60.3 isi “DS_PROMETHEUS Select a prometheus data source” dengan prometheus

60.4 klik “Connections” yg posisinya dikiri halaman > **View Configured Data Sources** > + **Add new data source** > **Loki**

60.5 Connection > URL * <http://loki.monitoring.svc.cluster.local:3100>

60.6 klik **Save & test**, harus keluar **Data source is working**

60.7 diatas bagian kanan search bar ada tanda “+”, klik dan pilih “Import dashboard lagi

60.8 isi bagian “Find and import dashboards for common applications at grafana.com/dashboards” dengan 13639.

60.9 isi “DS_LOKI Select a loki data source” dengan **Loki**

60.10 Validasi: Lolos kalau ketiga dashboard nampilin grafik dengan data (bukan panel kosong "No data"). Kalau panel GPU kosong, balik cek Step 54-55 (kemungkinan ServiceMonitor DCGM belum ke-scrape).

Phase 7 — Multi-Tenant AI Platform

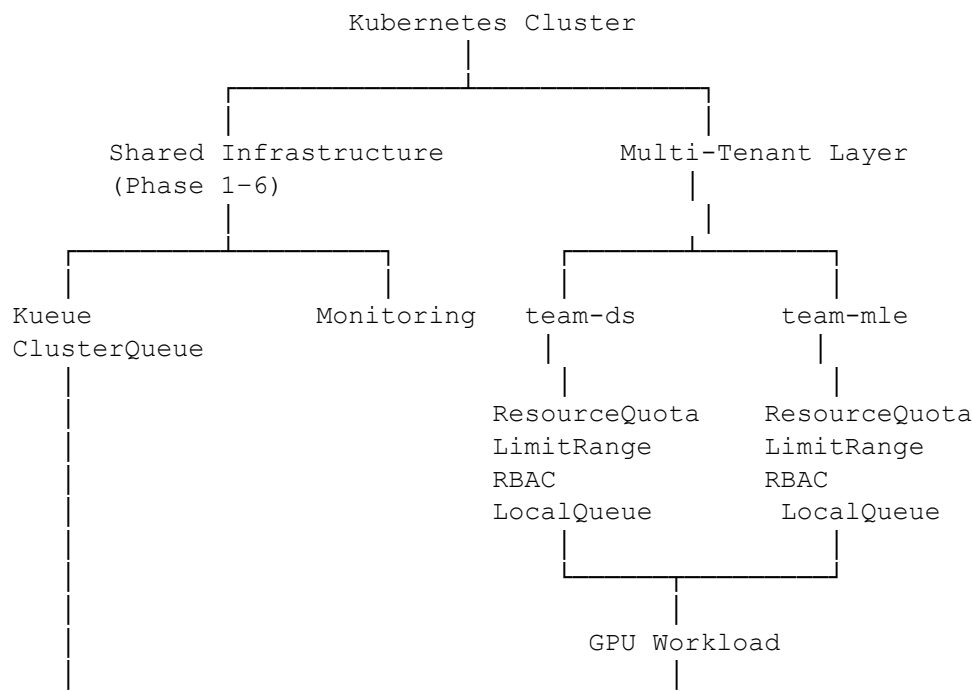
Pada phase ini, infrastruktur AI yang sebelumnya masih digunakan secara bersama mulai diubah menjadi multi-tenant platform. Artinya, beberapa tim dapat menggunakan cluster Kubernetes yang sama, tetapi masing-masing tim memiliki namespace, hak akses, resource limit, dan antrean GPU yang terpisah. Desain phase ini memisahkan dua jenis kontrol resource:

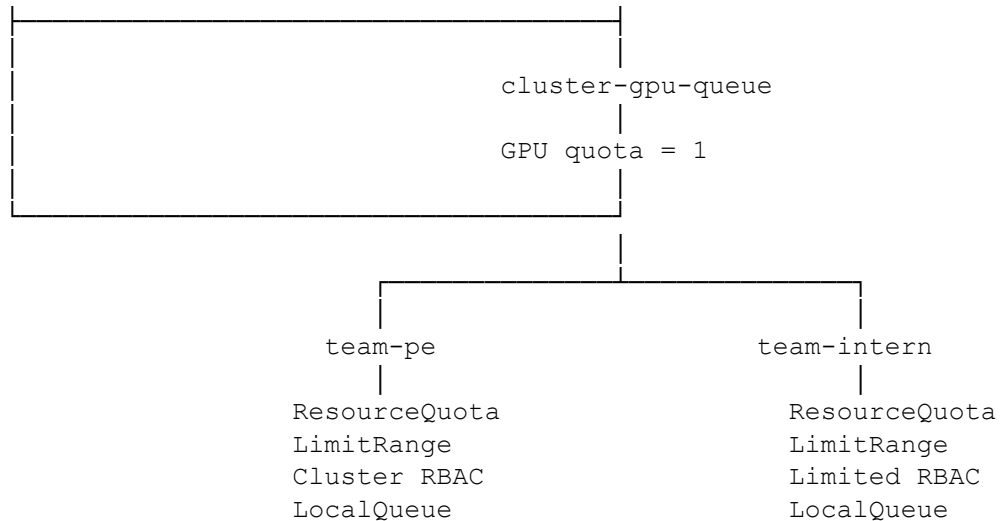
- ResourceQuota + LimitRange → mengontrol CPU, memory, dan jumlah Pod/object pada level namespace
- Kueue → mengontrol GPU scheduling dan antrean workload secara cluster-wide.

GPU sengaja tidak dimasukkan ke ResourceQuota native, karena Phase 5 sudah menjadikan Kueue sebagai mekanisme utama untuk mengatur penggunaan GPU. Dengan demikian, hanya ada satu sumber kebenaran untuk kapasitas GPU, yaitu cluster-gpu-queue. Empat tenant yang dibuat merepresentasikan persona pengguna yang berbeda:

Tenant	Persona	Akses utama
team-ds	Data Scientist	Mengelola workload di namespace sendiri
team-mle	ML Engineer	Workload + monitoring Kueue
team-pe	Platform Engineer	Administrasi resource dan Kueue
team-intern	Intern	Akses terbatas/read-only

Arsitektur:





Intinya, namespace memberikan boundary, RBAC menentukan siapa boleh melakukan apa, ResourceQuota/LimitRange membatasi CPU dan memory, sedangkan Kueue menentukan siapa yang mendapatkan GPU ketika resource tersebut diperebutkan.

Step 61. Buat Namespace per Tim

Step ini membuat boundary administratif untuk setiap tenant. Daripada seluruh user menjalankan workload pada namespace mlops, setiap persona mendapatkan namespace masing-masing. Hal ini penting karena Kubernetes menggunakan namespace sebagai salah satu boundary utama untuk RBAC, ResourceQuota, LimitRange, dan resource namespaced lainnya.

61.1 Empat namespace dibuat untuk memisahkan workload masing-masing tim: team-ds untuk Data Scientist, team-mle untuk ML Engineer, team-pe untuk Platform Engineer, team-intern untuk Intern. Namespace infrastructure seperti mlops, monitoring, argo, kueue-system, dan gpu-monitoring tetap dipertahankan sebagai shared infrastructure dan tidak dianggap sebagai tenant.

```

kubect1 create namespace team-ds      # Data Scientist
kubect1 create namespace team-mle     # ML Engineer
kubect1 create namespace team-pe      # Platform Engineer
kubect1 create namespace team-intern  # Intern
  
```

61.2 Validasi Lolos kalau 4 namespace baru muncul dengan status Active, terpisah dari namespace infra (mlops, monitoring, argo, kueue-system, gpu-monitoring).

```
kubect1 get ns | grep team-
```

Step 62. RBAC (Buat Role per Tim)

Setelah namespace dipisahkan, langkah berikutnya adalah menentukan siapa yang boleh melakukan apa. Namespace saja belum cukup untuk membuat multi-tenancy. User yang mempunyai akses terlalu luas masih dapat membaca atau mengubah resource milik tenant

lain. Karena itu, setiap persona diberikan permission berbeda menggunakan Kubernetes Role-Based Access Control (RBAC). Desain permission dibuat berdasarkan tanggung jawab masing-masing persona:

Data Scientist

└─ namespace-scoped workload management

ML Engineer

└─ namespace-scoped workload management

+ read-only Kueue visibility

Platform Engineer

└─ cluster-wide platform administration

Intern

└─ read-only workload visibility

62.1

```
mkdir -p ai-factory/phase7/multi-tenant
```

```
cd ai-factory/phase7/multi-tenant/
```

62.2 61a-role-data-scientist.yaml, Role Data Scientist dibuat dengan scope team-ds. Permission diberikan untuk mengelola workload yang diperlukan dalam proses development dan training, seperti Pod dan log Pod; Job; PVC; Argo Workflow dan WorkflowTemplate; Deployment; Service. Karena menggunakan Role, permission tersebut hanya berlaku di namespace team-ds.

```
cat <<EOF > 61a-role-data-scientist.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: team-ds
  name: data-scientist-role
rules:
- apiGroups: [ "", "batch", "argoproj.io" ]
  resources: [ "pods", "pods/log", "jobs", "persistentvolumeclaims",
"workflows", "workflowtemplates" ]
  verbs: [ "get", "list", "watch", "create", "delete", "update", "patch" ]
- apiGroups: [ "apps" ]
  resources: [ "deployments" ]
  verbs: [ "get", "list", "watch", "create", "delete", "update", "patch" ]
- apiGroups: [ "" ]
  resources: [ "services" ]
  verbs: [ "get", "list", "watch", "create", "delete", "update", "patch" ]
EOF
```

62.3 61b-role-ml-engineer.yaml, ML Engineer mendapatkan permission workload yang mirip dengan Data Scientist di namespace team-mle. Perbedaannya adalah ML Engineer juga mendapatkan read-only access terhadap LocalQueue dan Workload Kueue. Ini masuk akal karena ML Engineer

perlu dapat melihat status antrian dan workload GPU tanpa harus mendapatkan kemampuan untuk mengubah konfigurasi Kueue.

```
cat <<EOF > 61b-role-ml-engineer.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: team-mle
  name: ml-engineer-role
rules:
- apiGroups: ["", "batch", "argoproj.io"]
  resources: ["pods", "pods/log", "jobs", "persistentvolumeclaims",
"workflows", "workflowtemplates"]
  verbs: ["get", "list", "watch", "create", "delete", "update", "patch"]
- apiGroups: ["kueue.x-k8s.io"]
  resources: ["localqueues", "workloads"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["apps"]
  resources: ["deployments"]
  verbs: ["get", "list", "watch", "create", "delete", "update", "patch"]
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list", "watch", "create", "delete", "update", "patch"]
EOF
```

62.4 61c-clusterrole-platform-engineer.yaml, Platform Engineer memiliki tanggung jawab yang lebih tinggi sehingga menggunakan ClusterRole, bukan Role biasa. Permission mencakup resource platform seperti: Namespace, ResourceQuota, LimitRange, Pod dan Pod logs, ClusterQueue, LocalQueue, ResourceFlavor, Workload, WorkloadPriorityClass. Karena ClusterRole dapat digunakan secara cluster-wide, persona ini dapat melakukan administrasi terhadap resource yang berada di berbagai namespace.

```
cat <<EOF > 61c-clusterrole-platform-engineer.yaml
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: platform-engineer-role
rules:
- apiGroups: [""]
  resources: ["namespaces", "resourcequotas", "limitranges", "pods",
"pods/log"]
  verbs: ["get", "list", "watch", "create", "update", "delete"]
- apiGroups: ["kueue.x-k8s.io"]
  resources: ["clusterqueues", "localqueues", "resourceflavors",
"workloads", "workloadpriorityclasses"]
  verbs: ["get", "list", "watch", "create", "update", "delete"]
EOF
```

62.5 61d-role-intern.yaml, Intern mendapatkan permission paling terbatas. Akses workload hanya bersifat read-only, sehingga intern dapat melihat Pod, Pod logs, Job, LocalQueue, Workload Kueue. Tidak diberikan permission create, update, patch, atau delete. Tujuannya adalah merepresentasikan tenant dengan privilege rendah.

```
cat <<EOF > 61d-role-intern.yaml
```



```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: team-intern
  name: intern-role
rules:
- apiGroups: ["", "batch"]
  resources: ["pods", "pods/log", "jobs"]
  verbs: ["get", "list", "watch"]
- apiGroups: ["kueue.x-k8s.io"]
  resources: ["localqueues", "workloads"]
  verbs: ["get", "list", "watch"]
EOF

```

62.6 Apply

```

kubectl apply -f 61a-role-data-scientist.yaml
kubectl apply -f 61b-role-ml-engineer.yaml
kubectl apply -f 61c-clusterrole-platform-engineer.yaml
kubectl apply -f 61d-role-intern.yaml

```

62.7 Validasi (Lolos kalau ke-4 Role/ClusterRole muncul tanpa error)

```

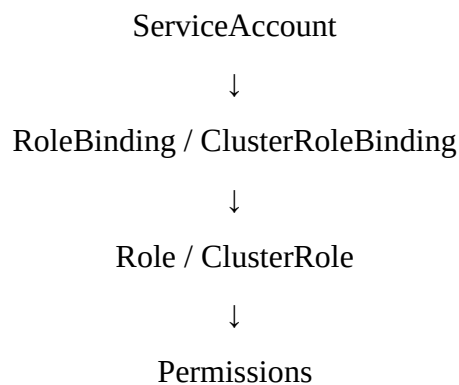
kubectl get role -n team-ds
kubectl get role -n team-mle
kubectl get clusterrole platform-engineer-role
kubectl get role -n team-intern

```

Step 63. Buat ServiceAccount & RoleBinding (Simulasi 4 Persona)

Role yang sudah dibuat pada Step 62 belum secara otomatis berlaku kepada siapa pun. Step ini membuat ServiceAccount sebagai representasi persona dan kemudian menghubungkannya dengan Role atau ClusterRole melalui Binding.

Modelnya:



Empat ServiceAccount digunakan untuk mensimulasikan empat persona:

ds-user → Data Scientist

mle-user → ML Engineer

pe-user → Platform Engineer

intern-user → Intern

```
kubectl create serviceaccount ds-user -n team-ds
```

```
kubectl create serviceaccount mle-user -n team-mle
```

```
kubectl create serviceaccount pe-user -n team-pe
```

```
kubectl create serviceaccount intern-user -n team-intern
```

63.1 Ikat ke Role masing-masing, Data Scientist, ML Engineer, dan Intern menggunakan RoleBinding karena akses mereka hanya diperlukan di namespace masing-masing. Platform Engineer menggunakan ClusterRoleBinding karena tanggung jawabnya bersifat cluster-wide. Ini merupakan perbedaan penting, RoleBinding adalah memberikan permission pada scope tertentu, umumnya satu namespace sedangkan ClusterRoleBinding adalah memberikan permission ClusterRole secara cluster-wide.

```
kubectl create rolebinding ds-binding \
```

```
--role=data-scientist-role --serviceaccount=team-ds:ds-user -n team-ds
```

```
kubectl create rolebinding mle-binding \
```

```
--role=ml-engineer-role --serviceaccount=team-mle:mle-user -n team-mle
```

```
kubectl create clusterrolebinding pe-binding \
```

```
--clusterrole=platform-engineer-role --serviceaccount=team-pe:pe-user
```

```
kubectl create rolebinding intern-binding \
```

```
--role=intern-role --serviceaccount=team-intern:intern-user -n team-intern
```

Catatan: pe-user pakai ClusterRoleBinding (bukan RoleBinding) karena Platform Engineer butuh akses lintas namespace (atur ResourceQuota/Kueue cluster-wide) sesuai definisi ClusterRole di Step 61c.

63.2 Validasi : simulasikan tiap persona pakai kubectl auth can-i. Lolos kalau hasilnya persis sesuai ekspektasi di komentar (yes/no). Ini bukti isolasi RBAC beneran jalan, bukan cuma dipasang.

```
echo "=== Data Scientist: bisa create Pod di namespace sendiri? ==="
```

```
kubectl auth can-i create pods --as=system:serviceaccount:team-ds:ds-user -n team-ds
```

```
echo "=== Data Scientist: bisa create Pod di namespace tim lain? (harus NO) ==="
```

```
kubectl auth can-i create pods --as=system:serviceaccount:team-ds:ds-user -n team-mle
```

```
echo "=== Intern: bisa create Pod? (harus NO, read-only) ==="
```

```
kubectl auth can-i create pods --as=system:serviceaccount:team-
intern:intern-user -n team-intern

echo "=== Platform Engineer: bisa atur ResourceQuota lintas namespace?
(harus YES) ==="

kubectl auth can-i create resourcequotas --
as=system:serviceaccount:team-pe:pe-user -n team-ds
```

Step 64. ResourceQuota per Tim (CPU/Memory/Objek, bukan GPU)

Setelah akses dipisahkan, setiap tenant juga perlu memiliki batas penggunaan resource. Step ini menggunakan Kubernetes ResourceQuota untuk membatasi CPU request, Memory request, CPU limit, Memory limit, Jumlah Pod, GPU sengaja tidak dimasukkan. Alasannya adalah GPU sudah memiliki mekanisme scheduling tersendiri melalui Kueue dari Phase 5.

ResourceQuota

- ├── CPU
- ├── Memory
- └── Pod count

Kueue

- └── GPU

Dengan pemisahan ini tidak terjadi dua sistem yang sama-sama mencoba mengontrol kapasitas GPU.

64.1 Total kalau dijumlah (2+2+1+0.5 = 5.5 CPU) sengaja melebihi kapasitas fisik minikube (4 CPU). ini normal (disebut overcommit), karena nggak semua tim pasti pakai jatah penuhnya barengan. Kalau mau lebih ketat/konservatif, turunkan total supaya ≤ 4 CPU.

```
cat <<EOF > 64-resourcequota.yaml
apiVersion: v1
kind: ResourceQuota
metadata:
  name: ds-quota
  namespace: team-ds
spec:
  hard:
    requests.cpu: "2"
    requests.memory: 4Gi
    limits.cpu: "2"
    limits.memory: 4Gi
    pods: "10"
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: mle-quota
  namespace: team-mle
spec:
  hard:
```

```

        requests.cpu: "2"
        requests.memory: 4Gi
        limits.cpu: "2"
        limits.memory: 4Gi
        pods: "10"
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: pe-quota
  namespace: team-pe
spec:
  hard:
    requests.cpu: "1"
    requests.memory: 2Gi
    limits.cpu: "1"
    limits.memory: 2Gi
    pods: "5"
---
apiVersion: v1
kind: ResourceQuota
metadata:
  name: intern-quota
  namespace: team-intern
spec:
  hard:
    requests.cpu: "500m"
    requests.memory: 512Mi
    limits.cpu: "500m"
    limits.memory: 512Mi
    pods: "3"
EOF

```

64.2 Apply

```
kubectl apply -f 64-resourcequota.yaml
```

64.3 Validasi, Lolos kalau tiap namespace nunjukin Used: 0 dari Hard yang sesuai (belum ada Pod jalan).

```
kubectl describe resourcequota -n team-ds
```

```
kubectl describe resourcequota -n team-mle
```

```
kubectl describe resourcequota -n team-pe
```

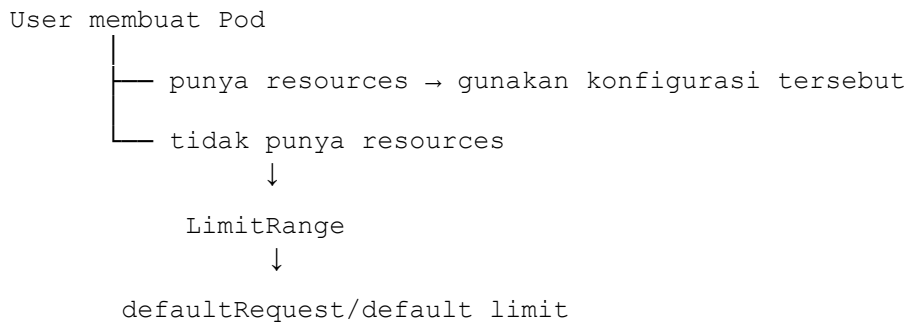
```
kubectl describe resourcequota -n team-intern
```

64.4 Test isolasi (opsional tapi disarankan): coba deploy Pod di team-intern yang minta cpu: 1 (lebih dari quota 500m). Lolos kalau muncul error exceeded quota, bukti ResourceQuota beneran menahan, bukan cuma dekorasi. Hapus lagi setelah tes: kubectl delete pod test-pod -n team-intern --ignore-not-found

```
kubectl run test-pod --image="nginx" -n team-intern --dry-run="client"
-o yaml --overrides="{\"spec\":{\"containers\":[{\"name\":\"test-
pod\",\"image\":\"nginx\",\"resources\":{\"requests\":{\"cpu\":\"1\"}}
}}}" | kubectl apply -f -
```

Step 65. LimitRange per Tim (Default Request/Limit Otomatis)

ResourceQuota membatasi total resource namespace, tetapi belum menentukan apa yang terjadi ketika user membuat container tanpa mendefinisikan request dan limit. LimitRange digunakan sebagai defaulting dan constraint mechanism pada level container. Sederhananya:



Tujuannya mencegah workload tanpa resource specification yang dapat mengonsumsi resource secara tidak terkontrol.

65.1 Setiap tenant diberikan default CPU dan memory yang berbeda berdasarkan kebutuhan dan tingkat privilege. Contohnya, team-intern mendapatkan default yang lebih kecil dibanding Data Scientist atau ML Engineer.

```
cat <<EOF > 65-limitrange.yaml
apiVersion: v1
kind: LimitRange
metadata:
  name: ds-limits
  namespace: team-ds
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: 512Mi
    defaultRequest:
      cpu: "250m"
      memory: 256Mi
---
apiVersion: v1
kind: LimitRange
metadata:
  name: mle-limits
  namespace: team-mle
spec:
  limits:
  - type: Container
    default:
      cpu: "500m"
      memory: 512Mi
```

```

        defaultRequest:
          cpu: "250m"
          memory: 256Mi
---
apiVersion: v1
kind: LimitRange
metadata:
  name: pe-limits
  namespace: team-pe
spec:
  limits:
  - type: Container
    default:
      cpu: "300m"
      memory: 256Mi
    defaultRequest:
      cpu: "100m"
      memory: 128Mi
---
apiVersion: v1
kind: LimitRange
metadata:
  name: intern-limits
  namespace: team-intern
spec:
  limits:
  - type: Container
    default:
      cpu: "200m"
      memory: 256Mi
    defaultRequest:
      cpu: "100m"
      memory: 128Mi
EOF

```

65.2 Apply

```
kubectl apply -f 65-limitrange.yaml
```

65.3 Validasi, Lolos kalau output nunjukin requests/limits udah otomatis keisi (cpu: 100m, memory: 128Mi, dst) padahal tadi nggak lu tulis manual. Hapus lagi: kubectl delete pod no-limit-pod -n team-intern

```
kubectl run no-limit-pod --image=nginx -n team-intern
```

```
kubectl get pod no-limit-pod -n team-intern -o
jsonpath='{.spec.containers[0].resources}'
```

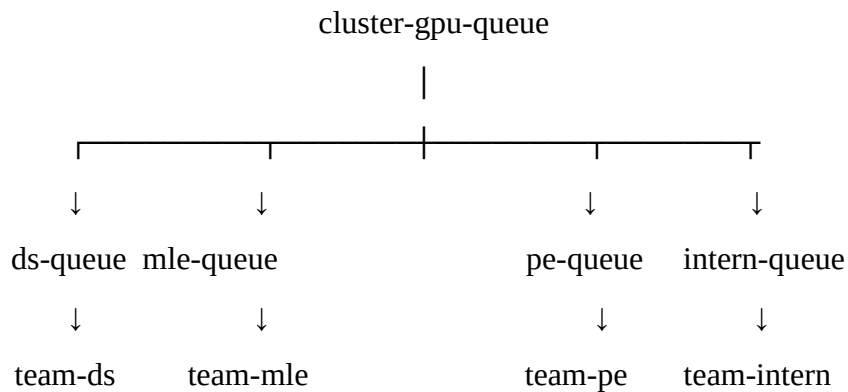
Step 66. Integrasikan Tiap Tim ke Kueue (GPU Multi-Tenant, Lanjutan Phase 5)

Ini merupakan bagian penting dari desain multi-tenant karena GPU tidak lagi hanya dimiliki oleh satu namespace mlops. Phase 5 sebelumnya mempunyai:

cluster-gpu-queue

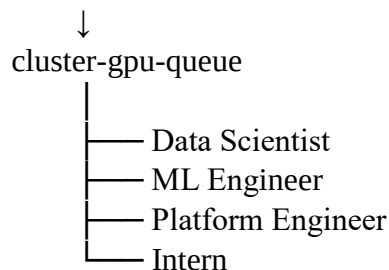
↓
mlops-queue

Pada phase ini diperluas menjadi:



Semua LocalQueue tetap menunjuk ke ClusterQueue yang sama. Dengan demikian, GPU fisik tetap mempunyai satu sumber kapasitas:

GPU physical capacity



66.1 Buat WorkloadPriorityClass WorkloadPriorityClass digunakan untuk menentukan prioritas workload ketika beberapa workload sedang menunggu resource. Dua tingkat priority dibuat high-priority → Platform Engineer dan ML Engineer sedangkan low-priority → Data Scientist dan Intern. Ini memungkinkan workload dengan prioritas lebih tinggi mendapatkan perlakuan berbeda ketika GPU sedang diperebutkan.

```
cat <<EOF > 66a-priority-classes.yaml
apiVersion: kueue.x-k8s.io/v1beta1
kind: WorkloadPriorityClass
metadata:
  name: high-priority
  value: 100
  description: "Platform Engineer & ML Engineer"
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: WorkloadPriorityClass
metadata:
  name: low-priority
  value: 10
  description: "Data Scientist & Intern"
EOF
```

66.2 Apply

```
kubectl apply -f 66a-priority-classes.yaml
```

66.3 Buat LocalQueue per tim, semuanya nempel ke ClusterQueue: cluster-gpu-queue yang udah ada dari Phase 5 (jadi kuota GPU fisik tetap 1 sumber kebenaran, nggak dobel)

```
cat <<EOF > 66b-localqueues.yaml
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  namespace: team-ds
  name: ds-queue
spec:
  clusterQueue: cluster-gpu-queue
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  namespace: team-mle
  name: mle-queue
spec:
  clusterQueue: cluster-gpu-queue
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  namespace: team-pe
  name: pe-queue
spec:
  clusterQueue: cluster-gpu-queue
---
apiVersion: kueue.x-k8s.io/v1beta1
kind: LocalQueue
metadata:
  namespace: team-intern
  name: intern-queue
spec:
  clusterQueue: cluster-gpu-queue
EOF
```

66.4 Apply

```
kubectl apply -f 66b-localqueues.yaml
```

66.5 Preemption mengubah perilaku scheduling ketika resource sedang penuh. Dengan konfigurasi ini, workload dengan priority lebih tinggi dapat menyebabkan workload dengan priority lebih rendah kehilangan resource yang sedang digunakan, sesuai kebijakan preemption Kueue. Secara konseptual, ketika GPU dalam keadaan penuh dan ada workload priority tinggi yang datang saat workload priority rendah sedang berjalan, Kueue akan mengevaluasi aturan preemption terlebih dahulu. Jika memenuhi syarat, Kueue akan menghentikan workload priority rendah tersebut lalu memberikan alokasi GPU-nya ke workload dengan priority yang lebih tinggi. Ini membuat GPU tidak hanya menggunakan FIFO sederhana, tetapi dapat mempertimbangkan priority.


```
kubectl patch clusterqueue cluster-gpu-queue --type='merge' -
p='{ "spec": { "preemption": { "withinClusterQueue": "LowerPriority" } } }'
```

66.6 Validasi memastikan seluruh LocalQueue tenant tersedia dan semuanya mengarah ke cluster-gpu-queue. Hal ini membuktikan bahwa empat tenant tetap berbagi satu pool GPU, bukan mendapatkan GPU quota terpisah yang dapat menyebabkan konfigurasi quota saling bertabrakan.

```
kubectl get localqueue -A
```

Step 67. Simulasi 4 Job dari 4 Tim, Rebutan 1 GPU

Step terakhir merupakan demonstrasi nyata multi-tenant GPU scheduling. Konfigurasi sebelumnya sekarang diuji dalam kondisi kompetisi resource. Karena cluster hanya mempunyai 1 GPU, dua atau lebih workload yang membutuhkan GPU secara bersamaan tidak dapat semuanya berjalan sekaligus. Template workload dibuat agar dapat digunakan oleh seluruh tenant dengan mengganti: Namespace, LocalQueue, PriorityClass, Nama Job. Dengan demikian satu template dapat merepresentasikan berbagai persona.

67.1 67-job-template.yaml (dipakai berulang, ganti NAMESPACE/QUEUE/PRIORITY), Template Job menggunakan GPU request = 1 dan GPU limit = 1 dan menjalankan nvidia-smi kemudian sleep. Suspended: true digunakan karena workload akan menunggu keputusan Kueue sebelum benar-benar mendapatkan resource.

```
cat <<EOF > 67-job-template.yaml
apiVersion: batch/v1
kind: Job
metadata:
  name: JOBNAME
  namespace: NAMESPACE
  labels:
    kueue.x-k8s.io/queue-name: QUEUE
    kueue.x-k8s.io/priority-class: PRIORITY
spec:
  suspend: true
  template:
    spec:
      restartPolicy: Never
      containers:
      - name: sim
        image: nvidia/cuda:12.9.1-base-ubuntu24.04
        command: ["sh", "-c", "nvidia-smi && sleep 60"]
        resources:
          limits: { nvidia.com/gpu: 1 }
          requests: { nvidia.com/gpu: 1 }
```

EOF

67.2 TES SKENARIO BERDARAH: INTERN vs BOS (PE)

67.2.0 Bersihkan sisa-sisa job dari perang sebelumnya sampai akar

```
echo "=== Bersih-bersih arena... ==="
```

```
kubectl delete jobs --all -n team-intern --cascade=foreground
```

```

kubect1 delete jobs --all -n team-pe --cascade=foreground
kubect1 delete jobs --all -n team-mle --cascade=foreground
kubect1 taint nodes --all node-role.kubernetes.io/control-plane-2>/dev/null || true

sleep 5

# 67.2.1 Submit Job Intern (low-priority), Karena GPU masih kosong,
workload tersebut dapat memperoleh GPU dan mulai berjalan. low-
priority → intern-queue → cluster-gpu-queue → GPU tersedia → Admitted

echo -e "\n=== 1. Si Intern Datang Duluan ==="

sed -e 's/JOBNAME/job-intern-long/' -e 's/NAMESPACE/team-intern/' -e
's/QUEUE/intern-queue/' -e 's/PRIORITY/low-priority/' 67-job-
template.yaml | kubect1 apply -f -

# 67.2.2 Tunggu 10 detik biar Job Intern benar-bener nge-lock GPU,
kemudian memeriksa Kueue Workload dan Pod pada team-intern. Tujuannya
memastikan low-priority workload benar-benar sudah mendapatkan GPU
sebelum workload prioritas tinggi masuk.

echo "=== Nunggu 10 detik biar Intern duduk manis... ==="

sleep 10

echo -e "\n=== Status Sebelum PE Datang (Intern Harus Admitted: True)
=== "

kubect1 get workloads -A

kubect1 get pods -n team-intern

# 67.2.3 Datang Bos (PE) dengan Job urgent (high-priority)

echo -e "\n=== 2. Bos (PE) Datang Bawa Tiket VIP! (PREEMPTION DIMULAI)
=== "

sed -e 's/JOBNAME/job-pe-urgent/' -e 's/NAMESPACE/team-pe/' -e
's/QUEUE/pe-queue/' -e 's/PRIORITY/high-priority/' 67-job-
template.yaml | kubect1 apply -f -

# 67.2.4 Pantau pembantaian secara real-time, Ini menjadi demonstrasi
akhir bahwa Tenant memiliki queue masing-masing, Semua queue
menggunakan GPU pool yang sama, Kueue mengetahui priority workload,
GPU yang terbatas dapat menyebabkan workload masuk antrean, Priority
dapat memengaruhi workload mana yang mendapatkan resource, Isolasi
namespace dan RBAC tetap berjalan bersamaan dengan GPU scheduling.

echo -e "\n=== 3. PANTAU PEMBANTAIAN REAL-TIME (Tekan Ctrl+C untuk
keluar) ==="

kubect1 get workloads -A -w

```

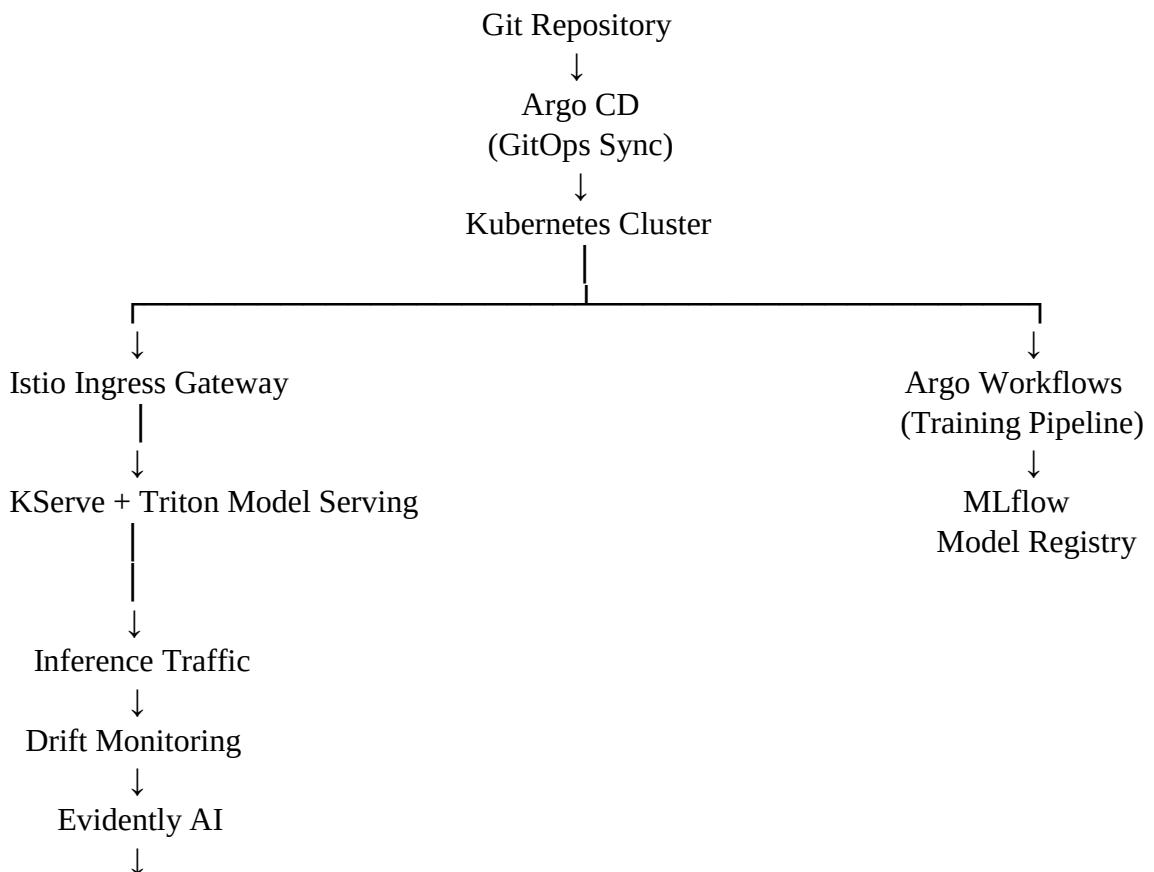
Phase 8 — GitOps, Model Serving & Automated MLOps Operations

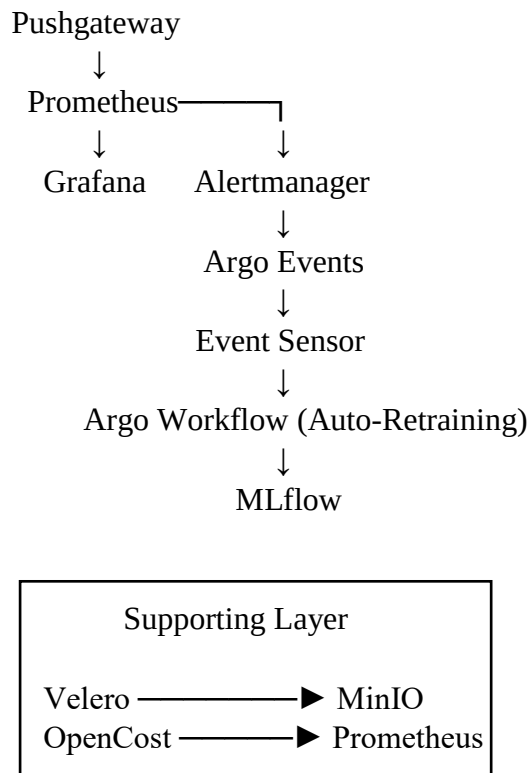
Phase ini memperluas platform dari sistem training dan monitoring menjadi platform MLOps yang memiliki deployment management, model serving, drift monitoring, automated retraining, disaster recovery, dan cost visibility. Fokus utama fase ini adalah menghubungkan lifecycle aplikasi dan model melalui dua jalur automation:

- GitOps deployment path : Perubahan konfigurasi pada repository Git direkonsiliasi secara otomatis oleh Argo CD ke Kubernetes.
- MLOps automation path : Model yang berjalan di production dipantau menggunakan Evidently AI. Ketika drift terdeteksi, metric dikirim ke Prometheus, alert diteruskan oleh Alertmanager ke Argo Events, lalu Argo Events memicu kembali pipeline training melalui Argo Workflows.

Pada fase ini, Istio digunakan sebagai ingress dan traffic-routing layer, sedangkan KServe + Triton digunakan untuk menjalankan model inference melalui API yang terstandarisasi. Selain itu, Velero digunakan untuk backup dan restore resource Kubernetes ke MinIO, sementara OpenCost digunakan untuk memberikan estimasi penggunaan resource dan biaya berdasarkan namespace. Phase ini telah membentuk automated retraining loop, tetapi belum sepenuhnya menjadi closed-loop deployment karena model hasil retraining belum otomatis divalidasi, dipromosikan, dan di-rollout kembali ke production.

Target Architecture:





Arsitektur Phase 8 terdiri dari dua automation path utama.

GitOps path: Git Repository → Argo CD → Kubernetes Cluster

Path ini memastikan konfigurasi deployment yang berjalan di cluster tetap sesuai dengan desired state yang didefinisikan di Git.

MLOps automation path: Model Serving → Drift Monitoring → Alerting → Argo Events → Retraining → MLflow

Path ini memungkinkan pipeline training dipicu berdasarkan kondisi data tanpa harus dijalankan secara manual. Hasil retraining pada fase ini disimpan dan diregistrasikan ke MLflow. Proses validasi model, approval, promotion, dan deployment otomatis ke production belum termasuk dalam automation loop.

Step 68. Setup Argo CD (GitOps)

Tahap ini memperkenalkan Argo CD sebagai control plane untuk deployment berbasis GitOps. Sebelum tahap ini, deployment Kubernetes masih dapat dilakukan secara imperative menggunakan perintah seperti `kubectl apply`. Pada tahap ini, konfigurasi deployment dipindahkan ke repository Git dan dikelola sebagai desired state. Argo CD bertugas melakukan reconciliation antara konfigurasi yang tersimpan di Git dan resource yang berjalan di Kubernetes. Jika terdapat perubahan pada repository, Argo CD akan melakukan sinkronisasi ke cluster. Jika terdapat perubahan manual pada resource yang dikelola Argo CD, konfigurasi tersebut dapat dikembalikan ke state yang sesuai dengan repository.

68.1 Menyiapkan Namespace dan Instalasi Argo CD, Namespace argocd digunakan sebagai lokasi deployment seluruh komponen Argo CD. Pemisahan namespace ini bertujuan untuk: menjaga komponen GitOps tetap terisolasi dari workload aplikasi; memudahkan monitoring dan troubleshooting; memisahkan control plane deployment dari workload MLOps; menyederhanakan pengelolaan resource Argo CD.

```
kubectl create namespace argocd
```

```
kubectl apply -n argocd -f  
https://raw.githubusercontent.com/argoproj/argo-  
cd/stable/manifests/install.yaml
```

```
kubectl wait --for=condition=ready pod -l  
app.kubernetes.io/name=argocd-server -n argocd --timeout=300s
```

68.2 Mengambil Password Admin Awal dan Meng-install Argo CD CLI, Setelah Argo CD berhasil di-install, password awal user admin dapat diambil dari Secret Kubernetes. dilanjutkan dengan Install Argo CD CLI sekaligus Membuka Akses Argo CD melalui Port Forward

```
ARGOCD_PASS=$(kubectl -n argocd get secret argocd-initial-admin-secret  
-o jsonpath="{.data.password}" | base64 -d)
```

```
echo "Password: $ARGOCD_PASS"
```

```
curl -sSL -o argocd-linux-amd64 https://github.com/argoproj/argo-  
cd/releases/latest/download/argocd-linux-amd64
```

```
sudo install -m 555 argocd-linux-amd64 /usr/local/bin/argocd
```

```
rm argocd-linux-amd64
```

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
```

68.3 Di terminal lain (jangan dimatiin sampe step 68 selesai)

```
argocd login localhost:8080 --username admin --password "$ARGOCD_PASS"  
--insecure
```

68.4 Repository GitOps digunakan sebagai sumber konfigurasi deployment. Tanpa repository Git, Argo CD tidak memiliki desired state yang dapat direkonsiliasi. Argo CD bukan pengganti kubectl apply manual; Argo CD melakukan sinkronisasi berdasarkan manifest yang tersimpan di Git. Manifest dari fase sebelumnya, termasuk training pipeline; konfigurasi Kueue; monitoring stack; namespace dan resource multi-tenancy; konfigurasi workload MLOps; dikumpulkan ke dalam struktur repository yang lebih terorganisasi.

```
cat > ~/ai-factory/setup-gitops.sh <<'EOF'  
#!/bin/bash  
set -e  
SOURCE="$HOME/ai-factory"  
DEST="$HOME/ai-factory/gitops-repo"  
echo "=== Creating GitOps repository structure ==="  
rm -rf "$DEST"  
mkdir -p \  
    "$DEST/phase4" \  
    "$DEST/phase5/kueue" \  
    "$DEST/phase6/monitoring" \  
'EOF'
```

```

"$DEST/phase7/multi-tenant"
echo "=== Copy Phase 4 manifests ==="
cp "$SOURCE/phase4/minio-values.yaml" \
  "$DEST/phase4/"
cp "$SOURCE/phase4/mlflow.yaml" \
  "$DEST/phase4/"
cp "$SOURCE/phase4/mlflow-service.yaml" \
  "$DEST/phase4/"
cp "$SOURCE/phase4/ml-project/argo-training-workflow.yaml" \
  "$DEST/phase4/"
echo "=== Copy Phase 5 Kueue manifests ==="
cp "$SOURCE/phase5/kueue/"*.yaml \
  "$DEST/phase5/kueue/"
echo "=== Copy Phase 6 monitoring manifests ==="
cp "$SOURCE/phase6/monitoring/"*.yaml \
  "$DEST/phase6/monitoring/"
echo "=== Copy Phase 7 multi-tenant manifests ==="
cp "$SOURCE/phase7/multi-tenant/"*.yaml \
  "$DEST/phase7/multi-tenant/"
echo
echo "=== GitOps repository created ==="
echo
tree "$DEST"
echo
echo "=== Checking for obvious secret files ==="
find "$DEST" -type f \
  \( -name "*.env" -o -name "*secret*" -o -name "*password*" \) \
  -print
echo
echo "Done."
EOF

```

```

chmod +x ~/ai-factory/setup-gitops.sh

```

```

~/ai-factory/setup-gitops.sh

```

68.5 buka github lu, buka settings > SSH and GPG keys > new SSH key

68.6 ketika di terminal → ssh-keygen -t ed25519 -C "412biyubiy@github.com"

68.7 cat <folder private key disimpan> lalu copy key nya

68.8 buka github yg tadi, title : Laptop-GitOps, Key : <paste>

68.9

```

cd ~/ai-factory/gitops-repo
git init
git branch -M main
git add .
git commit -m "init gitops repo"

```

Ganti pake username dan repo GitHub lu yg masih kosong

```

git remote set-url origin git@github.com:412biyubiy/Enterprise-NVIDIA-
AI-Factory-with-MLOps-Platform.git

```

```
git push -u origin main
```

```
kubectl create namespace mlops --dry-run=client -o yaml | kubectl  
apply -f -
```

68.10 Daftarkan sebagai Application di ArgoCD, Konfigurasi tersebut memiliki arti: --repo: repository Git yang menjadi sumber manifest; --path phase4: direktori manifest yang digunakan; --dest-server: cluster Kubernetes tujuan; --dest-namespace mlops: namespace target; --sync-policy automated: sinkronisasi otomatis; --self-heal: koreksi otomatis terhadap perubahan manual; --sync-option CreateNamespace=true: membuat namespace jika belum tersedia. Jika repository menggunakan struktur multi-directory dan membutuhkan seluruh fase, Application dapat dibuat menggunakan path root atau Application tambahan per fase. Untuk validasi awal, Application ai-factory-phase4 digunakan terlebih dahulu agar scope deployment tetap terkontrol.

```
argocd app create ai-factory-phase4 \  
  
  --repo https://github.com/412biyubiy/Enterprise-NVIDIA-AI-Factory-  
with-MLOps-Platform.git \  
  
  --path phase4 \  
  
  --dest-server https://kubernetes.default.svc \  
  
  --dest-namespace mlops \  
  
  --sync-policy automated \  
  
  --self-heal \  
  
  --sync-option CreateNamespace=true
```

68.11 Validasi. Lolos kalau semua pod argocd-* Running, argocd app list nunjukin ai-factory-phase4 dengan status Synced dan Healthy. Coba juga: edit manual mlflow.yaml di cluster lewat kubectl edit, tunggu ~3 menit, ArgoCD harus otomatis ngebalikin ke versi yang ada di Git (bukti "GitOps beneran jalan", bukan cuma alat sync sekali doang).

68.11.1 Cek kesehatan Pod ArgoCD

```
kubectl get pods -n argocd
```

68.11.2 Trigger sync pertama agar langsung Synced tanpa nunggu 3 menit

```
argocd app sync ai-factory-phase4
```

68.11.3 Cek status aplikasi

```
argocd app list
```

```
argocd app get ai-factory-phase4
```

68.11.4 Tes GitOps Auto-Healing (Koreksi Otomatis, misal ubah replicas

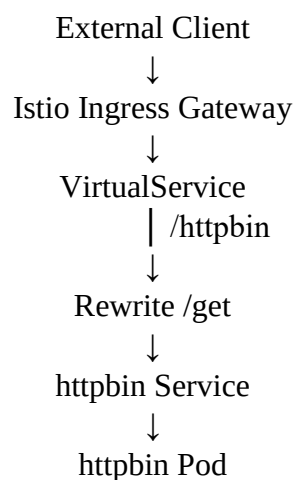
```
kubectl edit deployment mlflow -n mlops
```

Cek apakah ArgoCD merevisi balik ke versi Git secara otomatis

```
argocd app get ai-factory-phase4
```

Step 69. Istio Ingress Gateway (Smoke Test Dulu Sebelum Dipakai buat Model)

Step ini menyiapkan Istio sebagai entry point traffic HTTP sekaligus routing layer sebelum model serving dipasang. Namespace workload diberi label Istio injection agar pod yang relevan mendapatkan Envoy sidecar sebagai data plane. Selanjutnya dibuat Istio Gateway sebagai endpoint penerima traffic dari luar cluster dan VirtualService untuk menentukan bagaimana request diarahkan ke service internal. httpbin digunakan sebagai smoke-test workload sehingga jalur network dapat diuji terlebih dahulu tanpa bergantung pada KServe atau Triton. Routing diuji melalui port-forward menuju istio-ingressgateway, kemudian request /httpbin diarahkan oleh VirtualService ke service httpbin dan di-rewrite menjadi /get. Alur step ini adalah:



Validasi memastikan request berhasil melewati Ingress Gateway, routing VirtualService bekerja, URI rewrite berjalan, dan response berhasil dikembalikan ke client. # 69.1 Pastikan istio sudah terinstall

```
istioctl version
```

69.2 Enable Istio Sidecar Injection pada namespace mlops & restart deployment terkait, Namespace yang membutuhkan kemampuan service mesh diintegrasikan dengan Istio sidecar injection. Dengan konfigurasi tersebut, workload yang berjalan di namespace terkait akan mendapatkan Envoy sidecar secara otomatis. Envoy bertugas menangani komunikasi network sesuai konfigurasi Istio. Tidak semua namespace harus menggunakan sidecar injection. Aktivasi dilakukan secara selektif agar overhead resource tetap terkendali dan hanya workload yang membutuhkan service-mesh capability yang menggunakannya.

```
kubectl label namespace mlops istio-injection=enabled --overwrite
```

```
kubectl rollout restart deployment -n mlops
```

69.3 Buat Istio Ingress Gateway manifest, Istio Gateway digunakan sebagai entry point HTTP untuk traffic dari luar cluster. Gateway mendefinisikan: port yang dibuka; protocol yang digunakan; host yang diterima; selector yang menghubungkan Gateway dengan Istio Ingress Gateway. Gateway hanya mendefinisikan bagaimana traffic masuk ke mesh. Routing request ke service tertentu dilakukan melalui VirtualService.


```

cat <<EOF > gateway.yaml
apiVersion: networking.istio.io/v1alpha3
kind: Gateway
metadata:
  name: ai-factory-gateway
  namespace: mlops
spec:
  selector:
    istio: ingressgateway
  servers:
  - port:
      number: 80
      name: http
      protocol: HTTP
    hosts: ["*"]
EOF

```

69.4 Apply Gateway

```
kubectl apply -f gateway.yaml
```

69.5 Smoke test setup: Deploy httpbin berbasis Go (lebih cepat, ringan, dan stabil dibanding sample resmi Python), httpbin digunakan untuk menguji routing sebelum model serving dipasang. Ini membantu memisahkan masalah Istio dari masalah KServe atau Triton.

Hapus webhook kueue agar tidak memblokir deployment jika ada

```
kubectl delete mutatingwebhookconfiguration kueue-mutating-webhook-configuration --ignore-not-found
```

```
kubectl delete validatingwebhookconfiguration kueue-validating-webhook-configuration --ignore-not-found
```

```

cat <<EOF | kubectl apply -n mlops -f -
apiVersion: apps/v1
kind: Deployment
metadata:
  name: httpbin
spec:
  replicas: 1
  selector:
    matchLabels:
      app: httpbin
      version: v1
  template:
    metadata:
      labels:
        app: httpbin
        version: v1
    spec:
      containers:
      - image: mccutchen/go-httpbin:v2.14.0
        imagePullPolicy: IfNotPresent
        name: httpbin
        ports:
        - containerPort: 8080

```

```

apiVersion: v1
kind: Service
metadata:
  name: httpbin
spec:
  ports:
    - name: http
      port: 8000
      targetPort: 8080
  selector:
    app: httpbin
EOF

```

69.6 Buat VirtualService dengan FQDN untuk routing /httpbin -> /get di port 8000

```

cat <<EOF > httpbin-vs.yaml
apiVersion: networking.istio.io/v1alpha3
kind: VirtualService
metadata:
  name: httpbin-test
  namespace: mlops
spec:
  hosts: ["*"]
  gateways: ["ai-factory-gateway"]
  http:
    - match: [{uri: {prefix: "/httpbin"}}]
      rewrite: {uri: "/get"}
      route:
        - destination: {host: httpbin.mlops.svc.cluster.local, port:
{number: 8000}}
EOF

```

69.7 Apply VirtualService

```
kubectl apply -f httpbin-vs.yaml
```

69.8 Test Port-Forward & Curl

Terminal 1:

```
kubectl port-forward -n istio-system svc/istio-ingressgateway 8081:80
```

Terminal 2:

```
curl -v http://localhost:8081/httpbin
```

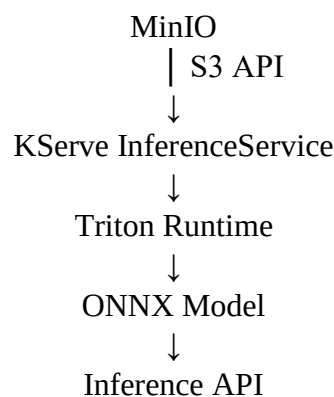
69.9 Validasi Hasil:

```
# Smoke test LOLOS jika mendapatkan status "HTTP/1.1 200 OK", header
"server: istio-envoy",
```

```
# serta URL response berubah/ter-rewrite menjadi
"http://localhost:8081/get".
```

STEP 70. Model Serving dengan KServe (RawDeployment Mode)

Step ini mengubah model hasil training menjadi inference service yang dapat diakses melalui API. KServe digunakan sebagai abstraction dan lifecycle layer untuk deployment model, sedangkan Triton digunakan sebagai inference runtime yang benar-benar melakukan proses inferensi. KServe terlebih dahulu dipasang bersama cert-manager dan dikonfigurasi menggunakan RawDeployment agar workload serving berjalan sebagai Kubernetes Deployment biasa tanpa bergantung pada serverless runtime. Model .joblib kemudian dikonversi menjadi ONNX supaya dapat dijalankan oleh Triton melalui ONNX Runtime. Model ONNX ditempatkan ke dalam struktur Triton Model Repository, yang minimal terdiri dari nama model, version directory, model file, dan config.pbtxt. config.pbtxt mendefinisikan parameter runtime seperti nama model, backend/platform, max_batch_size, nama input, datatype, dimensi input, serta output model. Repository tersebut kemudian di-upload ke MinIO melalui S3 API. Kubernetes Secret dan ServiceAccount digunakan agar KServe dapat mengambil artifact model dari MinIO tanpa memasukkan credential langsung ke konfigurasi model. InferenceService kemudian menghubungkan seluruh komponen:



Request inference menggunakan Triton V2 Inference Protocol, sehingga payload harus sesuai dengan metadata model, terutama nama input, datatype, shape, dan data yang dikirim. Validasi memastikan InferenceService berhasil dibuat, predictor pod berjalan, Triton berhasil me-load model, endpoint tersedia, dan request inference menghasilkan output yang valid.

70.1 Prasyarat, Install cert-manager, cert-manager menyediakan mekanisme certificate management yang dibutuhkan komponen tertentu dalam Kubernetes/KServe. Parameter --timeout=180s membatasi waktu tunggu. sleep 15 memberikan waktu tambahan bagi webhook/controller untuk benar-benar siap setelah kondisi Pod berubah.

```
kubectl apply -f https://github.com/cert-manager/cert-
manager/releases/download/v1.14.0/cert-manager.yaml
```

```
echo "Menunggu cert-manager webhook server aktif sempurna..."
```

```
kubectl wait --for=condition=ready pod -l app=cert-manager -n cert-
manager --timeout=180s
```

```
sleep 15
```

70.2 Install KServe (RawDeployment mode, Server-side apply membuat Kubernetes API server menangani field ownership dan merge manifest. KServe memiliki beberapa mode deployment, RawDeployment menggunakan

resource Kubernetes seperti Deployment, Service, Pod secara langsung. Ini lebih sederhana untuk environment lab karena tidak membutuhkan Knative sebagai serving layer.

```
kubectl apply --server-side -f
https://github.com/kserve/kserve/releases/download/v0.14.0/kserve.yaml
```

```
kubectl patch configmap/inferenceservice-config -n kserve --type merge \
```

```
\-p '{"data": {"deploy": "{\\"defaultDeploymentMode\\":
\\"RawDeployment\\"}}}'
```

```
kubectl rollout restart deployment kserve-controller-manager -n kserve
```

```
kubectl rollout status deployment kserve-controller-manager -n kserve
-w
```

70.3 Konversi Model .joblib ke ONNX & Auto-Generate Config Triton, n_features_in_ digunakan supaya jumlah feature tidak di-hardcode. Misalnya model mempunyai 8 features, maka ONNX input [None, 8]. Jika model mempunyai 12 features, maka [None, 12]. Alur validasi shape: Scikit-learn → n_features_in_ → ONNX input shape → Triton config.pbtxt → Inference request. Semua harus konsisten.

```
cd ~/ai-factory/phase4/ml-project
```

```
source ../.venv/bin/activate
```

```
pip install skl2onnx onnx boto3
```

```
python3 << 'EOF'
import joblib
import numpy as np
from skl2onnx import to_onnx
from skl2onnx.common.data_types import FloatTensorType
model = joblib.load("models/random_forest.joblib")
n_features = getattr(model, "n_features_in_", 8)
initial_type = [('float_input', FloatTensorType([None, n_features]))]
# Paksa target_opset ke 13 agar kompatibel dengan Triton Opset
Runtime!
onnx_model = to_onnx(model, initial_types=initial_type,
target_opset=13)
with open("model.onnx", "wb") as f:
    f.write(onnx_model.SerializeToString())
print(f"☑ Model berhasil dikonversi ulang ke ONNX (Opset 13) dengan
{n_features} features!")
EOF
```

70.4 Pastikan port-forward MinIO berjalan di background (atau buka terminal terpisah dan jgn dimatiin sampe step 70 selesai)

```
kubectl port-forward -n mlops svc/minio 9000:9000 &
```

70.5 Buat struktur folder repositori Triton, angka 1 adalah untuk nomer version, misalkan housing-model/1/model.onnx, nanti kalo ada versi 2 tinggal bikin folder baru

```
mkdir -p triton-repo/housing-model/1
```

```
mv model.onnx triton-repo/housing-model/1/model.onnx
```

70.6 Buat config.pbtxt yang valid & presisi (Nama input HARUS 'float_input') Parameter beserta Fungsinya: name → Nama; model → Triton; platform → Backend/runtime model; max_batch_size → Batch maksimum; input.name → Nama input; data_type → Tipe data; dims → Dimensi tensor; output.name → Nama output. Catatan: Nilai dims: [12] harus sama dengan jumlah feature aktual model. Jika model memiliki 8 feature dims: [8], Jika model memiliki 12 dims: [12] Jangan menganggap 12 selalu benar.

```
cat << 'EOF' > triton-repo/housing-model/config.pbtxt
name: "housing-model"
platform: "onnxruntime_onnx"
max_batch_size: 8
input [
  {
    name: "float_input"
    data_type: TYPE_FP32
    dims: [ 12 ]
  }
]
output [
  {
    name: "variable"
    data_type: TYPE_FP32
    dims: [ 1 ]
  }
]
EOF
```

70.7 Export Environment Variables, Variable dan Fungsinya :
AWS_ACCESS_KEY_ID → Access key S3-compatible; AWS_SECRET_ACCESS_KEY → Secret key; MLFLOW_S3_ENDPOINT_URL → Endpoint object storage. Untuk Kubernetes workload, credential sebaiknya disimpan dalam Kubernetes Secret atau secret-management system, bukan plaintext di Git.

```
export AWS_ACCESS_KEY_ID=minioadmin
export AWS_SECRET_ACCESS_KEY=minioadmin123
export MLFLOW_S3_ENDPOINT_URL=http://localhost:9000
```

70.8 Script Python Upload Rapi ke MinIO, v1 pada path S3 digunakan sebagai logical model release. Sedangkan: housing-model/1 merupakan model version directory Triton. Jadi ada dua konsep versioning yang berbeda.

```
python3 << 'EOF'
import boto3, os
from botocore.client import Config
s3 = boto3.client(
    "s3",
    endpoint_url=os.environ["MLFLOW_S3_ENDPOINT_URL"],
    aws_access_key_id=os.environ["AWS_ACCESS_KEY_ID"],
    aws_secret_access_key=os.environ["AWS_SECRET_ACCESS_KEY"],
    config=Config(signature_version='s3v4'),
    region_name='us-east-1'
```

```

)
buckets = [b['Name'] for b in s3.list_buckets()['Buckets']]
if 'models' not in buckets:
    s3.create_bucket(Bucket='models')
    print("☑ Bucket 'models' berhasil dibuat.")

for root, _, files in os.walk("triton-repo"):
    for f in files:
        local = os.path.join(root, f)
        key = os.path.relpath(local, "triton-repo")
        s3.upload_file(local, "models", f"v1/{key}")
        print("☑ Uploaded:", f"v1/{key}")

```

EOF

70.9 Cleanup Triton lama & Deploy Secret + ServiceAccount + InferenceService, kind: Secret menyimpan credential yang dibutuhkan KServe untuk mengakses MinIO. kind: ServiceAccount memberikan identity kepada workload KServe. kind: InferenceService adalah custom resource KServe yang mendefinisikan model serving. Parameter beserta Fungsinya: deploymentMode → Cara KServe melakukan deployment; serviceAccountName → Identity workload; modelFormat.name → Format/runtime model; runtime → Runtime container; storageUri → Lokasi model; requests.cpu → CPU minimum; requests.memory → Memory minimum; limits.cpu → CPU maksimum; limits.memory → Memory maksimum.

```
kubectl delete deployment triton -n ai --ignore-not-found
```

```
kubectl delete service triton -n ai --ignore-not-found
```

```

cat <<EOF | kubectl apply -f -
apiVersion: v1
kind: Secret
metadata:
  name: kserve-minio-creds
  namespace: mlops
  annotations:
    serving.kserve.io/s3-endpoint:
"minio.mlops.svc.cluster.local:9000"
    serving.kserve.io/s3-usehttps: "0"
    serving.kserve.io/s3-verifyssl: "0"
type: Opaque
stringData:
  AWS_ACCESS_KEY_ID: minioadmin
  AWS_SECRET_ACCESS_KEY: minioadmin123
---
apiVersion: v1
kind: ServiceAccount
metadata:
  name: kserve-sa
  namespace: mlops
secrets:
  - name: kserve-minio-creds
---
apiVersion: "serving.kserve.io/v1beta1"
kind: "InferenceService"
metadata:

```

```

    name: "housing-model-v1"
    namespace: "mlops"
    annotations:
      serving.kserve.io/deploymentMode: "RawDeployment"
spec:
  predictor:
    serviceAccountName: kserve-sa
  model:
    modelFormat:
      name: triton
    runtime: kserve-tritonserver
    storageUri: "s3://models/v1"
    resources:
      requests:
        cpu: 200m
        memory: 512Mi
      limits:
        cpu: 500m
        memory: 1Gi
EOF

```

70.10 Validasi Deployment KServe

```

kubectl get inferencservice -n mlops

kubectl get pods -n mlops -l
serving.kserve.io/inferencservice=housing-model-v1

```

70.11 Jalankan port-forward di terminal terpisah

```

kubectl port-forward -n mlops svc/housing-model-v1-predictor 8082:80

```

70.12 Tembak payload dengan nama input "float_input" (bukan "X")

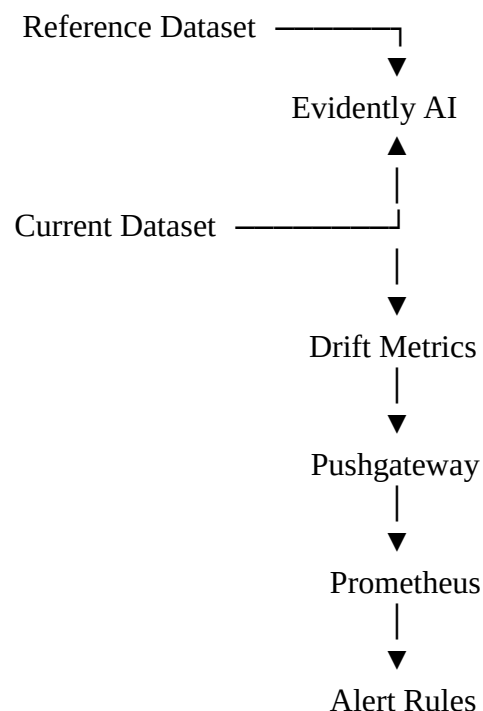
```

curl -X POST http://localhost:8082/v2/models/housing-model/infer \
-H "Content-Type: application/json" \
-d '{
  "inputs": [
    {
      "name": "float_input",
      "shape": [1, 12],
      "datatype": "FP32",
      "data": [0.5, 0.2, 0.1, 0.8, 0.4, 0.3, 0.9, 0.6, 0.2, 0.1,
0.5, 0.4]
    }
  ]
}'

```

Step 71 . Monitoring Data & Concept Drift (Evidently AI)

Step ini membangun data drift monitoring pipeline yang membandingkan data reference dengan data current secara berkala. Reference dataset merepresentasikan distribusi data yang digunakan sebagai baseline, sedangkan current dataset merepresentasikan data terbaru yang masuk ke sistem. Evidently menjalankan pemeriksaan terhadap perubahan distribusi feature dan menghasilkan informasi seperti jumlah feature yang mengalami drift serta proporsi feature yang berubah. Hasil tersebut kemudian diterjemahkan menjadi Prometheus metrics menggunakan Pushgateway. Drift checker dikemas sebagai container image dan dijalankan melalui Kubernetes CronJob. CronJob menentukan frekuensi pemeriksaan, sedangkan PVC menyediakan lokasi persistent untuk menyimpan reference.csv dan current.csv. Alurnya adalah:



Metric utama yang dihasilkan mencakup status dataset drift, persentase feature yang mengalami drift, jumlah feature yang drift, dan timestamp pemeriksaan terakhir. Prometheus kemudian menggunakan metric tersebut untuk menentukan apakah threshold drift telah terlampaui. Selain alert drift, metric timestamp digunakan untuk mendeteksi apabila drift checker sendiri berhenti berjalan.

71.1 Install Pushgateway, Pushgateway menjadi intermediary: Batch Job -push→ Pushgateway -Scrape→ Prometheus. Pushgateway cocok untuk metric yang dihasilkan oleh short-lived batch job seperti drift checker.

```
helm install pushgateway prometheus-community/prometheus-pushgateway \
  -n monitoring \
  --set serviceMonitor.enabled=true \
  --set serviceMonitor.namespace=monitoring \
  --set serviceMonitor.additionalLabels.release=prometheus
```


71.2 Membuat Script Drift Checker, Script melakukan: Load Reference
 → Load Current → Evidently Comparison → Calculate Drift → Create
 Prometheus Metrics → Push to Pushgateway. Metric yang dibuat:
 model_dataset_drift, model_data_drift_share,
 model_drifted_columns_count,
 model_drift_check_last_run_timestamp_seconds. Metric dan Fungsinya:

model_dataset_drift → Status drift

model_data_drift_share → Proporsi feature drift

model_drifted_columns_count → Jumlah feature drift

last_run_timestamp → Waktu terakhir checker berjalan. Penjelasan
 Parameter: reference_data adalah baseline; current_data adalah data
 terbaru; DataDriftPreset() adalah mendefinisikan pemeriksaan drift.
 Definisi Threshold: Misalnya dataset_drift = 1 if drift_share >= 0.5
 else 0, berarti dataset dianggap drift jika minimal 50% feature
 mengalami drift berdasarkan definisi metric tersebut. Threshold harus
 konsisten dengan threshold yang digunakan Alertmanager/Prometheus.

```
cat <<EOF > drift-check.py
import os
import sys
import time
import pandas as pd
# Import Evidently 0.7.x resmi
from evidently import Report
from evidently.preset import DataDriftPreset
# Import Prometheus Client
from prometheus_client import CollectorRegistry, Gauge,
push_to_gateway
def run_drift_check():
    ref_path = "/data/reference.csv"
    cur_path = "/data/current.csv"
    if not os.path.exists(ref_path) or not os.path.exists(cur_path):
        print(f"✗ Error: File CSV tidak ditemukan di path: {ref_path}
atau {cur_path}")
        sys.exit(1)
    print("📂 Loading datasets...")
    reference_data = pd.read_csv(ref_path)
    current_data = pd.read_csv(cur_path)
    print("📊 Calculating Data Drift using Evidently AI (0.7.21)...")
    report = Report(metrics=[DataDriftPreset()])
    # Executing report -> mengembalikan objek Snapshot
    snapshot = report.run(reference_data=reference_data,
current_data=current_data)
    # Extract hasil via snapshot.dict()
    results = snapshot.dict()
    drift_val = results["metrics"][0]["value"]
    number_of_drifted_columns = int(drift_val["count"])
    drift_share = float(drift_val["share"])
    # Dataset drift dianggap 1 (True) jika drift_share >= 50% (0.5)
    dataset_drift = 1 if drift_share >= 0.5 else 0
    print("✅ Calculation finished successfully!")
    print(f"    - Dataset Drift: {dataset_drift}")
```

```

print(f"    - Drift Share: {drift_share:.4f}")
print(f"    - Drifted Columns: {number_of_drifted_columns}")
# Push Metrics ke Prometheus Pushgateway
gateway_url = os.getenv(
    "PUSHGATEWAY_URL",
    "http://pushgateway-prometheus-
pushgateway.monitoring.svc.cluster.local:9091"
)
# Jaga-jaga jika env var PUSHGATEWAY_URL diisi tanpa http://
if not gateway_url.startswith("http://") and not
gateway_url.startswith("https://"):
    gateway_url = f"http://{gateway_url}"
registry = CollectorRegistry()
g_drift = Gauge("model_dataset_drift", "Dataset drift status
(1=True, 0=False)", registry=registry)
g_share = Gauge("model_data_drift_share", "Share of drifted
features", registry=registry)
g_cols = Gauge("model_drifted_columns_count", "Number of drifted
columns", registry=registry)
g_last_run = Gauge("model_drift_check_last_run_timestamp_seconds",
"Timestamp of last run", registry=registry)
g_drift.set(dataset_drift)
g_share.set(drift_share)
g_cols.set(number_of_drifted_columns)
g_last_run.set(time.time())
print(f"🚀 Pushing metrics to Pushgateway ({gateway_url})...")
try:
    push_to_gateway(gateway_url, job="drift_check_cronjob",
registry=registry)
    print("🎉 Metrics successfully pushed to Prometheus!")
except Exception as e:
    print(f"❌ Failed to push metrics to Pushgateway: {e}")
    sys.exit(1)
if __name__ == "__main__":
    run_drift_check()
EOF

```

71.3 Membuat Image Drift Checker, imagePullPolicy: Never Karena image berada di Docker/Minikube lokal. Kubernetes tidak perlu mencoba Docker Hub untuk mengambil image tersebut.

```

cat <<EOF > Dockerfile.drift
FROM python:3.11-slim
RUN pip install --no-cache-dir pandas evidently prometheus_client
COPY drift-check.py /app/drift-check.py
WORKDIR /app
CMD ["python", "drift-check.py"]
EOF

```

```

docker build --no-cache -t drift-check:v1 -f Dockerfile.drift . &&
minikube image load drift-check:v1

```

71.4 Membuat CronJob, Parameter beserta Fungsinya: schedule → Frekuensi execution; concurrencyPolicy → Mencegah job overlap; successfulJobsHistoryLimit → Jumlah history sukses; failedJobsHistoryLimit → Jumlah history gagal; restartPolicy →

Behavior restart; imagePullPolicy → Cara image diambil; **mountPath** → Lokasi dataset dalam container. ***/10 * * * *** Artinya setiap 10 menit **Forbid**. Jika job sebelumnya belum selesai: Job A running → jadwal berikutnya → Job B tidak dibuat Ini mencegah overlapping drift checks.

```
cat <<EOF > drift-cronjob.yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: drift-check
  namespace: mlops
spec:
  schedule: "*/10 * * * *"
  concurrencyPolicy: Forbid
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 3
  jobTemplate:
    spec:
      template:
        spec:
          restartPolicy: Never
          containers:
            - name: drift-check
              image: drift-check:v1
              imagePullPolicy: Never
              volumeMounts:
                - name: driftdata
                  mountPath: /data
          volumes:
            - name: driftdata
              persistentVolumeClaim:
                claimName: drift-data-pvc
EOF
```

71.5 Apply

```
kubectl apply -f drift-cronjob.yaml
```

71.6 Populate PVC dengan File CSV (Penting!), Local File → kubectl cp → Helper Pod → PVC → Drift Checker

```
kubectl run pvc-helper --namespace=mlops --image=alpine --
restart=Never \

  --overrides='{
    "spec": {
      "volumes": [{"name": "driftdata", "persistentVolumeClaim":
{"claimName": "drift-data-pvc"}}],

      "containers": [{"name": "helper", "image": "alpine", "command":
["sleep", "3600"], "volumeMounts": [{"mountPath": "/data", "name":
"driftdata"}]}]
    }
  }'
```

Wait helper pod ready

```
kubectl wait --for=condition=Ready pod/pvc-helper -n mlops --
timeout=30s
```

71.7 Copy CSV ke dalam PVC

```
kubectl cp reference.csv mlops/pvc-helper:/data/reference.csv
kubectl cp current.csv mlops/pvc-helper:/data/current.csv
```

71.8 Delete helper pod

```
kubectl delete pod pvc-helper -n mlops
```

71.9 Trigger Manual First Run & Wait

```
kubectl create job --from=cronjob/drift-check initial-drift-run -n
mlops
```

```
kubectl wait --for=condition=complete job/initial-drift-run -n mlops -
-timeout=60s
```

71.10 Label ServiceMonitor Pushgateway

```
kubectl label servicemonitor pushgateway-prometheus-pushgateway -n
monitoring release=kube-prometheus --overwrite
```

71.11 Alert rule (tambahan ke alert-rules.yaml Phase 6), Parameter beserta Fungsinya: `expr` → PromQL condition; `for` → Durasi condition sebelum alert firing; `severity` → Tingkat alert; `summary` → Ringkasan; `description` → Penjelasan. Alur alert: Metric → PromQL → Condition true -selama 5 menit→ Firing. Dua jenis alert: `DataDriftDetected` → Data problem dan `DriftCheckJobFailing` → Monitoring problem. Ini penting karena Tidak ada drift metric, belum tentu berarti Tidak ada drift, Bisa saja Drift checker mati.

```
cat <<EOF > alert-rules.yaml
apiVersion: monitoring.coreos.com/v1
kind: PrometheusRule
metadata:
  name: ai-factory-alerts
  namespace: monitoring
  labels:
    release: kube-prometheus
spec:
  groups:
    - name: gpu-and-pod-health
      rules:
        - alert: GPUHighTemperature
          expr: DCGM_FI_DEV_GPU_TEMP > 85
          for: 5m
          labels:
            severity: critical
          annotations:
            summary: 'GPU {{ $labels.gpu }} di Node {{ $labels.node }}
overheat (>85°C)!'
        - alert: PodCrashLooping
          expr: rate(kube_pod_container_status_restarts_total[15m]) * 60 >
0
          for: 5m
          labels:
```

```

        severity: critical
      annotations:
        summary: 'Pod {{ $labels.pod }} di namespace {{
$labels.namespace }} bermasalah/restart!'
    - name: model-drift-alerts
      rules:
    - alert: DataDriftDetected
      expr: model_data_drift_share > 0.3
      for: 5m
      labels:
        severity: warning
      annotations:
        summary: 'Data drift terdeteksi (Proporsi kolom berubah: {{
printf "%.2f" $value }})'
    - alert: DriftCheckJobFailing
      expr: time() - model_drift_check_last_run_timestamp_seconds >
1800
      for: 5m
      labels:
        severity: critical
      annotations:
        summary: 'CronJob drift-check tidak berhasil running > 30
menit!'
EOF

```

71.12 Apply

```
kubectl apply -f alert-rules.yaml
```

71.13 Terminal 1, traffic flow: Drift Cronjob → Pushgateway → prometheus → promQL

```
pkill -f "port-forward" || true
```

```
kubectl port-forward -n monitoring svc/kube-prometheus-kube-prometheus
prometheus 9090:9090 > /dev/null 2>&1 &
```

```
sleep 3
```

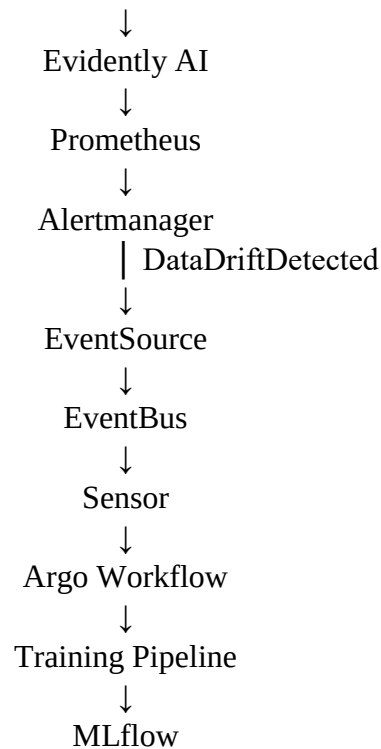
71.14 Terminal 2

```
curl "localhost:9090/api/v1/query?query=model_data_drift_share"
```

Step 72. Argo Events (Auto-Retraining dari Alert Drift)

Step ini menghubungkan hasil monitoring dengan training pipeline sehingga retraining tidak lagi harus dijalankan secara manual. Argo Events dipasang sebagai event-driven automation layer. EventSource menyediakan endpoint webhook yang menerima event dari Alertmanager, sedangkan Sensor bertugas mendeteksi event tertentu dan menjalankan trigger. EventSource menerima HTTP POST dari Alertmanager dan meneruskannya melalui EventBus. Sensor kemudian memonitor event tersebut berdasarkan dependency yang didefinisikan. Ketika event DataDriftDetected diterima, Sensor membuat dan menjalankan Workflow yang menggunakan WorkflowTemplate training yang sudah tersedia dari fase sebelumnya. Alurnya adalah:

Current Data



Webhook terlebih dahulu diuji secara manual untuk memastikan EventSource dan Sensor dapat menerima event dan menghasilkan Workflow. Setelah jalur internal berhasil, Alertmanager dikonfigurasi agar alert DataDriftDetected secara otomatis dikirim ke endpoint EventSource. Validasi akhir memastikan event benar-benar melewati seluruh chain dan menghasilkan Workflow retraining yang berjalan sampai training selesai serta mencatat hasilnya ke MLflow.

72.1 Install Argo Events, EventSource → EventBus → Sensor → Trigger

72.1.1 Bikin namespace khusus argo-events

```
kubectl create namespace argo-events || true
```

72.1.2 Install Controller Argo Events versi stabil

```
kubectl apply -n argo-events -f  
https://raw.githubusercontent.com/argoproj/argo-  
events/stable/manifests/install.yaml
```

72.1.3 Install jalur EventBus (Pakai path folder contoh resmi yang benar)

```
kubectl apply -n argo-events -f  
https://raw.githubusercontent.com/argoproj/argo-  
events/stable/examples/eventbus/native.yaml
```

72.1.4 Tunggu sampai semua 3 pod NATS EventBus siap (Ready)

```
kubectl rollout status statefulset/eventbus-default-stan -n argo-  
events
```

72.1.5 Validasi akhir pods di namespace argo-events

```
kubectl get pods -n argo-events
```

72.2 EventSource, dengarin webhook (dari Alertmanager), EventSource menyediakan HTTP endpoint POST /drift-alert yang dapat menerima event dari Alertmanager. Parameter beserta Fungsinya: port → Port EventSource; endpoint → URL path; method → HTTP method.

```
cat <<EOF > eventsource.yaml
apiVersion: argoproj.io/v1alpha1
kind: EventSource
metadata:
  name: drift-webhook
  namespace: argo-events
spec:
  service:
    ports:
      - port: 12000
        targetPort: 12000
  webhook:
    drift-alert:
      port: "12000"
      endpoint: /drift-alert
      method: POST
```

EOF

72.2.1 Apply

```
kubectl apply -f eventsource.yaml
```

72.3 Membuat SENSOR & TESTING, Sensor bertindak sebagai IF event X occurs THEN execute Y. Dalam kasus ini: IF drift-alert received THEN submit Argo Workflow. Parameter beserta Fungsinya: dependencies → Event yang harus terpenuhi eventSourceName → Sumber event; eventName → Nama event; operation: submit → Submit workflow; generateName → Prefix nama workflow; workflowTemplateRef → Workflow yang dijalankan.

```
echo "🤖 Membuat Sensor (Satpam)..."
```

```
cat <<EOF > sensor.yaml
apiVersion: argoproj.io/v1alpha1
kind: Sensor
metadata:
  name: retrain-sensor
  namespace: argo-events
spec:
  template:
    serviceAccountName: argo-events-sa
  dependencies:
    - name: drift-dep
      eventSourceName: drift-webhook
      eventName: drift-alert
  triggers:
    - template:
        name: trigger-retrain
        argoWorkflow:
          operation: submit
        source:
          resource:
            apiVersion: argoproj.io/v1alpha1
```

```

        kind: Workflow
        metadata:
          generateName: auto-retrain-
          namespace: mlops
        spec:
          workflowTemplateRef:
            name: housing-training-pipeline
EOF

```

72.3.1 Apply

```
kubectl apply -f sensor.yaml
```

72.3.2 Jalankan port-forward & uji coba webhook, Memisahkan troubleshooting menjadi: Webhook → EventSource → Sensor → Workflow sebelum Alertmanager dimasukkan ke jalur.

```
pkill -f "port-forward.*12000" || true
```

```
kubectl port-forward -n argo-events svc/drift-webhook-eventsource-svc
12000:12000 > /dev/null 2>&1 &
```

```
sleep 2
```

```
echo "📬 Mengirim simulasi Webhook POST..."
```

```
curl -X POST http://localhost:12000/drift-alert -d
'{"status":"firing","alertname":"DataDriftDetected"}'
```

```
echo -e "\n"
```

```
echo "👁️ Memantau workflow retraining otomatis di namespace mlops:"
```

```
kubectl get workflows -n mlops -w
```

72.3.3 Integrasi Alertmanager (prometheus), Jika sebuah alert masuk dengan kondisi alertname != DataDriftDetected, maka alert tersebut akan diproses melalui jalur routing standar (misalnya dikirim ke kanal notifikasi seperti Slack, PagerDuty, atau email). Sebaliknya, jika alertname == DataDriftDetected, Alertmanager akan mendispatch payload tersebut ke drift-retrain-webhook, yang kemudian bertindak sebagai pemicu (trigger) masuk ke dalam EventSource (seperti eventbus pada Argo Events atau sistem orkestrasi MLOps) untuk memulai proses otomatisasi model retraining.

```

cat <<EOF | kubectl apply -f -
apiVersion: monitoring.coreos.com/v1alpha1
kind: AlertmanagerConfig
metadata:
  name: drift-alert-config
  namespace: monitoring
spec:
  route:
    receiver: drift-retrain-webhook
    routes:
    - matchers:
      - name: alertname
        value: DataDriftDetected
    receivers:
    - name: drift-retrain-webhook

```



```

webhookConfigs:
- url: http://drift-webhook-eventsource-svc.argo-
events.svc.cluster.local:12000/drift-alert
EOF

# 72.3.4 Cek apakah AlertmanagerConfig udah sah terdaftar di namespace
monitoring

kubectl get alertmanagerconfig -n monitoring

# 72.3.5 Cek log alertmanager buat mastiin konfigurasinya sukses
dimuat (gak ada error syntax)

kubectl logs -n monitoring alertmanager-kube-prometheus-kube-promete-
alertmanager-0 -c alertmanager --tail=30

# 72.3.6 Uji coba nembak langsung ke webhook EventSource dari dalam
klaster (opsional buat pastikan jalurnya nyambung), di test dari dalam
cluster Karena ini menguji: Kubernetes DNS + Service discovery +
Network connectivity + EventSource endpoint tanpa bergantung pada
port-forward.

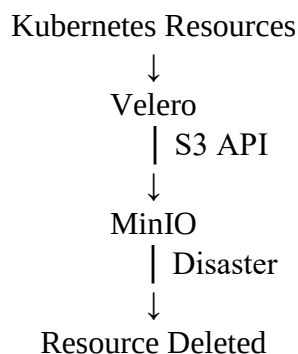
kubectl run curl-test --rm -it --image=curlimages/curl --restart=Never
-n argo-events -- \

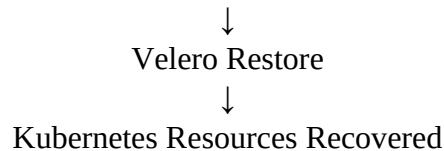
    curl -X POST http://drift-webhook-eventsource-svc.argo-
events.svc.cluster.local:12000/drift-alert \

    -d '{"status":"firing","alertname":"DataDriftDetected"}'
```

Step 73. Disaster Recovery dengan Velero

Step ini membangun mekanisme backup dan restore resource Kubernetes menggunakan Velero dengan MinIO sebagai backend object storage. Credential MinIO disiapkan secara terpisah dari repository Git, kemudian Velero dikonfigurasi menggunakan AWS-compatible plugin agar dapat berkomunikasi dengan MinIO melalui S3 API. Backup location diarahkan secara eksplisit ke endpoint MinIO dan volume snapshot dimatikan karena environment menggunakan storage lokal yang tidak menyediakan cloud volume snapshot. Setelah Velero aktif, dibuat workload sederhana pada namespace team-ds sebagai resource uji. Namespace tersebut kemudian di-backup ke MinIO. Untuk membuktikan bahwa backup benar-benar usable, namespace sengaja dihapus sebagai simulasi disaster dan kemudian direstore menggunakan backup yang telah dibuat. Alurnya adalah:





Validasi tidak berhenti pada keberhasilan proses backup. Backup harus memiliki status valid, restore harus berhasil, namespace harus kembali, dan workload di dalam namespace harus dapat dijalankan kembali.

73.1 Buat file kredensial untuk MinIO, File ini digunakan oleh Velero plugin untuk melakukan autentikasi ke object storage. Dari sisi Security, Jangan commit credentials-velero ke Git.

```
cat > credentials-velero << 'EOF'

[default]

aws_access_key_id=minioadmin

aws_secret_access_key=minioadmin123

EOF
```

73.2 Download dan install Velero CLI versi 1.14.0

```
curl -fsSL -o velero-linux-amd64.tar.gz https://github.com/vmware-tanzu/velero/releases/download/v1.14.0/velero-v1.14.0-linux-amd64.tar.gz

tar -xzf velero-linux-amd64.tar.gz

sudo mv velero-v1.14.0-linux-amd64/velero /usr/local/bin/velero

rm -rf velero-linux-amd64.tar.gz velero-v1.14.0-linux-amd64
```

73.3 Setup MinIO Bucket

```
echo "□ Bikin bucket 'velero-backup' di MinIO..."

kubectl exec -n mlops deployment/minio -- mc alias set local http://localhost:9000 minioadmin minioadmin123 2>/dev/null || true

kubectl exec -n mlops deployment/minio -- mc mb local/velero-backup || true
```

73.4 Install Velero to Kubernetes, Parameter beserta Fungsinya: --provider aws → Menggunakan AWS S3-compatible provider; --plugins → Plugin MinIO/S3; --bucket → Bucket backup; --secret-file → Credential; region=minio → Logical region; s3ForcePathStyle=true → S3 path-style compatibility; s3Url → Endpoint MinIO; --use-volume-snapshots=false → Tidak menggunakan volume snapshot. pake provider AWS karena MinIO kompatibel dengan S3 API. Jadi urutannya Velero AWS Plugin → S3 API → MinIO, tidak berarti backup dikirim ke AWS.

```
echo "🚀 Menginstal Velero ke cluster..."

velero install \

  --provider aws \

  --plugins velero/velero-plugin-for-aws:v1.10.0 \
```

```

--bucket velero-backup \

--secret-file ./credentials-velero \

--backup-location-config
region=minio,s3ForcePathStyle=true,s3Url=http://minio.mlops.svc.cluste
r.local:9000 \

--use-volume-snapshots=false

# Tunggu sampai pod Velero siap

echo "🕒 Menunggu Velero Pods siap..."

kubectl rollout status deployment/velero -n velero --timeout=90s

# 73.5 Simulasi Backup di namespace team-ds (pastikan namespace sudah ada)

kubectl get ns

# 73.5.1 Isi namespace team-ds dengan pod Nginx

kubectl run test-app --image=nginx -n team-ds || true

# 73.5.2 Pastikan pod benar-benar siap (Ready/Running)

kubectl wait --for=condition=Ready pod/test-app -n team-ds --timeout=60s

# 73.5.3 Eksekusi Backup

echo "📦 Membuat backup 'tes-backup-ds'..."

velero backup create tes-backup-ds --include-namespaces team-ds --wait

# 73.5.4 Validasi apakah sudah terbackup

velero backup describe tes-backup-ds --details

# 73.5.5 SIMULASI BENCANA: Hapus namespace team-ds

echo "💣 Menghapus namespace team-ds..."

kubectl delete namespace team-ds

# 73.5.6 RESTORE: Kembalikan dari Velero

echo "🔄 Memulihkan namespace team-ds..."

velero restore create --from-backup tes-backup-ds --wait

# 73.5.7 VALIDASI AKHIR: Cek apakah team-ds & isinya balik utuh

echo "✅ Validasi Restore:"

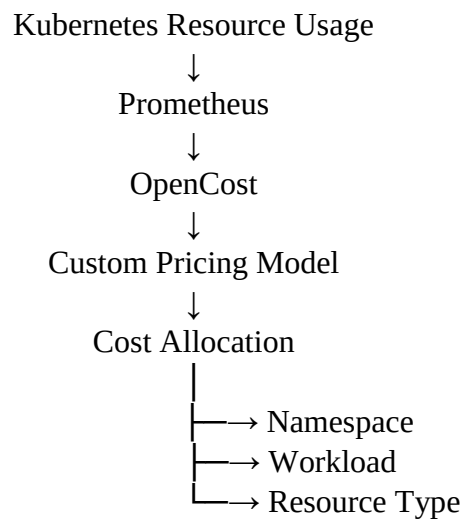
kubectl get ns team-ds

kubectl get pods -n team-ds

```

Step 74. Cost Visibility dengan OpenCost

Step ini menambahkan resource cost visibility ke platform dengan OpenCost. Resource usage yang sebelumnya hanya digunakan untuk scheduling, quota, dan monitoring sekarang diterjemahkan menjadi estimasi biaya berdasarkan konfigurasi pricing yang ditentukan. OpenCost dikonfigurasi agar mengambil data resource dari Prometheus. Custom pricing digunakan karena environment ini merupakan lab lokal, sehingga angka CPU, RAM, GPU, storage, dan network bukan merupakan tagihan cloud aktual tetapi model harga simulasi. Konfigurasi OpenCost juga menentukan clusterId, ServiceAccount, RBAC, resource request/limit, endpoint Prometheus, serta ServiceMonitor agar metric OpenCost dapat diintegrasikan kembali ke monitoring stack. Alurnya adalah:



OpenCost kemudian diekspos melalui API dan hasil allocation digunakan untuk melihat estimasi penggunaan berdasarkan namespace seperti team-ds, team-mle, team-pe, team-intern, dan namespace platform lainnya. Validasi memastikan OpenCost dapat membaca metric resource, terhubung ke Prometheus, menerapkan custom pricing, dan menghasilkan nilai totalCost serta breakdown cost yang tidak kosong..

74.1 Konfigurasi OpenCost, definisi custom pricing: Environment lab tidak mempunyai invoice cloud provider, Maka digunakan synthetic pricing model. Contoh konsep: CPU → cost / core-hour; RAM → cost / GB-hour; GPU → cost / GPU-hour; Storage → cost / GB-month. Angka tersebut adalah estimasi simulasi, bukan harga aktual cloud provider. Parameter beserta fungsinya: defaultClusterId → Identitas cluster; requests.cpu → Resource OpenCost; requests.memory → Resource OpenCost; customPricing → Mengaktifkan pricing model; CPU → Harga CPU; RAM → Harga memory; GPU → Harga GPU; storage → Harga storage.

```
cat <<EOF opencost-values.yaml
opencost:
  # Wajib enable ServiceAccount dan RBAC agar tidak terkena
  Unauthorized
  serviceAccount:
    create: true
    name: opencost
  rbac:
    create: true
  exporter:
```

```

    defaultClusterId: "ai-factory-lab"
    resources:
      requests:
        cpu: 50m
        memory: 128Mi
      limits:
        cpu: 200m
        memory: 256Mi
    prometheus:
      internal:
        enabled: false
      external:
        enabled: true
        url: "http://prometheus-
operated.monitoring.svc.cluster.local:9090"
    customPricing:
      enabled: true
      provider: "custom"
    costModel:
      description: "Estimasi harga lab lokal"
      CPU: "0.031611"
      RAM: "0.004237"
      GPU: "0.95"
      storage: "0.00005479"
      zoneNetworkEgress: "0.01"
      regionNetworkEgress: "0.01"
      internetNetworkEgress: "0.12"
    serviceMonitor:
      enabled: true
      additionalLabels:
        release: kube-prometheus-stack
EOF

```

74.2 memberikan izin view ke ServiceAccount milik OpenCost agar dia bisa membaca data cluster. OpenCost membutuhkan permission untuk membaca resource Kubernetes. Secara sederhana: OpenCost ServiceAccount → ClusterRole: view → Kubernetes API. Tanpa RBAC yang cukup: OpenCost → Kubernetes API → Forbidden

```

kubectl create clusterrolebinding opencost-cluster-view \
  --clusterrole=view \
  --serviceaccount=opencost:opencost

```

74.3 Download Helm Chart OpenCost

```

helm repo add opencost https://opencost.github.io/opencost-helm-chart
helm repo update

```

74.3 Buat Namespace dan Install menggunakan values yang sudah diperbaiki

```

kubectl create namespace opencost --dry-run=client -o yaml | kubectl
apply -f -

helm install opencost opencost/opencost -n opencost -f opencost-
values.yaml

```

74.4 Validasi, Lolos jika pod opencost-* berstatus Running.

```
kubectl get pods -n opencost
```

74.5 Expose service OpenCost ke localhost:

```
kubectl port-forward -n opencost svc/opencost 9999:9003
```

74.6 Terminal 2

```
kubectl port-forward -n monitoring svc/kube-prometheus-grafana 3000:80
```

74.7 Validasi via API, Lolos jika response JSON menampilkan breakdown biaya per namespace (team-mle, team-ds, mlops, kube-system) dan totalCost bernilai angka (bukan array kosong).

```
curl
```

```
"http://localhost:9999/allocation/compute?window=1d&aggregate=namespace"
```

End-to-End Platform Validation

User melakukan validasi end-to-end terhadap AI Factory untuk memastikan workload Kubernetes dapat dijalankan dan resource consumption-nya dapat dimonitor serta dihitung oleh OpenCost pada level namespace.

Step 75. Deploy E2E Test Workload

75.1 Buat workload sederhana pada namespace team-ds.

```
kubectl apply -f - <<EOF
apiVersion: v1
kind: Pod
metadata:
  name: e2e-test-workload
  namespace: team-ds
  labels:
    app: ai-factory-e2e
spec:
  restartPolicy: Never
  containers:
  - name: model-runner
    image: python:3.9-slim
    command: ["python", "-c", "import time; print('E2E Job Started');
time.sleep(120); print('E2E Job Finished')"]
    resources:
      requests:
        cpu: "200m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
EOF
```

Step 76. Validate Workload

```
kubectl get pod e2e-test-workload -n team-ds
```

Step 77. Expose OpenCost API

77.1 Buka terminal baru dan lakukan port-forward terhadap OpenCost.

```
kubectl port-forward -n opencost svc/opencost 9999:9003
```

Step 78. Query Resource Allocation

78.1 Pada terminal kedua, query allocation untuk namespace team-ds. Response harus memiliki daftar cost

```
curl
"http://localhost:9999/allocation/compute?window=10m&aggregate=namespa
ce&filter=namespace:\"team-ds\""
```

THANK YOU